

Welcome to Intuit Backend Bootcamp

 **Develop**Intelligence

A PLURALSIGHT COMPANY



Hello

About me...

-

We teach over 400 technology topics

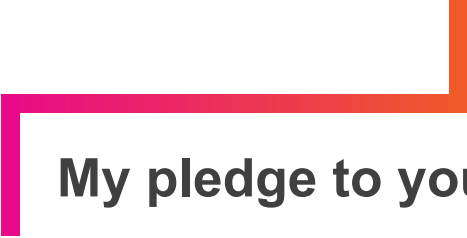


We teach over 400 technology topics



You experience our impact on a daily basis!





My pledge to you

I will...

- Make this interactive
- Ask you questions
- Ensure everyone can speak
- Use an on-screen timer



Prerequisites

This course assumes you:

- Have some background in software development

Class Logistics



8 hours



1 hour lunch + breaks
every hour



Course Agenda

- Week 1:
 - Java Fundamentals + exercises, OOP with java
 - Spring Fundamentals (what is IoC/DI) + Spring boot + JPA
 - Code smells + refactoring
 - Docker/K8s overview (as time permits, else week 2)



Course Agenda

- Week 2:
 - Hexagonal/Ports+Adapter Architecture
 - Kotlin deep dive + exercises
 - Docker/K8s continued
 - ArgoCD
 - Performance Tuning in Java (GC, memory management in Java, JVM tuning)



Introductions



Student Introductions

Let's get a sense for class knowledge and experience, I will go through the participants list

- Role at Intuit
- Any experience with Java/Kotlin?
- Fun fact about yourself

Java basics

Java Basics

Let's go through the Java workbooks to level-set and go over some basics of Java - your instructor should give you access

Java Fundamentals:

https://docs.google.com/document/d/1Jgt26qJq0qGc92GG_TJsTf--CnYhnhqR/edit

Java OOP:

https://docs.google.com/document/d/1AF_sjNOyk2OBdLP4VU_7ZHLVFav-xsTd/edit

Spring Workbooks

 **Develop**Intelligence

A PLURALSIGHT COMPANY

Spring Basics

Let's go through some Spring workbooks to go over some basics of Spring - your instructor should give you access:

Maven, Spring/boot:

<https://docs.google.com/document/d/1eOOO1Aqww-qchAAc0wqPszfKj1mBdqol/edit#heading=h.gjdgxs>

Springboot web/data:

<https://docs.google.com/document/d/1ObgekdQAFBp1bVpr27xaloorPKLb1UVW/edit>

Code Smells

 **Develop**Intelligence

A PLURALSIGHT COMPANY

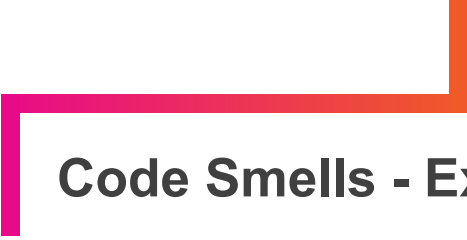


Code Smells

We are going to learn about code smells by practicing with a sample app here:

<https://github.com/varoonsahgal/banking-app-java.git>

Clone it down and follow instructions in the README



Code Smells - Exercise 1

The first exercise you should do is here:

<https://github.com/varoonsahgal/banking-app-java/blob/main/exercises/Exercise1-LongMethods.md>

The instructor will introduce the exercise and walk through it when you are done. Take about 25 minutes for this one

Code Smells - Exercise 2

The second exercise you should do is here:

<https://github.com/varoonsahgal/banking-app-java/blob/main/exercises/Exercise2-PrimitiveObsession.md>

The instructor will introduce the exercise and walk through it when you are done. Take about 5 minutes for this one

Code Smells - Exercise 3

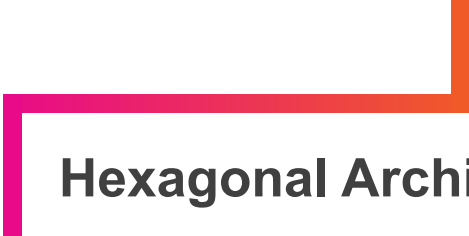
The third exercise you should do is here:

<https://github.com/varoonsahgal/banking-app-java/blob/main/exercises/Exercise3-FixPrimitiveObsession.md>

The instructor will introduce the exercise and walk through it when you are done. Take about 50 minutes for this one



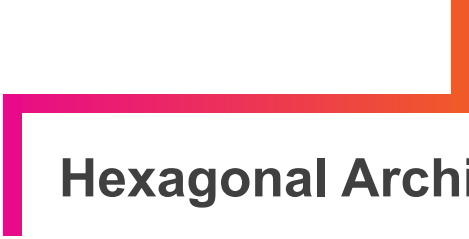
Hexagonal Architecture



Hexagonal Architecture

To introduce Hexagonal architecture, you will run through this exercise, take about 45 minutes:

<https://github.com/varoonsahgal/banking-app-java/blob/main/exercises/Exercise4-HexagonalArchIntro.md>

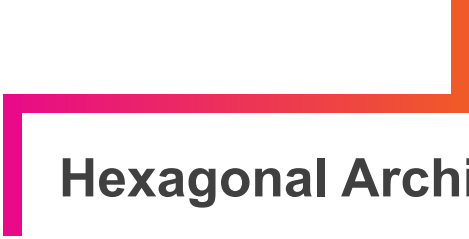


Hexagonal Architecture exercise 5 and 6

Now, let's integrate Springboot and later JPA into our application. Perform this exercise:

<https://github.com/varoonsahgal/banking-app-java/blob/main/exercises/Exercise5-SpringWebAdapter.md>

Take about 1 hour for this one as its complex!



Hexagonal Architecture exercise 5 and 6

Now, let's integrate persistence via JPA into our application.
Perform this exercise:

<https://github.com/varoonsahgal/banking-app-java/blob/main/exercises/Exercise6-PersistenceJPA.md>

Take about 1 hour for this one as well as its complex!



INSTRUCTOR SLIDE ONLY

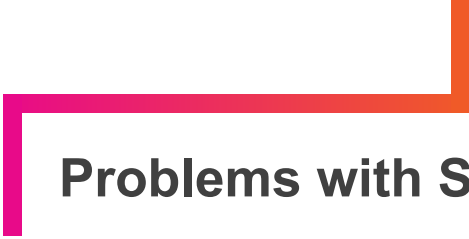
INSTRUCTOR USE ONLY - DON'T SHARE WITH STUDENTS UNTIL AFTER LIVE WALKTHROUGH

Solutions to the Banking app exercises here - see the different commits:

<https://github.com/varoonsahgal/banking-app-solutions/commits/main/>

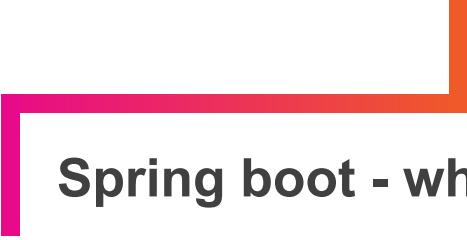


Spring Framework



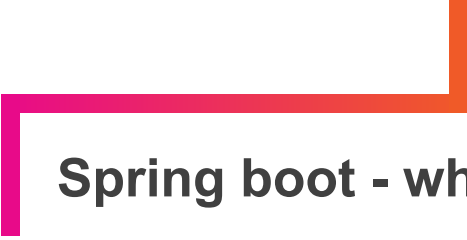
Problems with Spring Framework

- Lots of boilerplate code is necessary
- Lots of configuration - many configuration steps
- Many steps to build and deploy
 - To see output of an app you will need to get a war/jar and deploy it to the local server in Spring Framework



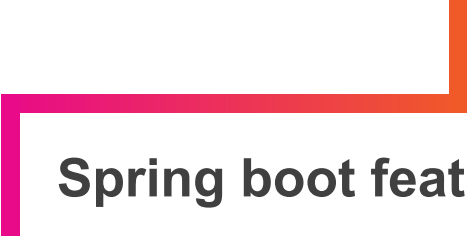
Spring boot - what and why?

- As developers ideally we want to only focus on business logic - not on setup/config
- Spring boot: let's not focus on config/setup/deployment anymore!
- Spring boot started with a ticket
- Spring boot is a module of a Spring framework that allows us to build production ready apps quickly



Spring boot - what and why?

- Spring boot == spring framework + prebuilt configuration + embedded servers
- With Spring Framework we do not get the prebuilt config
- Spring boot comes with an embedded server



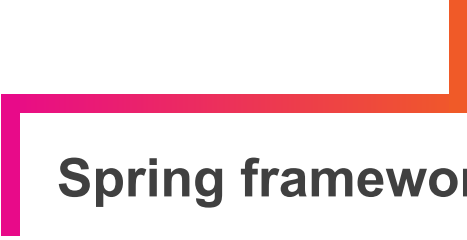
Spring boot features

- You can create a stand alone app with minimum development time
- Provides a spring starter project that helps you get started quickly
- Auto configuration of dependencies
 - Example: if you want to use mysql with Spring boot, you can specify mysql, sql server, etc. as dependencies and then spring boot will auto-configure it



Why do developers love Spring Boot?

- Spring boot has an embedded server
 - So, there's no need to deploy to a server rather you just hit "run" and it will run your application for you on the embedded server
- Reduces cost and development time of the application
- Makes it easier to integrate with other frameworks like hibernate, struts, etc..



Spring framework features

- Spring framework (not boot) is a java based web framework for building web apps
- Main feature of Spring is **dependency injection**
- Goal of dependency injection: helps you *develop loosely coupled* applications

Springboot architecture

 **Develop**Intelligence

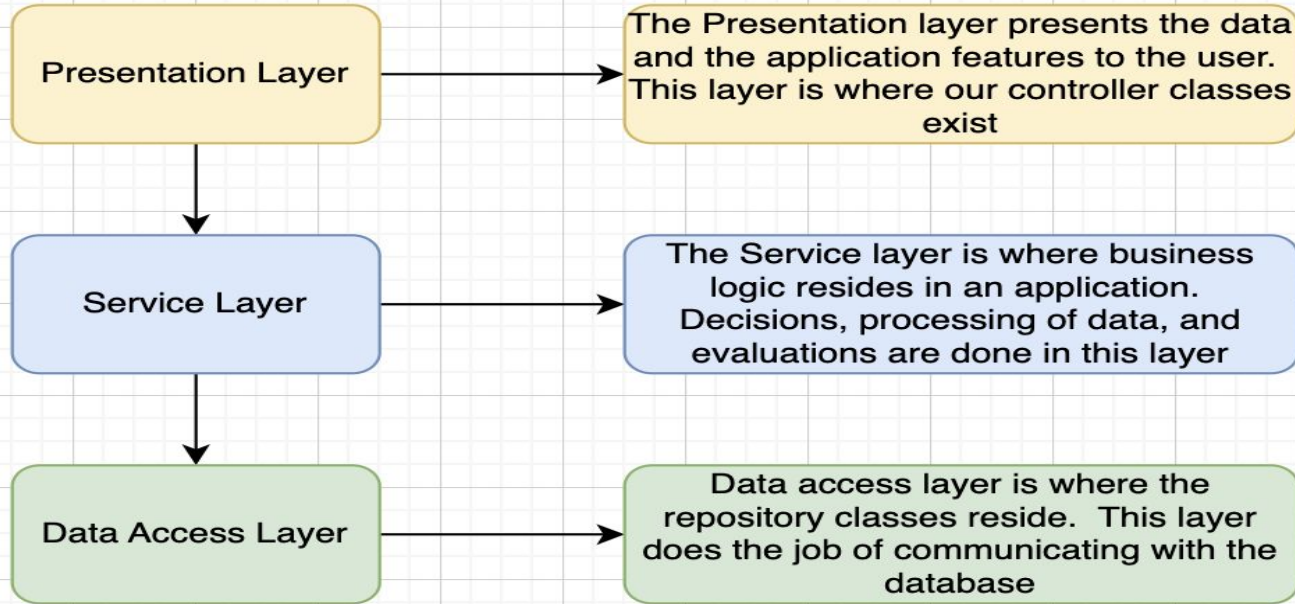
A PLURALSIGHT COMPANY



Springboot layers

- Spring Boot (not Spring) has a layered architecture where one layer communicates with the other layer
- These are the different layers:
 - Presentation layer
 - Service layer
 - Data access layer

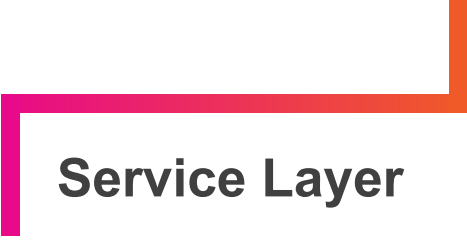
Springboot layers





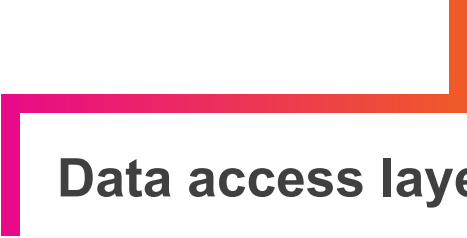
Presentation Layer

- Presentation layer - presents data and application features to the user
- This layer is where all controller classes exist
- This layer: accepts request from user
 - Validates input from user
 - And passes it on to the next layer which is the ***service*** layer



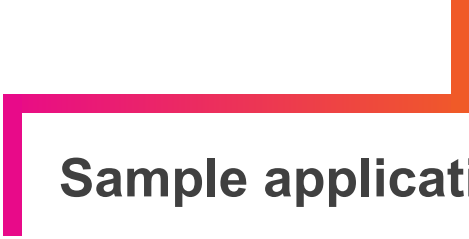
Service Layer

- Where business logic resides in the application
- Evaluations/decision making/processing of data is done at this layer
- Service layer will interact with BOTH the presentation layer and data access layer



Data access layer

- Does the job of talking to the database and saving/retrieving data...
- Where all repository classes reside
- Underlying databases that you use with springboot can be relational or non-relational
 - But the job of communicating with the database is done at the data access layer...

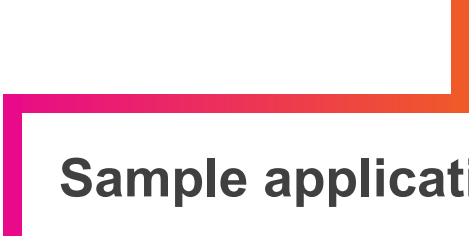


Sample application - Library management system

- User accesses app via the browser
- then request goes to the controller
- Controller passes request to the service
- Then service passes request to the repository
- Controller->Service->Repository
- Suppose user wants to get a list of books
-

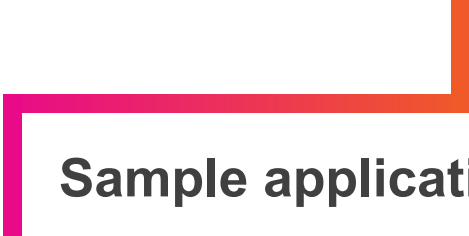
Sample application - Library management system

- Suppose user wants to get a list of books
- Like a GET request to /booklist
- Once the controller is sure that the method being requested exists and validates the request, it will then pass it on to the next step - the service layer
- Then service layer will process the request with the help of the repository
- Repository will interact with the database and retrieve list of books
- Then Repository passes list of books to Service which then passes it on to the Controller.. and finally list of results displayed on web browser



Sample application - Library management system

- This is a typical Spring Boot flow - how a request flows through different layers...



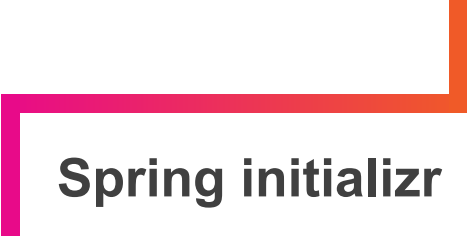
Sample application - Library management system

- This is a typical Spring Boot flow - how a request flows through different layers...

Spring initializr

 **Develop**Intelligence

A PLURALSIGHT COMPANY



Spring initializr

- Group is the name of the package
- Artifact is the name of application
- The package name is then auto-generated, you can change it but we will leave it
- Select maven, Java, and 3.1.5 for Spring Boot version
- Keep JAR selected, select Java 11
- Top right we see “Add dependency” - if you want the application to talk to MySQL, you can add the dependency
-

Spring initializr

- We will add 2 dependencies:
 - Spring Web and Spring Data JPA
- Just generate, and then do File->Open Project in IntelliJ and open the project
- IJ will do some indexing and you can open the project and see theres a test folder with some sample tests here as well...
- You will also see the pom.xml that will define the dependencies your project will need
 - You can see both JPA and web dependencies...
 - And if you want to add more dependencies you can

Pom.xml

- You can see a parent in the pom.xml - what is that?
- spring-boot-starter-parent is a special starter project that provides the default configurations for your project

```
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xs
<modelVersion>4.0.0</modelVersion>
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>3.1.5</version>
  <relativePath/> <!-- lookup parent from repository -->
</parent>
<groupId>com.example</groupId>
```

The main Java file

- If you go to the file FirstSpringApplication, we can see that there's an annotation up top
- The annotation tells us it's a Spring Boot application

```
@SpringBootApplication
public class FirstSpringApplication {

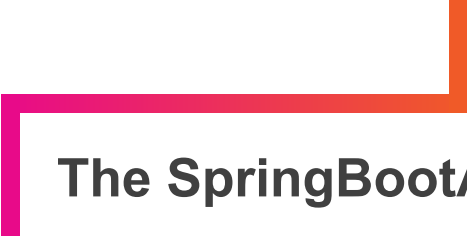
    public static void main(String[] args) { SpringApplication.run(FirstSpring
}
|
```

The main Java file

- If you click on that annotation and on a Mac click on COMMAND + click, and you will be taken to an interface with the annotations defined
 - On Windows, you can click CONTROL

```
@SpringBootApplication
public class FirstSpringApplication {

    public static void main(String[] args) { SpringApplication.run(FirstSpring
}
}
```



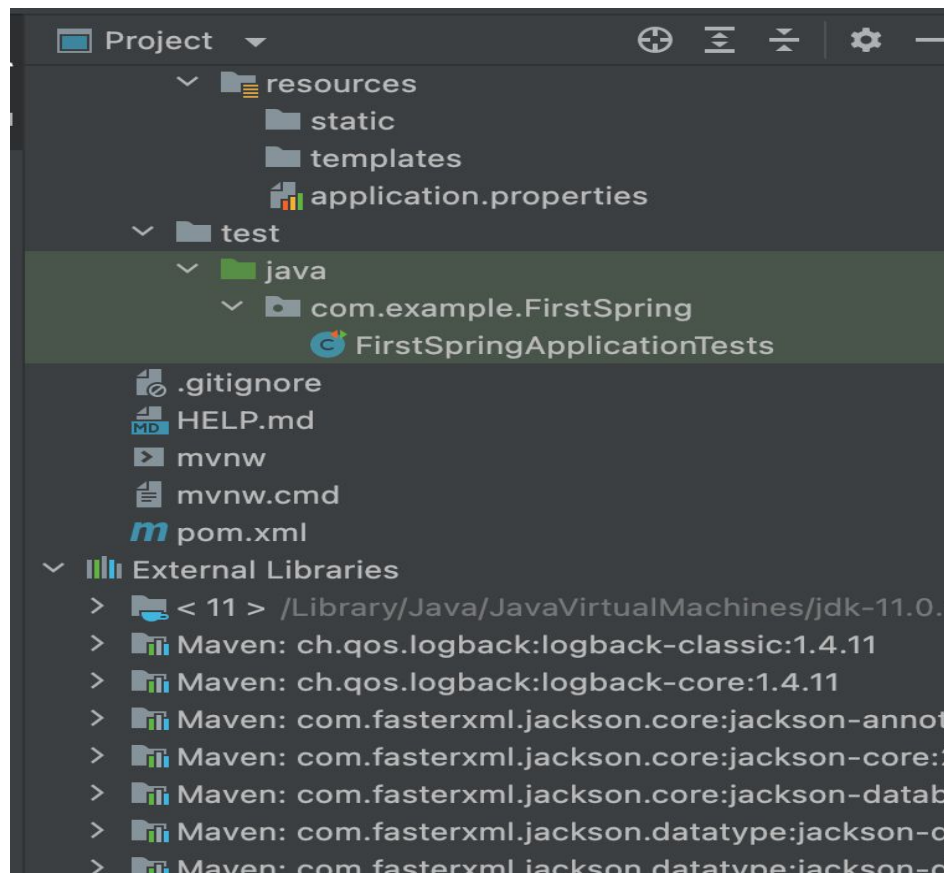
The SpringBootApplication Interface

- SpringBootConfiguration is an annotation that tells Spring Boot that this is a spring boot config file
- EnableAutoConfiguration - enables auto-config, a feature where you do not need to enable anything, it's enabled for you
 - If we go inside it, we can see we can exclude classes as well if we want to

The SpringBootApplication Interface

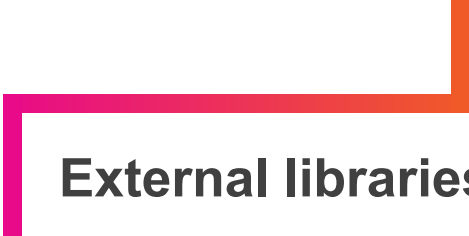
- SpringBootConfiguration is an annotation that tells Spring Boot that this is a spring boot config file
- EnableAutoConfiguration - enables auto-config, a feature where you do not need to enable anything, it's enabled for you
 - If we go inside it, we can see we can exclude classes as well if we want to
- We can also see an external libraries folder at the bottom too...
 - Here we can see our spring boot files
 - We can also see a file for Tomcat
 - If we are using purely Spring (NOT Springboot), you would need to first generate a JAR or a WAR file and deploy that file onto your server that you've set up

External libraries folder



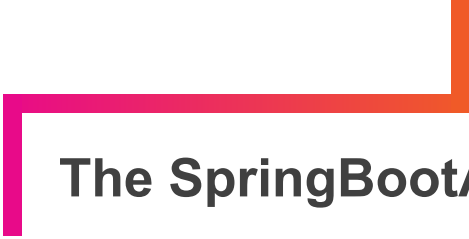
External libraries folder

- We can also see an external libraries folder at the bottom too...
 - Here we can see our spring boot files
 - We can also see a file for Tomcat
 - If we are using purely Spring (NOT Springboot), you would need to first generate a JAR or a WAR file and deploy that file onto your server that you've set up
 - But, with Spring**boot**, it comes with a server, if you just press the run button it will run the project for you!!
 - It has an embedded Tomcat server
 - There is a Mockito testing framework and JUnit libraries as well



External libraries folder

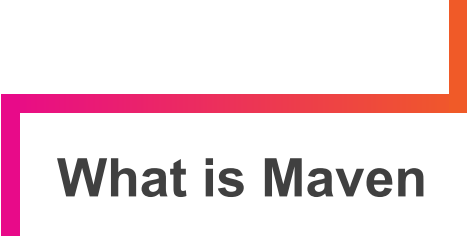
- So, if we add any dependencies to the pom.xml that would update the External Libraries folder as well!



The SpringBootApplication Interface

- SpringBootConfiguration is an annotation that tells Spring Boot that this is a spring boot config file
- EnableAutoConfiguration - enables auto-config, a feature where you do not need to enable anything, it's enabled for you
 - If we go inside it, we can see we can exclude classes as well if we want to

Maven



What is Maven

- Build automation/dependency management
- Uses pom.xml - list of all dependencies
- Maven lets you declare all dependencies in a single file and adds them to the classpath of your project
- Maven has its own repository which is hosted on a server
- If you go to <https://search.maven.org/?eh=>
- Then search for “spring boot devtools”

What is Maven

- If you go to <https://search.maven.org/?eh=>
- Then search for “spring boot devtools”, you can click on it, then click on the latest version, and then you will see a page that looks like this:

What is Maven

 **sonatype** | [Maven Central Repository Search](#) [Quick Stats](#)

 [GitHub](#)

org.springframework.boot:spring-boot-devtools: 3.1.5

[View on OSS Index](#) [Browse](#) [Downloads](#)

spring-boot-devtools
Spring Boot Developer Tools

Licenses [Apache License, Version 2.0](#)

Home page <https://spring.io/projects/spring-boot>

Source code <https://github.com/spring-projects/spring-boot>

Organization [VMware, Inc.](#)

Developers [Spring](#) [ask@spring.io](#), [VMware, Inc.](#)

 **Apache Maven**
maven.apache.org

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-devtools</artifactId>
  <version>3.1.5</version>
</dependency>
```

 **Gradle Groovy DSL**
gradle.org

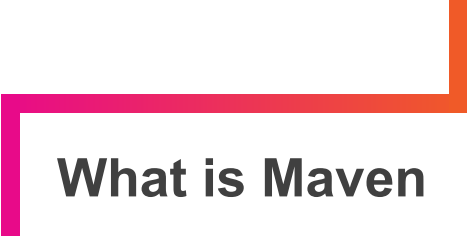
```
implementation 'org.springframework.boot:sp
```

 **Gradle Kotlin DSL**
github.com/gradle/kotlin-dsl

```
implementation ("org.springframework.boot:sp
```

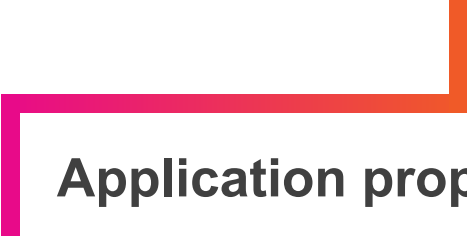
 **org.springframework.boot:spring-boot-devtools**
3.1.5

```
<?xml version="1.0" encoding="UTF-8"?>
<project xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
  <!-- This module was also published with a richer model, Gradle metadata, -->
  <!-- which should be used instead. Do not delete the following line which -->
  <!-- is to indicate to Gradle or any Gradle module metadata file consumer -->
  <!-- that they should prefer consuming it instead. -->
  <!-- do_not_remove: published-with-gradle-metadata -->
  <modelVersion>4.0.0</modelVersion>
  <groupId>org.springframework.boot</groupId>
```



What is Maven

- Grab the code from where it says Maven and paste it into your pom.xml and then click on the icon that shows up
- It will then download the dependency from the central repository and you should see it under External Libraries now!
-

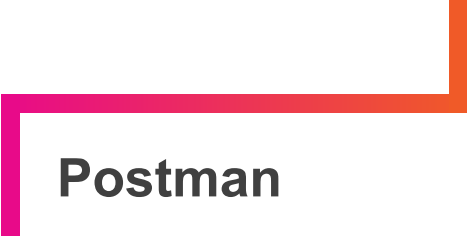


Application properties file

- If you go to the resources folder you will see an empty application.properties file
- What can we use this file for?
- Well, let's say for example we want to change the app from running on localhost:8080 to some other port like localhost:8081, we could make that configuration here
 - Maybe you have some other app running on 8080 and that's why you need to change it...

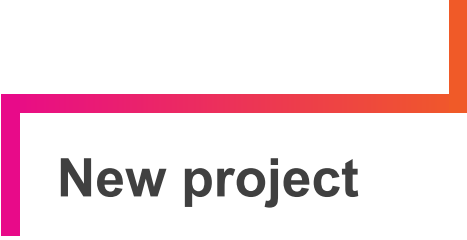
Application properties file

- Here you can see a list of properties that you can specify in your Spring Boot project:
<https://docs.spring.io/spring-boot/docs/current/reference/html/application-properties.html>
- For instance, we could set “server.port=8081” in that file as well



Postman

- Go ahead and download Postman from here:
<https://www.postman.com/downloads/>
-



New project

- Let's build an api
- go to spring initializer
- for group, use com.employee and for artifact use EmployeeApplication
- Go with Maven 3.1.5, Spring Boot version 2.7.17
- And add Spring Web as a dependency, then hit generate

New project

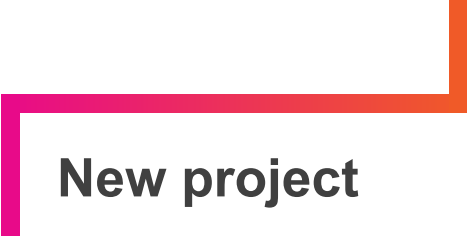
- Download the app and run it in IntelliJ, you should see the page below
- This is because we have not mapped this request to a controller yet - we have not handled the request at this port

Whitelabel Error Page

This application has no explicit mapping for /error, so you are seeing this as a fallback.

Tue Nov 21 10:38:49 CST 2023

There was an unexpected error (type=Not Found, status=404).



New project

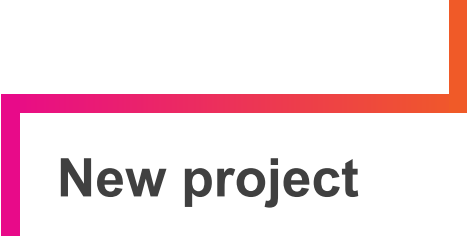
- What we want is to handle a request to `localhost:8080/employees`

Whitelabel Error Page

This application has no explicit mapping for `/error`, so you are seeing this as a fallback.

Tue Nov 21 10:38:49 CST 2023

There was an unexpected error (type=Not Found, status=404).



New project

- Looking back at the layout of a Spring Boot project

New project

- What we want is to handle a request to localhost:8080/employees - how do we do this???
- First, recall the layers in Spring boot...(next slide)

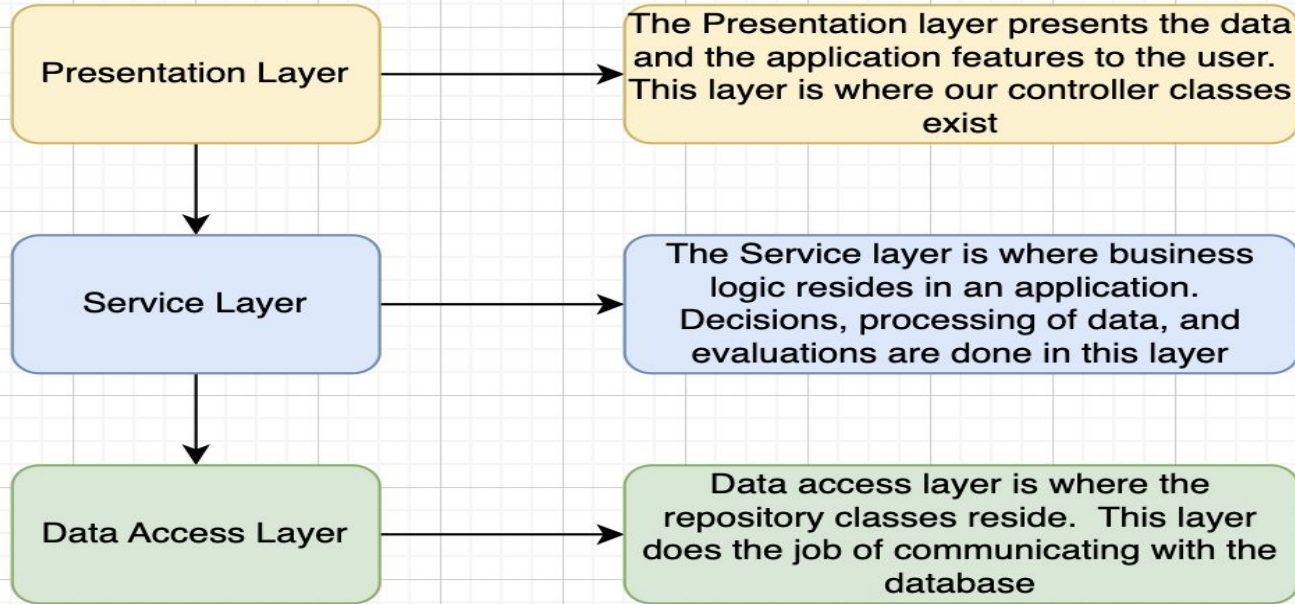
Whitelabel Error Page

This application has no explicit mapping for /error, so you are seeing this as a fallback.

Tue Nov 21 10:38:49 CST 2023

There was an unexpected error (type=Not Found, status=404).

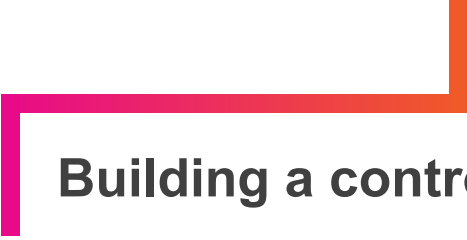
Springboot layers





What is a controller?

- Controller is part of presentation layer, and to handle the request that we are sending to the browser, we need to create a controller
- A controller is a Java class with an annotation
 - It knows what URL is going to trigger it
 - And what should happen when the URL is triggered
 - That is what a controller does!
 - Easiest to see it in action so lets go!



Building a controller...

- Let's create a package named controller in `com.Employee.EmployeeApp`
- Then, create a new Java file named `EmployeeController`
- How do we “mark” this class as a controller?
- We need this controller class to be able to handle a request
- And our Spring Boot application needs to know that this class is a Controller

Building a controller...

- For this, we have an annotation called “@Controller”, which you can add above the class name
- Add this code to the EmployeeController class
 - This code will respond when someone visits localhost:8080:

```
@Controller
public class EmployeeController {

    public List<Employee> employeeList;

    public List<Employee> findAllEmployees() {
        return employeeList;
    }
}
```

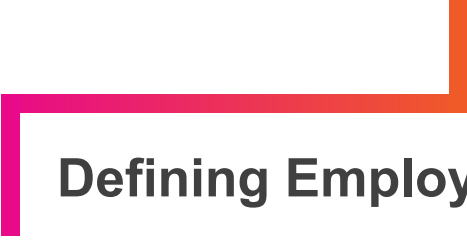
Defining Employee

- We have a list but no Employee class, let's define that - it's an entity so we will create an entity package under EmployeeApp
- Create the class in Entity with this code:

```
package com.employee.EmployeeApp.entity ;
```

```
public class Employee {  
    int employeeId;  
    String employeeName;  
    String employeeCity;  
}
```

- Then create a new class under that called Employee - you can do that with COMMAND + N -> constructor -> select all fields in IntelliJ on the Mac



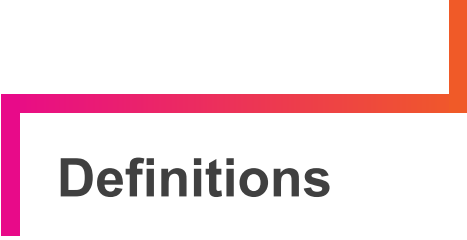
Defining Employee

- Be sure to import the required classes in the controller class
- Now we have an Employee entity class and a controller class that handles the incoming requests...
 - and will return a list of all the employees

Cloud Native: microservices

 **Develop**Intelligence

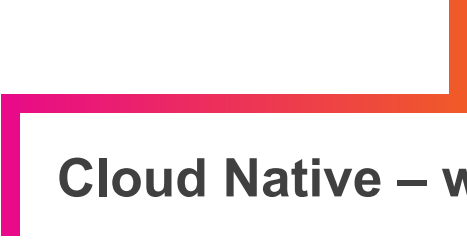
A PLURALSIGHT COMPANY



Definitions

Let's start with 2 definitions:

1. cloud native
2. microservices



Cloud Native – what does that mean?

What does the word “native” mean to you in a technology context?

Type your answers in chat...

Cloud Native – what does that mean?

- Story time: when the cloud started to become popular in the mid 2000s (2005 or so) everyone started to throw their applications on the cloud using the lift and shift strategy - hoping to get the cost/performance benefits the cloud promised
 - Key takeaway - lift and shift == migrating to cloud w/ minimal changes to the application before the migration
- But they found they were often getting **higher costs and worse performance**, so they soon realized that in order to get the benefits of running on the cloud they had to **first re-architect their applications to run effectively on the cloud**
- And hence the term **cloud native** was born - basically it means that if your app is cloud native then it is specifically architected to run on the cloud, and you can therefore get the benefits of **lower costs/better performance** that the cloud promises

Cloud Native – what does that mean?

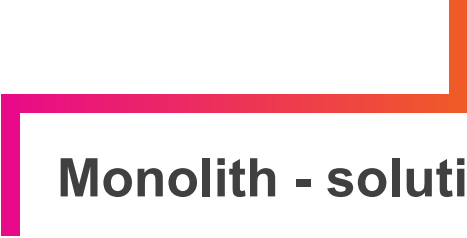
- Long story short: cloud native == your application has been ***designed/architected to run on the cloud effectively...***
- “Native” used in other contexts like mobile app development
 - - ie, if we develop a mobile app maybe we only develop it for iOS and not Android, in which case we would say it’s ”native” to the iOS operating system..
- Basically ***native*** means its targets a specific platform
- Cloud native is an attribute of an application - if your application is cloud native it can then run effectively on the cloud!

Microservices + Monolith - definition

- To best define microservices, I think it helps to first compare it to the term ***monolith***
- A monolithic application is basically one gigantic application sharing a single codebase
- *What challenges would we face if for example, we were running all of Amazon.com in a single codebase - a single Github repository perhaps?*
 - ***Type your answers in chat!***

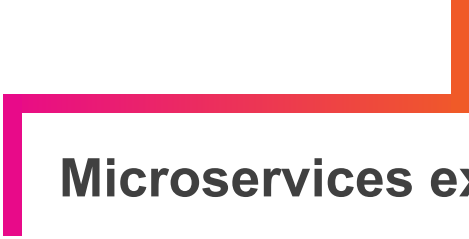
Monolith - disadvantages...

- **Deployment** - If we wanted to make any change - even a very small change - to the application we would have to redeploy the entire monolith
- **Shared libraries** - Also, if we were to separate [amazon.com](https://www.amazon.com) into smaller pieces - say a cart and checkout piece, an item recommendation piece, a payment processing piece, etc - then it's safe to say that those pieces of the monolithic application would share libraries, so every time we make a change to one of those libraries, we would have to figure out what piece of the monolith is using that shared library so that we don't break the entire monolith by changing a small part of the library
- **Coding language restrictions** - Another constraint w. monoliths is that if our monolith starts as say, a Java application, then as we add additional pieces to the Java app they would need to be written in Java as well - which means you are limited by decisions made in the past.
- **Overall complexity** - It would also mean that as the complexity of [amazon.com](https://www.amazon.com) grows, it becomes harder to understand if it's managed as a single entity under a single codebase.



Monolith - solution...

- So, how do we fight the complexity of monoliths?
- Think: how do you eat an elephant?
 - One bite at a time!
- By breaking something down into more manageable pieces - **microservices** - we can better tackle the problem and deal with complexity.
- **Microservices** are the “smaller pieces” of the larger application

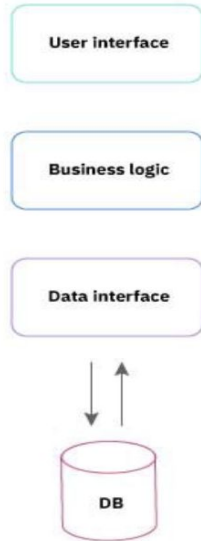


Microservices example

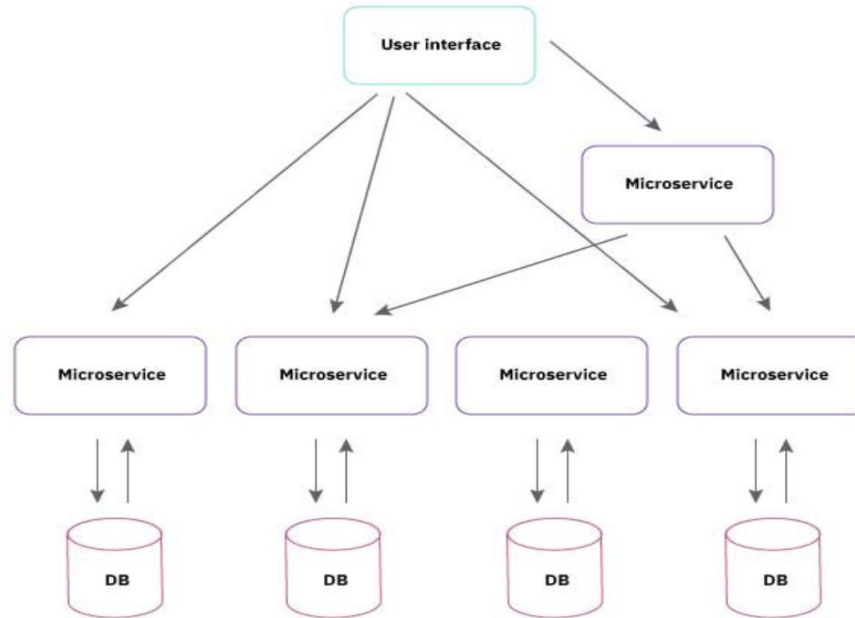
- So, if amazon.com is our “elephant” we can have broken it down into smaller, more manageable pieces
- For instance, search could function as its’ own microservice, the cart would be its’ own microservice, etc.
- Each microservice would be its’ own application, with its’ own separate Github repository
- And all of these microservices combined would give us a fully functional amazon.com site!

Microservices vs monoliths...

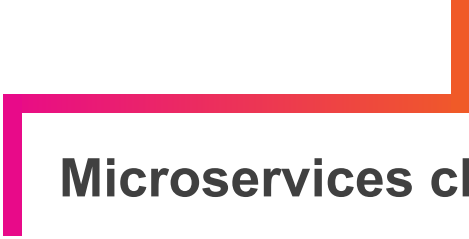
Monolithic Architecture



Microservices Architecture



Source: [IBM blogs](#)

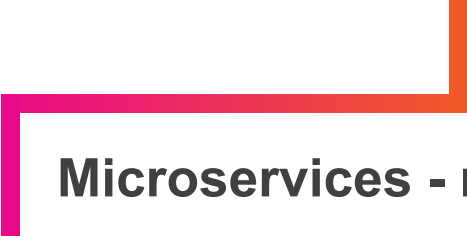


Microservices characteristics

- Small, fully independent and autonomous application. Isolated from other MS's, maybe even with own database.
- Responsible for a logical part of a larger application “ecosystem”. Singular responsibility.
- Can be any language. Developers on the same project can have completely different skill sets if necessary.
- Loosely coupled to other MS's. Communicate with other MS's using standardized protocols. Often a REST API or a message bus.
- People generally start with coarse grained services and then break them up into smaller and smaller pieces as the time goes by and they know more and more

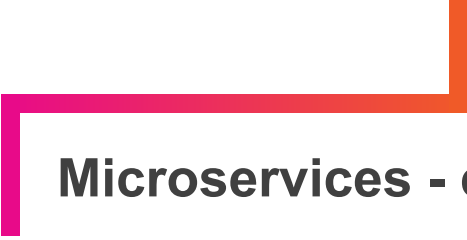
Microservices benefits

- By dividing and conquering w/ microservices, if we want to make a small change to the site, we would just update the micro service that is responsible for that portion of the site
- And the key here is that we would only need to redeploy that particular microservice, without disturbing/interfering with all the other stuff that powers amazon.com.
- This is as opposed to using a monolith where we would have to redeploy all of amazon.com for a small change which doesn't make sense of course.
- *One of the clear advantages of using microservices is that it allows us to be **more agile***
 - *Agile means we can deploy quickly when changes need to be made, and respond to the needs of the business faster!*



Microservices - more benefits

- Small and manageable pieces. No massive monoliths.
- No need to have every microservice on the same language and framework.
 - For instance, w/ ecommerce app - your payment services can be in Python, and your user cart can in Java
- Testing is more limited/easier since you don't have to test an entire monolith.
- Can have their own development and release cycle.
- Can be deployed independently without breaking other pieces. Can be done faster.
- Independent scaling of MS's. Faults are isolated. "Graceful degradation".
- Encourages encapsulation of business logic.
- Smaller size enables Agile development.



Microservices - challenges

Now that you understand bit more about what a microservice is, what challenges can you think of w/ microservices?

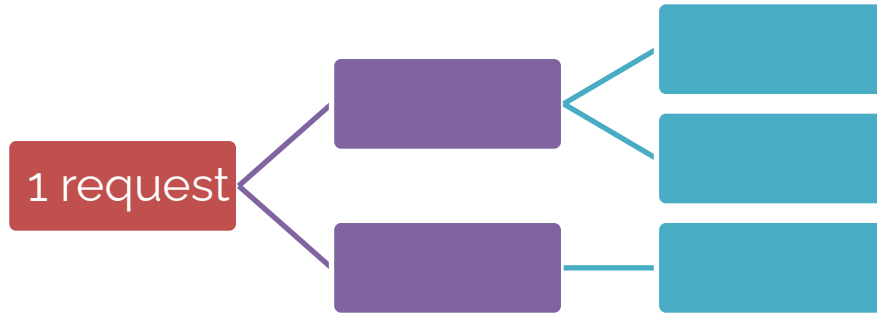
Type answers in chat...

Microservices - challenges

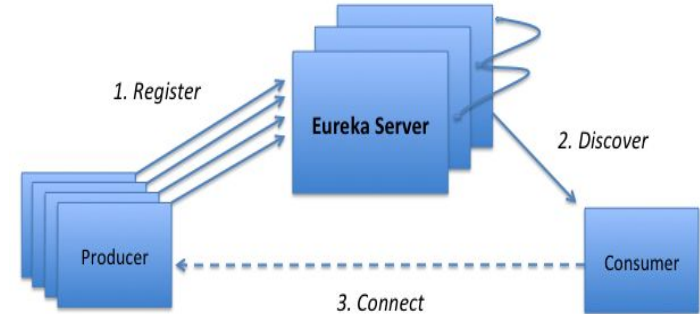
- Microservices come with their own challenges as well
 - Just like monoliths!
- A lot more moving parts. This often requires technologies that might not otherwise be used, such as message buses (like Kafka).
- Requires seasoned architects and leads to keep disaster at bay.
- Often requires more infrastructure to support, as each component has its own overhead.
 - With a monolith, overhead can often be shared.
- Coordination between dependent MS's can be more complex than a monolith.
- More parts to deploy.

Microservices - challenges

- Service Discovery (need Service Registry)
- Fault tolerance
- Integration Testing
- Requests can fan out



Service Registry:

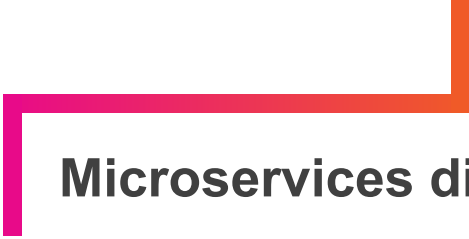


Microservices: Discussion

E-commerce application

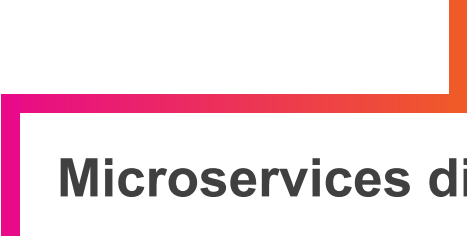
 **Develop**Intelligence

A PLURALSIGHT COMPANY



Microservices discussion - breakout rooms

- Think about Amazon.com - where each web widget on the homepage can be backed by hundreds of microservices. Think about microservices for personalization of home page & personalization of search.
- Pick a specific web widget from the homepage - it could be search bar, cart, item recommendations, sign in, etc...
- Now, how you would break this down into different functionalities? What are the microservices supporting those functionalities? How do they communicate? How is data stored for each service?
- Take 10 mins to think about it with your team and have one person from your breakout room share your findings.



Microservices discussion

Major functionality:

- Product catalog (admin functions, search, recommendations, related products, etc.)
- Shopping cart (and email reminders about items in cart)
- Order processing (payment processing, emails, etc)
- Customer order cancel and refunds
- Fulfillment (send to warehouse, inventory allocation, picking, shipping, backorders, etc.)
- Reporting (internally and to the customer)
- Email marketing
- Social media integration
- Shipment tracking
- Bonus: Digital content! Movies, music, etc.

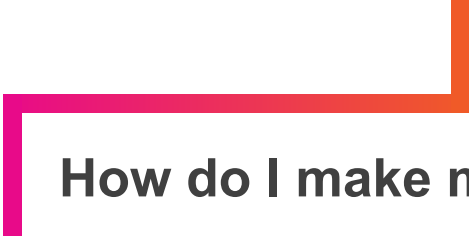


Cloud Native

Making it a reality

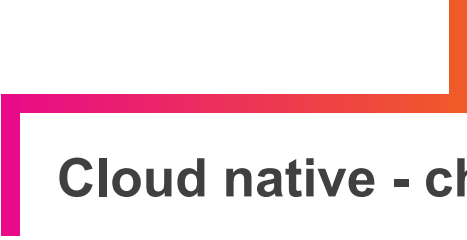
 **Develop**Intelligence

A PLURALSIGHT COMPANY



How do I make my application cloud-native?

- ***cloud native*** == your application has been ***designed/architected to run on the cloud effectively...***
- We've covered **what** cloud native means but now the question is **how** can we make our application cloud native?
 - What specific steps can we take to make an application cloud native?



Cloud native - characteristics

- **The Twelve-Factor App:** Baseline best practices for designing & building web applications for portability and resilience.
Reference: <https://12factor.net/>
- **Microservices:** Independently deployable services that do one thing well (Single Responsibility Principle).
- **Self-Service Infrastructure:** Platforms for rapid, repeatable, and consistent provisioning of app environments and backing services.
- **API-based Collaboration:** Published and versioned APIs that allow interaction between services.
- **Anti-Fragile:** Systems that are resilient to stress and failure.



Kotlin

Kotlin - Open Source and Statically Typed



- Open Source developed mainly by JetBrains with some community support
 - JetBrains == creators of the IntelliJ IDE
- Statically typed (like Java)
 - statically typed == every variable/expression has a type
 - You can omit types in Kotlin
 - Hence Kotlin often looks as concise as some dynamically typed languages

Kotlin - omitting types



- You can omit the type of a variable or function parameter if the type can be inferred from the context
- For example, in following code the variable name is inferred to be a String since it's assigned a String literal:

```
val name = "John Doe"
```

Kotlin - safety



- Kotlin is safer than Java
- In the sense that the compiler can help prevent even more types of errors

Kotlin Interopability with Java



- Main characteristic of Kotlin: ***interopability with Java***
- We might not call Java a “modern” programming language anymore
 - But because it has such a huge ecosystem, recreating all of it from scratch with a new language would be a huge task.
 - That is not the goal of Kotlin - we want to reuse every existing Java library and framework from Kotlin
- Basically, we want to re-use the Java ecosystem from Kotlin

Kotlin Interopability with Java



- Kotlin code can call Java code and vice-versa
- You can use Kotlin in existing Java projects
- So, you can gradually convert your Java projects to Kotlin



You can have Java & Kotlin code in one project



Kotlin and Java

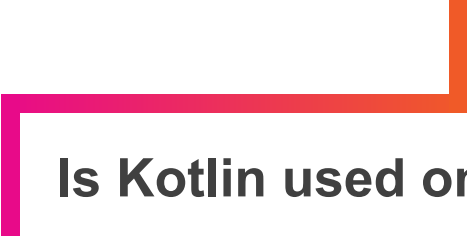


- We can have Kotlin (.KT) files and Java (.java) files in the same project - both types of files will be compiled to bytecode.
- But keep in mind we can not have Kotlin and Java being used in the ***same file***

Kotlin - history and tooling



- Kotlin has great tooling support from IntelliJ - IDE features like completion, refactorings, and navigation are all supported
- Kotlin started in 2010 and took 6 years to achieve its mature state and first release
- After that, it was acknowledged as a first-class citizen on Android, which really boosted the adoption.



Is Kotlin used only for Android dev?



- Many people now think that Kotlin is only used when doing Android development
- There are actually many server-side developers (including at Intuit) who already successfully use Kotlin for their projects.

Different target platforms



- JVM - when compiled on the JVM, Kotlin will be compiled down to Java compatible bytecode
- This means that Kotlin code can be run on any platform that has a Java Virtual Machine (JVM)
 - This includes Windows, Mac, Linux, and Android devices

Different target platforms



- Kotlin can also be compiled into Javascript or even Native code now
- Native code == machine-specific code (code specific to a given processor/chip)



Kotlin/JVM



Kotlin/JS



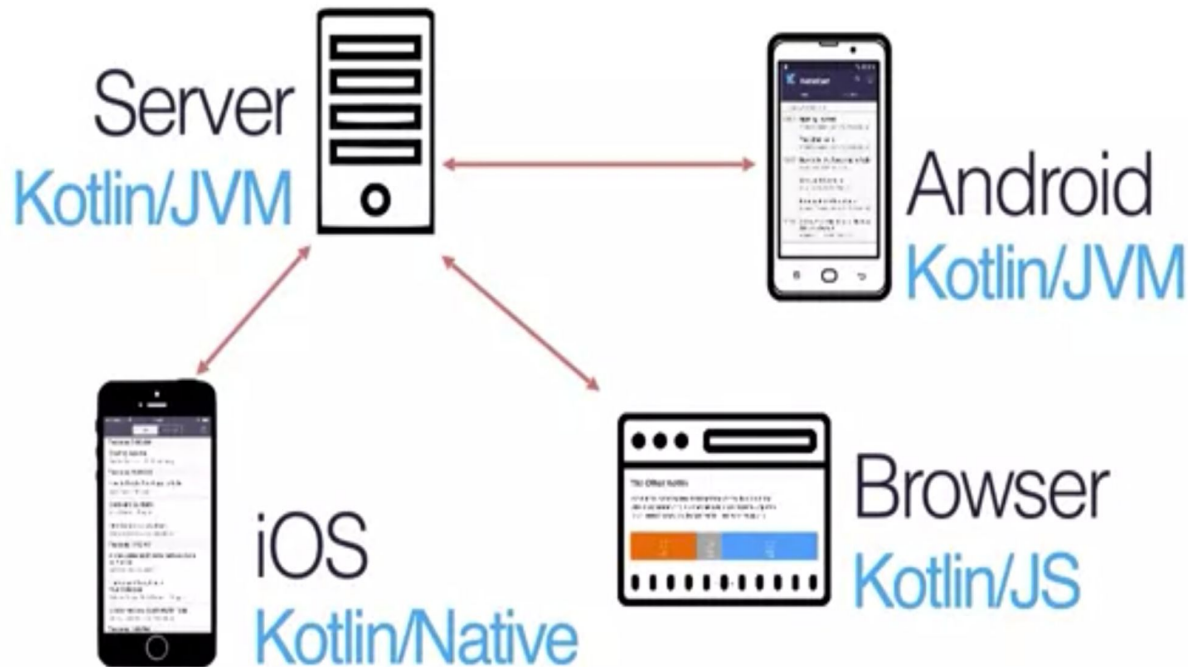
Kotlin/Native

Multiplatform projects



- We can write a full stack application entirely in Kotlin
 - Server side, front-end, and application code can all be written in Kotlin
 - There is a Kotlin-Javascript compiler
- Also, most importantly - you can share business logic and common code ***across different platforms***

Multiplatform projects



What is Kotlin compiled to?

- Java source code OR Java bytecode?



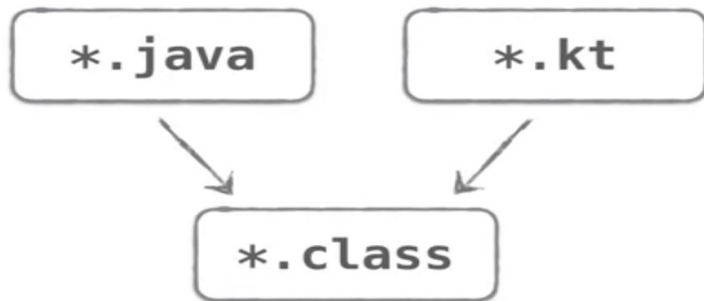
What is Kotlin compiled to?



Kotlin is compiled to Java bytecode

1. Java source code

2. Java bytecode



Going from Java -> Kotlin



- In IntelliJ, there is a converter that allows you to convert your Java code to Kotlin
- Suppose we have the following Java code that does nothing meaningful other than storing some data - what would it look like in Kotlin?

Going from Java -> Kotlin



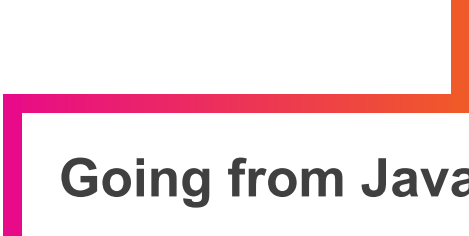
Person class in Java

```
public class Person {  
    private final String name;  
    private final int age;  
  
    public Person(String name, int age) {  
        this.name = name;  
        this.age = age;  
    }  
  
    public String getName() {  
        return name;  
    }  
  
    public int getAge() {  
        return age;  
    }  
}
```

Going from Java -> Kotlin



- If we convert it, we will end up the code on the following slide
- You can see how much simpler and more concise the code is!
- Under the hood, the same constructor and two getters are generated
- From a Java point of view that is the same exact class!!



Going from Java -> Kotlin



Person class in Kotlin

```
class Person(val name: String, val age: Int)
```


Going from Java -> Kotlin



- We can add a 'data' modifier to this class in Kotlin
- That will automatically generate several additional methods for it - like equals hashCode and toString.

Person class in Kotlin

```
data class Person(val name: String, val age: Int
```

- equals
- hashCode
- toString

Going from Java -> Kotlin



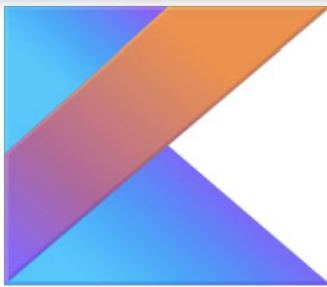
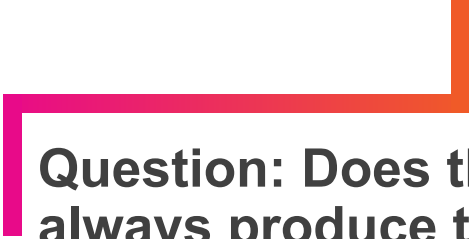
- We could instantiate the class like so in Kotlin - note that there is no usage of the **new** keyword

Converting usages of the Person class

```
Person person = new Person("Alice", 27);  
System.out.println(person.getName());
```

no new keyword

```
val person = Person("Alice", 27)  
println(person.name)
```



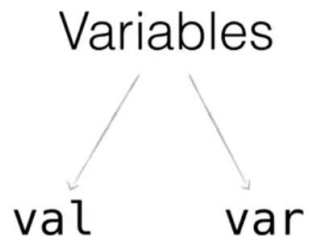
Question: Does the automatic Java to Kotlin converter always produce the idiomatic Kotlin code?

- What do we mean by the term idiomatic?
 - Idiomatic == code that follows the conventions and standards that are recommended by the Kotlin community
 - Idiomatic Kotlin code is easier to read, understand, and debug as well!

Kotlin - Variables

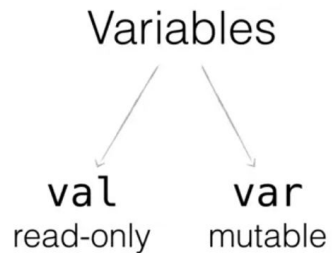
Val vs var

- There are two main keywords in Kotlin to define variables: 'val' and 'var'



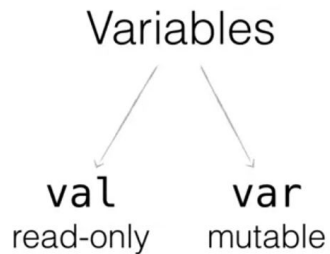
Val vs var

- 'val' comes from 'value' and denotes a read-only or assigned-once variable
 - So, val == read-only
- 'var' comes from 'variable' and denotes a mutable variable, one that can be modified.
 - var == mutable



Read-only variables

- 'Read-only variables can NOT be reassigned.'
- If you try to assign a new value to it, you'll get a compiler error.
- In essence, 'val' in Kotlin corresponds to a 'final' variable in Java
- If you need to reassign a variable, you declare it as 'var'.
- Then you can easily assign a new value to it.



Read-only variables

Read-only variable: `val`

```
val question: String =  
    "life, the universe, " +  
    "and everything"  
println("$question?")
```

life, the universe, and everything?

What happens if we re-assign the variable?

Read-only variable: `val`

```
val question: String =  
    "life, the universe, " +  
    "and everything"  
println("$question?")
```

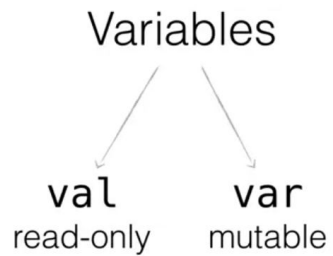
```
question = "sure?"
```

Compiler error:
val cannot be reassigned

corresponds to a `final` variable in Java

Val and Java

- 'val' is equivalent to 'final' in Java





Var is mutable

Mutable variable: **var**

```
var answer = 0  
answer = 42  
println(answer)           // 42
```

Local Type Inference



- Kotlin will infer types for you
 - If you omit a type, Kotlin will infer/deduce what type it is based on context
- Given this code for example:

Local type inference

```
val greeting = "Hi!"
```

```
var number = 0
```

Local Type Inference

- Kotlin will infer the types to be String and int:

Local type inference

`val greeting = "Hi!"`
: String

`var number = 0`
: Int

String and Int types are inferred

Question - type your answer from 1-3 in chat



Why doesn't the following code compile?

```
var string = 1  
string = "abc"
```

Because you can't

1. re-assign var
2. assign an integer value to a variable of `String` type
3. assign a string literal to a variable of `Int` type

Answer

- Type inference means it's actually an int
- The name string is deliberate to confuse you!



Why doesn't the following code compile?

```
var string = 1  
string = "abc"
```

Because you can't

1. re-assign var
2. assign an integer value to a variable of String type
3. assign a string literal to a variable of Int type

Answer

Local type inference

```
var string = 1  
string = "abc"
```

: Int

Compiler error: Type mismatch:
inferred type is String
but Int was expected

Question - val



Is it possible to modify
an object stored in `val`?

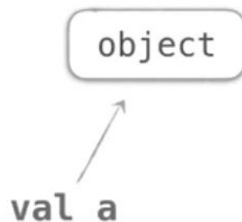
1. yes
2. no

Question - val



Is it possible to modify
an object stored in `val`?

- 1. yes
- 2. no



Answer - val



Is it possible to modify
an object stored in `val`?

1. yes

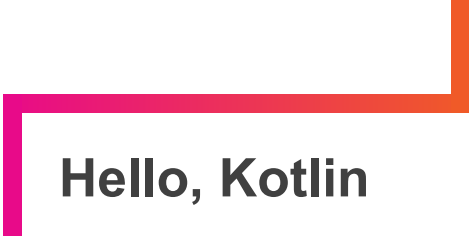
2. no

object

`val a`



```
graph BT; a["val a"] --> o["object"]
```



Hello, Kotlin

“Hello, Kotlin” example

```
package intro
```

```
fun main() {  
    val name = "Kotlin"  
    println("Hello, $name!")  
}
```

No class required

No class

```
package intro

fun main() {
    val name = "Kotlin"
    println("Hello, $name!")
}
```

Hello.kt

You can define functions at the top level



Main without args

main with or without arguments

```
fun main() { ... }    // starting from Kotlin 1.3
```

```
fun main(args: Array<String>) { ... }
```

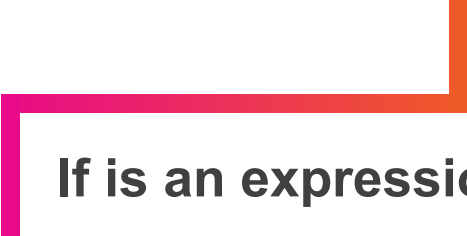
Arrays

Arrays

```
package intro

fun main(args: Array<String>) {
    val name = if (args.size > 0) args[0] else "Kotlin"
    println("Hello, $name!")
}
```

Array looks like a regular class



If is an expression

if expression

```
package intro
```

```
fun main(args: Array<String>) {  
    val name = if (args.size > 0) args[0] else "Kotlin"  
    println("Hello, $name!")  
}
```

if is an expression

String templates

String templates

```
package intro

fun main() {
    val name = "Kotlin"
    println("Hello, $name!")
}
```

Hello, Kotlin!

String templates: easy way to insert a value into a String

String templates

String templates

`"...$variable..."`

`"...${functionCall(...)}..."`

```
fun main(args: Array<String>) {  
    println("Hello, ${args.getOrNull(0)}!")  
}
```

Question



What will be printed
if no arguments are passed?

```
fun main(args: Array<String>) {  
    println("Hello, ${args.getOrNull(0)}!")  
}
```

1. Hello, null!
2. Hello, !
3. NPE will be thrown

Answer



What will be printed
if no arguments are passed?

```
fun main(args: Array<String>) {  
    println("Hello, ${args.getOrNull(0)}!")  
}
```

1. Hello, null!
2. Hello, !
3. NPE will be thrown

Explanation

Works as in Java

```
public static void main(String[] args) {  
    System.out.println("Hello, " + null + "!");  
}
```



Hello, null!

```
fun main(args: Array<String>) {  
    println("Hello, ${null}!")  
}
```



Hello, null!

Kotlin Exercise # 1

- Go ahead and work on this exercise within the Kotlin playground itself:
<https://pl.kotl.in/LiWGPcnvI>
- Take about 10 minutes
- We will go over solution together



What will be printed
if no arguments are passed?

```
fun main(args: Array<String>) {  
    println("Hello, ${args.getOrNull(0)}!")  
}
```

1. Hello, null!
2. Hello, !
3. NPE will be thrown

Question



What will be printed?

```
fun foo(): String {  
    println("Calculating foo...")  
    return "foo"  
}  
  
fun main() {  
    println("First ${foo()}, second ${foo()}")  
}
```

Calculating foo...

Calculating foo...

First foo, second foo

Calculating foo...

First foo, second foo

Question



What will be printed?

```
fun foo(): String {  
    println("Calculating foo...")  
    return "foo"  
}  
  
fun main() {  
    println("First ${foo()}, second ${foo()}")  
}
```

Calculating foo...
Calculating foo...
First foo, second foo

Calculating foo...
First foo, second foo

Kotlin Exercises



Kotlin Exercises



- Go ahead and do these exercises - we will walk through them together as well:
- <https://developer.android.com/codelabs/kotlin-bootcamp-basics#0>
- <https://developer.android.com/codelabs/kotlin-bootcamp-functions#0>
- <https://developer.android.com/codelabs/kotlin-bootcamp-classes#0>



Kotlin Exercises Part 2



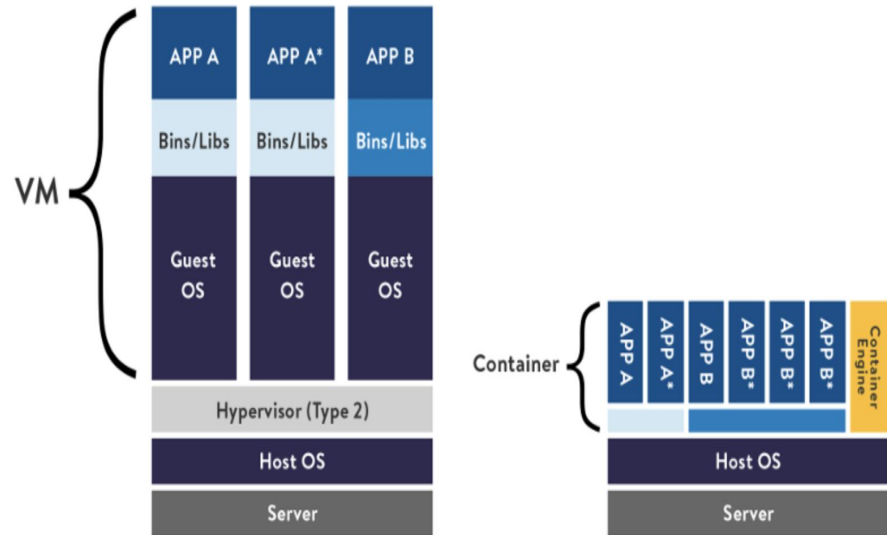
- Go ahead and do these exercises - we will walk through them together as well:
- <https://developer.android.com/codelabs/kotlin-bootcamp-extensions#0>
- <https://developer.android.com/codelabs/kotlin-bootcamp-generics#0>
- <https://developer.android.com/codelabs/kotlin-bootcamp-sams#0>

Docker and Kubernetes

Containers vs VM's revisited

- Both VM's and containers are essentially "wrappers" around our applications making it easier to run our apps on the cloud
- Containers allow us to share a copy of the OS between applications
- Apps running inside a VM each have their own copy of the OS
- Hence, VM's are heavier
- Containers are lightweight, low overhead, and we can run more apps

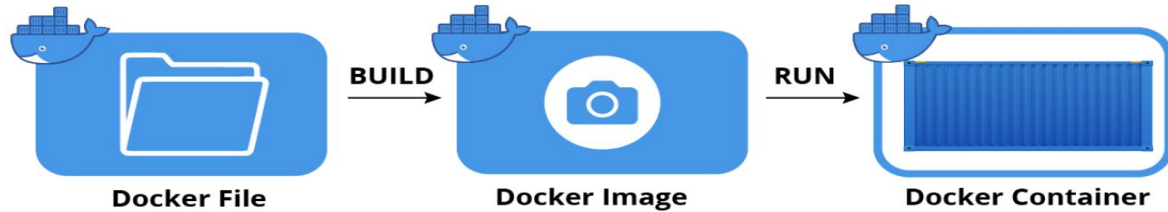
VM vs Containers



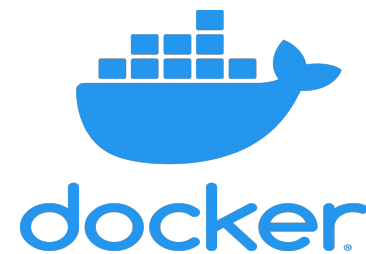
Docker and K8s – why?

- ❑ To really understand the AWS services that use Docker and K8s we should first try to understand Docker/K8s individually
- ❑ Services such as ECS, EKS,

Docker terminology



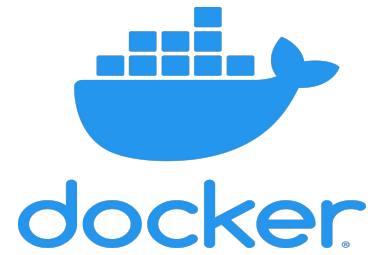
- DockerFile + your application code -> Looper -> Docker Image -> Docker container
- DockerFile will live in your application's repository
- Your source code + DockerFile will give us a Docker image
- Docker **images** are how we share our application
- An instance of a Docker image is a running container....this is a running instance of your application!
- Don't worry if this is not clear yet, we will be playing with images and containers shortly..



What problem does Docker solve?

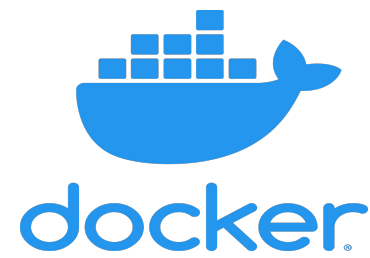
- ☐ K8s is container orchestration
- ☐ Before we get into K8s – container orchestrator - logical to cover containers first
- ☐ Concept of containers not invented by Docker
- ☐ But Docker executed the concept very well!
- ☒ ***Helps eliminate the 'works on my machine phenomenon'***
- ☐ Simple example: trying to get a Node.js app to work locally is tough – containerizing it makes it run consistently on different machine setups!
- ☐ The Docker ecosystem is excellent....

The Docker ecosystem

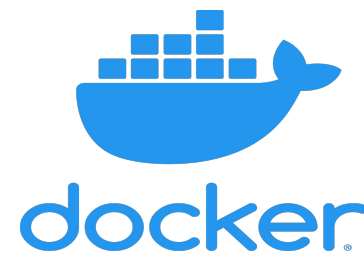


- ☐ K8s is container orchestration
- ☐ Before we get into K8s – container orchestrator - logical to cover containers first
- ☐ Concept of containers not invented by Docker
- ☐ But Docker executed the concept very well!
- ☒ ***Helps eliminate the 'works on my machine phenomenon'***
- ☐ Simple example: trying to get a Node.js app to work locally is tough – containerizing it makes it run consistently on different machine setups!
- ☐ The Docker ecosystem is excellent....

The Docker ecosystem pt 2



- ❑ Docker CLI – easily run images locally, create new images, and manage images
- ❑ Docker Engine – enables containers to run consistently on any infrastructure – various Linux distros + Windows/Server
- ❑ <https://www.docker.com/products/container-runtime>
- ❑ Docker Hub – easily share images with the world



Docker Registries

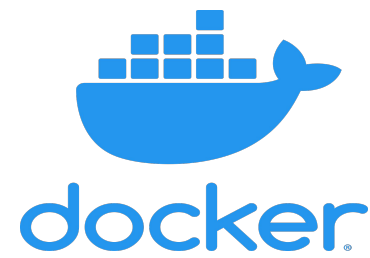
- ❑ We want to share our applications with the world – how do we do that?
 - ❑ With images – which encapsulate our application
- ❑ But where do we put those images?
- ❑ Inside a Docker registry!
- ❑ **Docker registry** is open source software that you can use to set up your own registry
 - ❑ You can see how to set one up here:
<https://docs.docker.com/registry/>
 - ❑ ***Anyone can set up their own Docker registry***
- ❑ Cloud providers also provide their own registries for storing images, like Azure, GCP, etc..



Docker Repositories

- ❑ Within a registry, you would have many repositories
- ❑ In Docker, a repository is a collection of images of the same application
- ❑ Basically, a repository is a place where multiple versions of a single application live
 - ❑ For example, there is a repository for a Jenkins image:
 - ❑ <https://hub.docker.com/r/jenkins/jenkins>
 - ❑ Repos can be public or private on Docker hub, just create an account..
- ❑ To summarize, smallest to largest: image -> repository -> Registry -> Docker Hub (Registry + Extras)

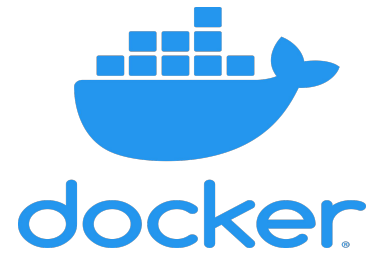
Docker images



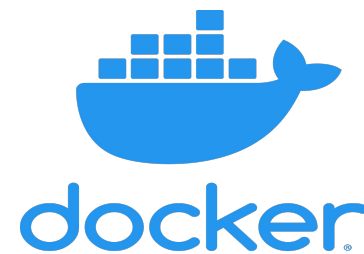
Docker images – the details

- ❑ An image is a template for containers
- ❑ Good analogy: Docker image is like a Java class – where a Java class is a template for creating Java objects, Docker images are templates for actual containers

Docker images – the details

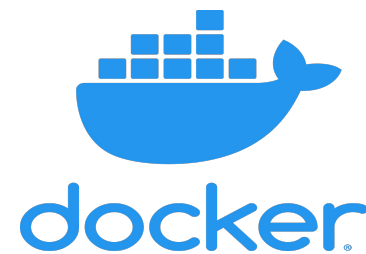


- ❑ Images are immutable – can not be changed
- ❑ If you want to make some changes to your application or Dockerfile then you would have to create a new image



Docker images – what's inside?

- ☐ Think about any application you've worked on in the past
- ☐ What would you need to run that app on a server that's not your local machine?
 - ☐ The source code, the dependencies, tools, etc..
 - ☐ All of that stuff is inside a Docker image!
- ☐ For instance, a React/Node.js application – what's inside that?
 - ☐ Our ReactJS files – our frontend
 - ☐ Our Node.js runtime – our backend
 - ☐ Our Node.js web framework – Fastify.js, or Express
 - ☐ Other JS dependencies/libraries – like Babel, Redux, etc..
 - ☐ Also the Alpine OS – CentOS, Ubuntu – all share the Linux kernel

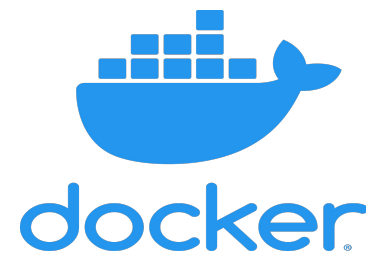


Docker images – layers?

- ❑ A Docker image is comprised of many layers internally
- ❑ You may also notice this when pulling an image – Docker will create a separate line in your command prompt for each layer it pulls for that particular image
- ❑ The layers within an image may look something like this:

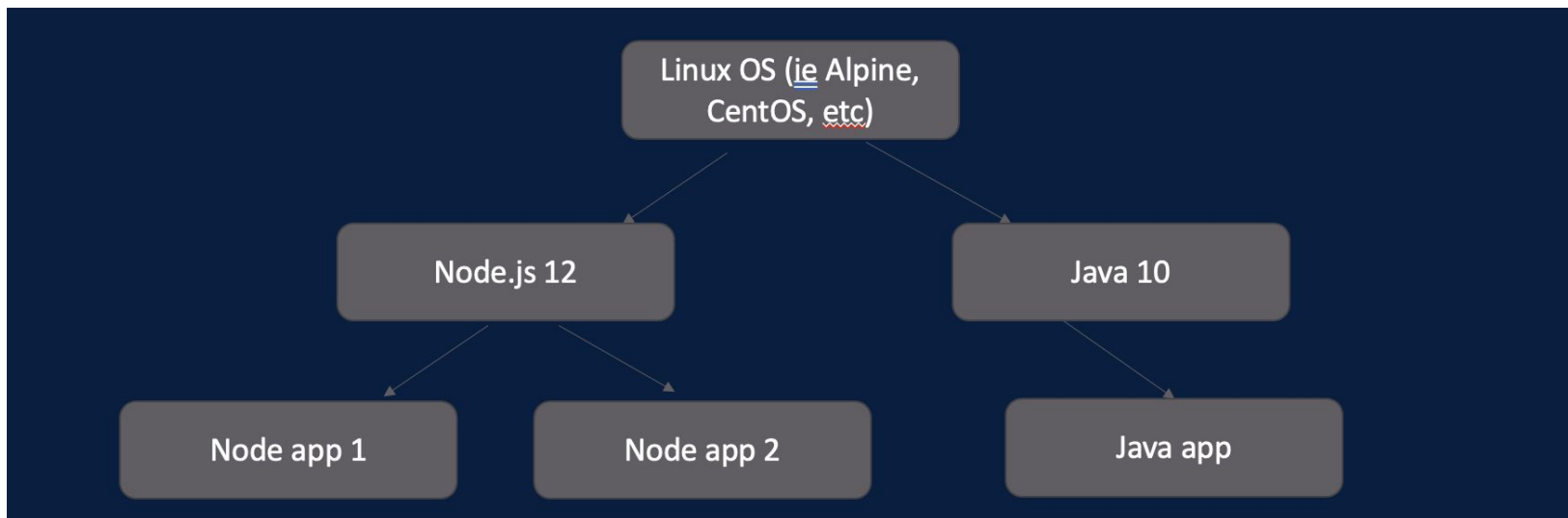
Layer 1: Linux OS: Alpine ~ 50 MB
Layer 2: Security fixes: ~ 2 MB
Layer 3: PHP runtime: ~ 30 MB
Layer 4: Your Wordpress files: ~10 MB

- ❑ When pushing/pulling images, only changed layers are pushed/pulled to save on Bandwidth



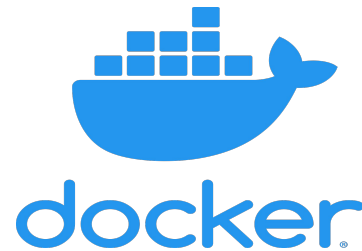
Docker image inheritance

- ❑ Images typically inherit from other images
- ❑ Where do we specify which image we want to inherit from? In the Dockerfile! We will see more later...



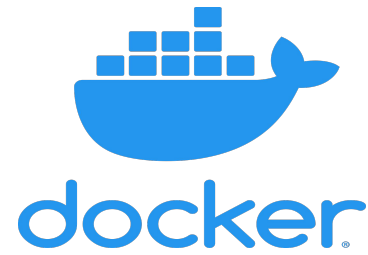
Docker CLI

Docker CLI

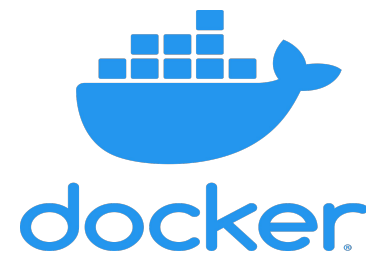


- ☐ The Docker CLI looks a lot like Linux commands
- ☐ All commands start with “docker”
- ☐ When working with the Docker CLI, there’s really 2 sets of commands:
- ☐ One set for working with *images*
- ☐ One set for working with *containers*
- ☐ Let’s cover the CLI for images first...

Docker CLI - images

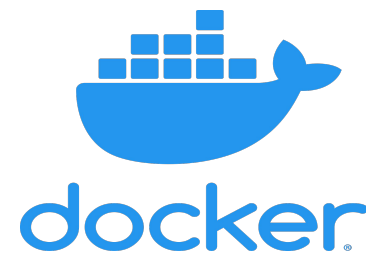


- ☐ `docker build ...` – used to build image from Dockerfile
- ☐ `docker pull ...` – to pull an image from a repo
- ☐ `Docker run ...` - to run an image
- ☐ `-it` – interactive and gives you command prompt inside of container
- ☐ `-p` – port mapping, exposes port from inside container to host
- ☐ `-e` – environment variable
- ☐ `-v` – volume mapping
- ☐ `docker images` – this will show you all of your images
- ☐ `docker rmi ...` – to delete an image (pass in the image name or id)
- ☐ Can't delete an image that has a running container (a running instance)



Docker CLI - containers

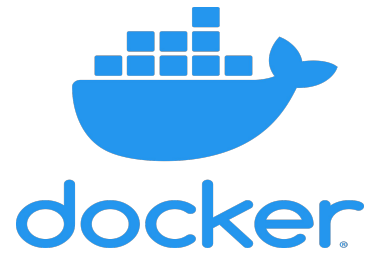
- ☐ `docker ps` – to list running containers
- ☐ Use `-a` arg to see stopped containers as well
- ☐ `docker inspect ...` – to inspect a running container
- ☐ `docker logs ...` - to view console output of a container
- ☐ `docker rm ...` – to remove a container
- ☐ `docker start/stop/restart ...` – self explanatory
- ☐ Difference between **stop** and **rm**?
- ☐ A container that's stopped can be restarted, not true if "rm"d (deleted)
- ☐ If you **stop** a container, you will still see it in **docker ps -a** , but not if you used **rm**



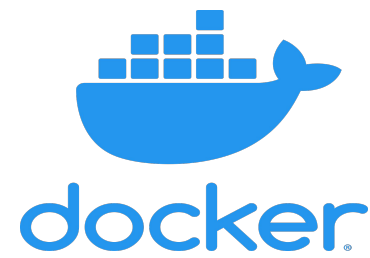
Docker CLI – port mapping

- ❑ By default, when you run a container with `docker run`, it doesn't publish any of its ports to the outside world – ie, your localhost when running Docker Desktop
- ❑ Which makes sense because a container is its own isolated environment!
- ❑ So, how do we expose that internal port?
- ❑ Through the `-p` flag (p for publish) when we do a **docker run**
- ❑ So, if we do **docker run -p 8080:80** that says map port 80 in the container to port 8080 on the Docker host – your localhost in this case
- ❑ `docker run -p [host_port]:[container_port] [docker_image]`

Docker CLI – foreground and detached



- ❑ By default, **docker run** will run in the foreground
- ❑ Which means that you will see all of the container output!
- ❑ Because the console executing docker will attach to standard input/output/error of the container
- ❑ **docker run [image_name]** – this will run in foreground mode
- ❑ In detached mode, the container will run in the background:
- ❑ **docker run -d [image_name]**
- ❑ Also, interactive mode can give you a shell inside the container, in this case bash:
- ❑ **docker run -it [docker_image] /bin/bash**



Docker ps (containers)

- ☐ To view all running containers you can use “docker ps”
- ☐ To view all running **plus** stopped containers you can do “docker ps -a”
- ☐ What is a stopped container?

Docker CLI - port mapping

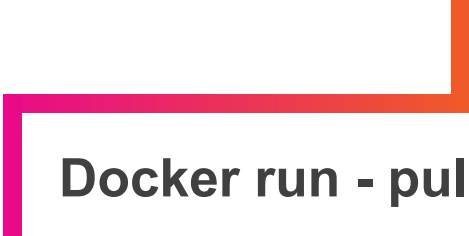
- By default, when you run a container with `docker run`, it doesn't publish any of its ports to the outside world – ie, your localhost when running Docker Desktop
 - Which makes sense because a container is its own isolated environment!
- So, how do we expose that internal port?
 - Through the '-p' flag (p for publish) when we do a **docker run**
 - So, if we do **docker run -p 8080:80** that says map port 80 in the container to port 8080 on the Docker host – your localhost in this case
- `docker run -p [host_port]:[container_port] [docker_image]`

Docker run command - overriding

- We can override the default command that is run by the image by specifying an extra parameter
- Example: ***docker run busybox***
 - This just starts the container
- But if we do “***docker run busybox ls***”
 - It will start the container and list the contents of the current directory in the container

Docker run - naming containers

- When creating a container, you can specify a name for the container
- If you don't specify a name, Docker will give the container a name automatically
- Docker has fun with the name when it generates it (if you don't specify one)
 - See Docker Dashboard!
- To specify a name for the container, simply do this:
 - **`docker run --name mynginxcontainer`**



Docker run - pulls images

If you try to use docker run with an image that you have not already pulled locally, it will pull it!

If an image doesn't exist on the host yet, it will be pulled before running.

Run is a shortcut operation for:

- Docker pull <image>
- Docker create <image>
- Docker start <container>

example: `docker run -it -p 27017:27017 mongo:4.0 /bin/bash`

Docker run - foreground/detached modes

- When running a container it can run in either foreground or detached mode
- Foreground mode is the **default**
 - This is why you get the container output in the terminal:
- Foreground - when running a container console executing docker run attaches to standard input, output and error of the container:

```
varoonsahgal@Varoons-MacBook-Pro ~ % docker run nginx
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
```

In detached mode the container runs in background.

`docker run [docker_image]`

`docker run -d [docker_image]`

`docker run -it [docker_image] [shell]`

example: `docker run -it -p 27017:27017 mongo /bin/bash`

Docker run - detached mode

- In detached mode the container runs in background.

- Use the - d flag to specify detached mode:

```
docker run -d [docker_image]
```

- foreground runs by default:

```
docker run [docker_image]
```

```
docker run -d [docker_image]
```

```
docker run -it [docker_image] [shell]
```

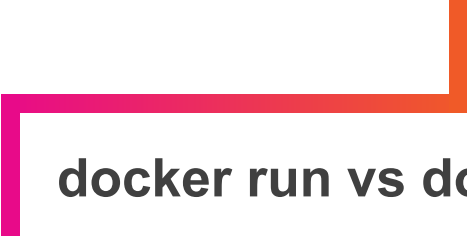
```
example: docker run -it -p 27017:27017 mongo /bin/bash
```

Docker exec command

- **Docker exec** can be used to **execute** any arbitrary process available in a **running** container.
- Note that you can NOT use docker exec with an **image**, just a **container** (running instance of an image)
- Example of it's usage: you can use **docker exec** to “log in” to a shell. This is dependent on the image the container was built from.
- For example: the alpine shell is /bin/ash
- This is much like an SSH connection to a server

Docker exec command

- Once inside the container, you can look around and inspect your container
- This is very useful for debugging.
- Example of why: Suppose you've created an image and you're not sure if the image looks like you expect. So you can get into a running container and view things.
- For example, when we built the nginx image, how did we know the file was there until we tried the web site?
- The location of the shell process depends on the container.
- ***docker exec -it nginx bash***
 - This will open up a shell in the nginx container



docker run vs docker exec

- Docker exec is NOT meant to start a new container from an image
- For instance, if you try to run **docker exec nginx**
 - You will get an error
 - However, **docker run nginx** works just fine
- Docker exec is meant to execute a process in a running container
 - So it's good for creating a shell inside a container!
- Docker run is meant to run a new container based on an image

Docker Lab 1



wordpress ☆

Docker Official Images

The WordPress rich content m.

Docker lab/exercise 1

- ☐ To best understand Docker, it helps to play with it!
- ☐ Let's create our own Docker image by pulling something from Docker Hub
- ☐ Let's try Wordpress: https://hub.docker.com/_/wordpress
- ☐ Read the instructions
- ☐ Your goals:
 - ☐ Run the image locally. Use the docker run command and a port mapping. Remember images must be run to get containers
 - ☐ View the Wordpress UI once the container is up and running
 - ☐ Try creating another Wordpress container while you still have this one running – remember not to use the same port on your localhost!
 - ☐ Also, try to view the new instance of Wordpress in your browser
 - ☐ This should help consolidate that a single image spins up multiple containers – becomes more important w/ K8s!

Docker lab/exercise 1 part 2



wordpress ☆

Docker Official Images

The WordPress rich content m.

Challenge mode!

1. You'll need a MySQL Docker container to be the DB for WP site
2. Containers are **isolated** by default, so you can put them in a **docker network** to share access

```
docker network create my-wp
```

3. Create the MySQL container and specify some setup like DB name and login info

```
docker run --name wp-mysql -e MYSQL_ROOT_PASSWORD=root -e MYSQL_DATABASE=wp -e  
MYSQL_USER=user  
-e MYSQL_PASSWORD=admin --net=my-wp -d mysql:latest
```

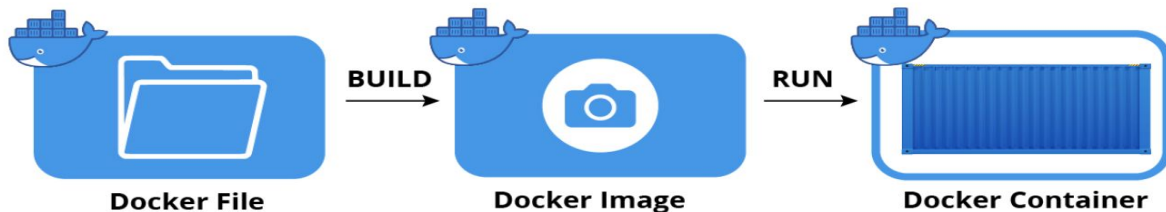
4. Run a WP container that connects to that container:

```
docker run --name wordpress -e WORDPRESS_DB_HOST=wp-mysql:3306 -e  
WORDPRESS_DB_USER=user -e WORDPRESS_DB_PASSWORD=admin -e  
WORDPRESS_DB_NAME=wp --net=my-wp -p 8081:80  
-d wordpress:latest
```

Finish setting up the site and confirm you can make posts!

Remember: the DB data is (currently) only stored in the container, so if you delete the container your data goes away!

Docker terminology revisited



- DockerFile + your application code -> Jenkins -> Docker Image -> Docker container
- DockerFile will live in your application's repository
- Your source code + DockerFile will give us a Docker image
- Docker *images* are how we share our application
- An instance of a Docker image is a running container...this is a running instance of your application!
- Don't worry if this is not clear yet, we will be playing with images and containers shortly..

Docker – tags and Dockerfile

Docker tags

- In Docker, we can tag our images to give them some metadata
 - Could be a version number, what OS they are meant to run on, etc
- Let's look at an example here for Jenkins tags:
https://hub.docker.com/r/jenkins/jenkins/tags?page=1&ordering=last_updated
- To tag an image, you can run **docker tag httpd:1.2** as an example of adding a tag of 1.2
 - After the colon, we specify the tag
- Every image will have one tag by default if none is specified

Docker tags pt 2

- What if we don't specify a tag for an image?
- By default, if you omit a tag, Docker will give it a tag name of **latest**
 - But **latest** is misleading, because it doesn't always mean that's the latest version of the image
 - Example - developer created an image, didn't tag it, so it's given the "default" tag, which is "**latest**"
 - then developer created a newer image and tagged it 1.2 – the newer one has a tag of 1.2, but **latest** will actually refer to an older image!

Dockerfile

- Remember that **Dockerfile + your app source code -> Jenkins (runs docker build) == Docker Image**
- Dockerfile is how to create Docker images
- A Dockerfile is an example of Infrastructure as code
 - *Infrastructure as code (IaC) means to manage your IT infrastructure using configuration files.*
 - Before IaC, IT admins would have to ***manually*** change configurations in order to manage their infrastructure



Dockerfile - more

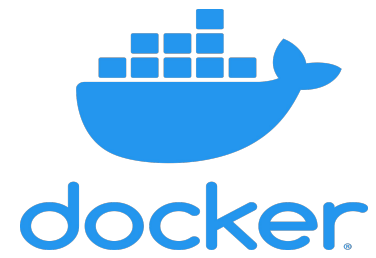
- The commands in a Dockerfile execute from top to bottom
- And each command is also independent of other commands
- Commands are relative to the directory in which the Dockerfile lives



Dockerfile - more

- The commands in a Dockerfile execute from top to bottom
- And each command is also independent of other commands
- Commands are relative to the directory in which the Dockerfile lives

Dockerfile – how we create images



- ❑ Where do Docker images even come from?
- ❑ A Dockerfile is how we create Docker images

Dockerfile build command

How do we create a new Docker image?

- By building a Dockerfile!
- **docker build -t <imagename:tag> <directory>**
 - "t" is to specify a tag

Example:

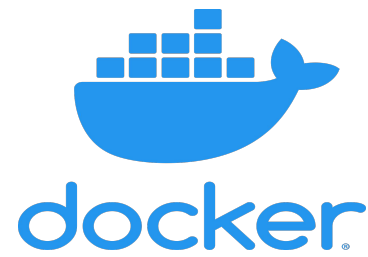
- **docker build -t myimage:v1 .**
- Where **<directory>** is wherever your Dockerfile lives
- Inside the **Dockerfile**, there is a reference to your app code
- We will be using this command shortly...

Dockerfile commands

A sampling of commands you may see in a Dockerfile:

- FROM – which image you want to inherit from
- RUN – Run a Linux command in a new layer. The command is dependent on which shell is in the image.
- EXPOSE – Tells what ports this container will listen on. Does not actually publish a port, but is rather like documentation.
- COPY – copy files into the container.
- ADD – copy files into the container (same as COPY, but supports 2 other sources - additional URL source, tar extraction from local)
- ENV – Define environment variable
- ENTRYPOINT – The command to be run when container is started. This is inherited from a base image.
- CMD – provides defaults to be run after the ENTRYPOINT. Also inherited from the base image

Dockerfile == Infra as Code



- ❑ A Dockerfile is an example of ***Infrastructure as code***
- ❑ *Infrastructure as code (IaC) means to manage your IT infrastructure using configuration files.*
- ❑ Before IaC, IT admins would have to ***manually*** change configurations in order to update/manage their infrastructure
- ❑ For instance, often IT admins would log in to remote machines, make updates through some commands (ie - sudo update apache, or whatever), and the problem was that there was no running history of changes
- ❑ With IaC we can commit things like Dockerfiles to our Git repositories thereby giving us a history of our infrastructure
- ❑ Along with the ability to restore old states

Docker Exercise

- Dockerfile

 **Develop**Intelligence

A PLURALSIGHT COMPANY

Dockerfile lab

- Visit https://hub.docker.com/_/httpd and see instructions on how to use this image
- Let's grab the Docker image for Apache – a popular web server:
- Docker pull httpd
 - By default this will pull the latest image
- Then run Apache locally:
 - `docker run -p 8080:80 httpd`
- Then visit <http://localhost:8080> to see a page that says “It works”



Docker lab 2 - Apache success

It works!

Docker lab 2 part 2 steps

Let's take these steps:

1. Create a file named **Dockerfile**

You can do this anywhere really, don't technically need a repo – best to create a folder

2. Inherit from the base Apache image we just pulled – simply use “**FROM httpd:2.4**”

3. Add a file named test.html so that we can give Apache something to actually serve up instead of the “it works!” text. The **test.html** would be our “application code” in a real application.

You can put this in the **Dockerfile as the second line:**

COPY ./test.html /usr/local/apache2/htdocs/

So, no need for the the public_html folder as mentioned in the docs here:
https://hub.docker.com/_/httpd

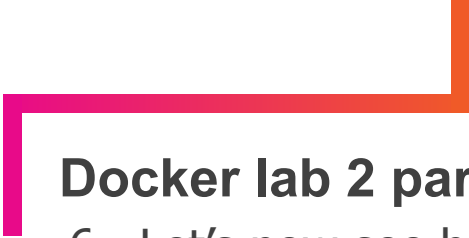
4. Now, build it! Use the **docker build** command:

docker build -t <imagename:tag> <directory> .

What happens when you leave out the tag?

5. Now that you've built the image – run it with **docker run -p [host_port]:[container_port] [docker_image]**

Try exposing it on a different port – the instructions say 8080, but try 3500 or something else. If you visit localhost:3500/test.html you should see your page!



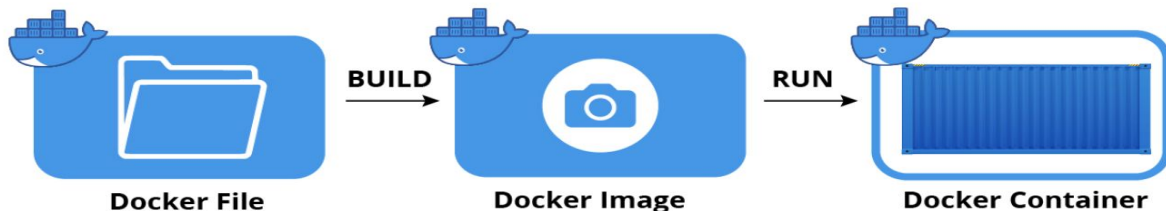
Docker lab 2 part 3 steps

6 . Let's now see how to bring up a bash shell so we can see what's going on inside the container: **docker run -it myfirstimage /bin/bash** or **docker run -it myfirstimage /bin/sh** (if bash not part of image)

Often, you will want to step inside your container to see what's going on. This is how you would do that!

7. Go inside the **/usr/local/apache2/htdocs** directory and you should see the test.html file that you just copied over!

Docker terminology revisited



- DockerFile + your application code -> Looper -> Docker Image -> Docker container
- DockerFile will live in your application's repository
- Your source code + DockerFile will give us a Docker image
- In KITT, profiles can essentially function as Dockerfiles
- Docker ***images*** are how we share our application
- An instance of a Docker image is a running container...this is a running instance of your application!
- Don't worry if this is not clear yet, we will be playing with images and containers shortly..

Docker Networks



Lab: Docker container isolation

Docker containers are isolated by default.

Validate this by doing a very simple exercise:

Open a terminal window and then do:

```
docker run -it busybox sh  
touch helloworld
```

Then, open a new terminal window and create another container w/ the same commands:

```
docker run -it busybox sh  
ls
```

What do you see as the output of “ls”? Do you see the helloworld file we created?



Think:

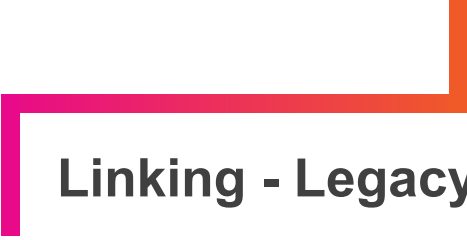
Suppose I have an application - like Wordpress.

The application needs a database to store information about the blog - specifically it needs and uses the MySQL database to store blog posts.

Wordpress will often need to write to the MySQL database - so it needs to communicate with MySQL

So, if I have a Wordpress container and a separate MySQL container - how can those containers communicate in Docker??

Answer: ***Networks!***



Linking - Legacy feature

- Networks are the way to go, but you should know a bit about linking....
- This is a **legacy** feature, but still useful to know.
- Networks (next) are the preferred way of communicating.
- You can explicitly link containers like wordpress and mysql to each other so that they can communicate by name.
- You link containers by including a --link option in the run command and provide the container name.
- Linking does not scale well - try linking 10 services with each other!

Networks in Docker

- You can create Docker networks to allow containers to easily communicate.
- Types:
 - Bridge (default) – this one is created by Docker and containers go here by default. Not recommended for production. No container communication by name.
 - Bridge (user-defined) – Recommended for standalone containers running on the same host. Containers can communicate by name.
 - Overlay – allows networking across Docker hosts. Works with Swarm.
- Networks are managed with the `docker network ...` commands
- `docker network` will show all commands, example **`docker network ls`** will display all existing networks including 3 pre-defined networks.
- Containers are put into a network with the `--network` option in the `docker run` command.
- Existing containers can be put into a network with **`docker network connect <network name> <container name>`**



LAB - Docker Network

Containers are isolated by default, so you can put them in a docker network to share access

Let's connect Wordpress to MySQL by using a ***user-defined*** Docker network!

LAB - Docker Network

1. You'll need a MySQL Docker container to be the DB for WP site
2. Containers are isolated by default, so you can put them in a docker network to share access - **docker network create my-wp**
3. Create the MySQL container and specify some setup like DB name and login info: **docker run --name wp-mysql -e MYSQL_ROOT_PASSWORD=root -e MYSQL_DATABASE=wp -e MYSQL_USER=user -e MYSQL_PASSWORD=admin --net=my-wp -d mysql:latest**
4. Run a WP container that connects to the mysql container: **docker run --name wordpress -e WORDPRESS_DB_HOST=wp-mysql:3306 -e WORDPRESS_DB_USER=user -e WORDPRESS_DB_PASSWORD=admin -e WORDPRESS_DB_NAME=wp --net=my-wp -p 8011:80 -d wordpress:5**
5. Finish setting up the site and confirm you can make posts!
6. Remember: the DB data is (currently) only stored in the container, so if you delete the container your data goes away!

Advanced Containers - Docker Compose

 **Develop**Intelligence

A PLURALSIGHT COMPANY

Docker Compose

In the last lab, we saw how to create a docker network manually. Imagine if we had more than 2 containers - the command could get quite long!

Is there a simpler, cleaner way to create a network? Answer: **yes, with Docker Compose**

- Compose is a lightweight way to run your application without the overhead of a Swarm or Kubernetes application.
- Compose is a tool for running multiple related containers together with one command. It is included with Docker by default.
- So, you don't have to start each container separately
- You specify all the configurations in **docker-compose.yml** file.
 - This is another example of **infrastructure-as-code!**

Docker Compose

- compose creates a new user defined network bridge (just like we did manually in last lab) network for your containers
- This way they containers can communicate with each other by name. Or you can specify your own.
- You can refer to an existing image in Docker Hub, on your local machine, a private repo, or you can refer to a Dockerfile location, and ***Compose will build the image for you before running the container.***
- To start up your app, use **docker-compose up**
- To stop it, use **docker-compose down**
- It will automatically look at the **docker-compose.yml** file



docker-compose down

- this command **will stop running containers**, but it also *removes the stopped containers as well as any networks* that were created.

Docker Compose - sample docker-compose.yml file

version: '3.3'

services:

wordpressapp:

image: "wordpress:2.0"

restart: always

ports:

- "8089:80"

mysqlldb:

image: "db:2.0"

restart: always

ports:

- "3306:3306"

environment:

- MY_ENV=my_value

Compose will do 3 major things:

1. create a network so our containers can communicate
2. create the image (by building the wordpressapp Dockerfile), and then run that image to create the container
3. create the container from the mysqlldb image

Lab: Docker Compose



LAB - Docker Compose

- Create a new Docker Compose file
- Use two Docker Hub images. One for Wordpress, and the other a MySQL image.
 - Review the parameters we passed in when we ran the command manually in the last lab
 - You'll need to review the documentation for the images to find out what important options to pass the containers.

Bring the stack up using **docker-compose up** and try to use Wordpress blog

Advanced Containers - exec, it

 **Develop**Intelligence

A PLURALSIGHT COMPANY

Interactive Shell

- Containers are mostly intended to run as headless without interaction.
 - headless == no interface, not bash shell, etc..
 - Sometimes however you may want to interact and provide input
- To interact, you can run attached or attach later to interact with it.

`docker run -it <image-name>`

`docker attach <container-name>`

- You can now get it's output (like with docker logs), provide input if necessary.
- If you attach at run then to detach without stopping it, use the detach sequence (hold CTRL, press p then q), CTRL+ C will end the container process.
- We will see how to 'login' in the container next, and how to install software

Exec command

- With a **running container**, you can execute any arbitrary process available on the container using **docker exec**
- Frequently you can use this to “log in” to a shell. This is dependent on the image the container was built from. example: `/bin/sh` or just `bash`
- This is much like SSH connection to a server
- Once inside, you can look around and inspect your container
- This is very useful for debugging. You’ve built an image and you’re not sure if the image looks like you expect. So you can get into a running container and view things.
 - We saw how to do this earlier with `docker run -it <image name> /bin/sh`
 - differences between `run` and `exec` in next slide
- The location of the shell process depends on the container.
- **docker exec -it <container name> bash**

Docker run vs docker exec

- docker run:
 - runs an image to create a container
 - you can specify you want to create a shell to connect to the running container by doing `docker run <imagename> /bin/sh`
- docker exec:
 - can NOT use docker exec with an **image**
 - has to be a RUNNING container
 - can't be a stopped/paused container
 - Can connect a shell to a **running** container:
 - so if you do “docker ps” you can find the container name/ID, then do a “docker exec <container name> /bin/sh
- To summarize: docker exec is to be used with running containers, docker run is to be used with images (and it also pulls images if they don't exist locally)

LAB - docker exec

- Start a new container from the nginx image on Docker Hub
 - Use docker run but don't specify the -it param
- Using exec, get into the container and have a look around
 - docker exec -it <container name> bash
- Visit the web site and make sure you see the default home page.
- Now go change the home page index.html file located in /usr/share/nginx/html/
 - Wait, how do you edit the file?
 - Try installing vim. This is a Debian OS, so use **apt-get install vim**. First update the package manager with **apt-get update**
 - apt – advanced package tool – which is used to install/remove software on various Linux distros
 - Now update the headline
- Verify that your changes show in the web site.
- Let's discuss "How will it impact the image?"
- Let's discuss "How do I persist the changes?"

Docker - security: containers and root

 **Develop**Intelligence

A PLURALSIGHT COMPANY

Running Docker containers as root

- By default containers are run as root in Docker
- Everything we've done so far has been with the root user
- To verify:
 - Run whoami inside a container - try apache:
 - `docker run -it httpd /bin/bash`

```
varoonsahgal@Varoons-MacBook-Pro-2 lib % docker run -it httpd /bin/bash
root@6d8a4096c73d:/usr/local/apache2#
```

```
root@6d8a4096c73d:/usr/local/apache2# whoami
root
```

What about images that we created?

- Even Docker images you create are run as root by default
- For instance, the image we created earlier in the class
 - Let's run it: `docker run -it myveryownimage:v1 /bin/bash`

```
varoonsahgal@Varoons-MacBook-Pro-2 lib % docker run -it myveryownimage:v1 /bin/bash
root@bb07c76c5879:/usr/local/apache2#
```

Fun fact - shell prompt signals current user info

- Also notice the “#” at the end of the shell prompt
 - That’s another signal that our shell is running as root
- If we see a “\$” instead then that means our shell is running as a regular user
- This is a Unix/Linux tradition

```
varoonsahgal@Varoons-MacBook-Pro-2 lib % docker run -it myveryownimage:v1 /bin/bash
root@bb07c76c5879:/usr/local/apache2#
```

What's the problem with ROOT?

- We often need to run as ROOT user to install software
- But, the issue is that leaving your containers running as ROOT leaves you vulnerable to attack from hackers
 - Larger attack surface area...
- But i thought containers are isolated from the host (the server on which they are running)?
 - Yes and no - hackers are always trying to figure out how to break the isolation between containers and host
- So, if container is running as ROOT, may give hacker access to the host machine or K8s node if running K8s
- It's not so much the damage they can do **inside** the container, but rather on the **host machine/K8s node**

Fixing the ROOT problem

- So, once we are done doing ROOT activities like installing software, we should change the default user used by our container
- How and where do we do this?
- In our Dockerfile:

```
3  RUN useradd --uid 10000 runner
4  USER 10000
```

Fixing the ROOT problem

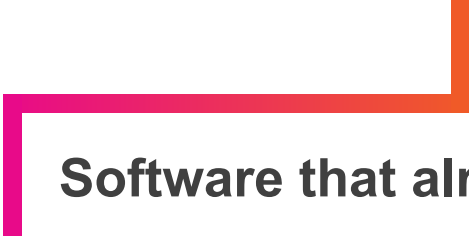
- First line creates a new user **without** ROOT permissions
 - Gives that user a name of “runner”
- The second line tells Dockerfile to switch to that newly created user

```
3 RUN useradd --uid 10000 runner
4 USER 10000
```


Fixing the ROOT problem - what's a UID

```
3 RUN useradd --uid 10000 runner
4 USER 10000|
```

- A **UID (user identifier)** is a number assigned by Linux to each user on the system.
- This number is used to identify the user to the system and to determine which system resources the user can access.
- UID 0 (zero) is reserved for the root.
- UIDs 1–99 are reserved for other predefined accounts.
- UID 100–999 are reserved by system for administrative and system accounts/groups.
- UID 1000–10000 are occupied by applications account.
- UID 10000+ are used for user accounts.



Software that already runs as ROOT

- You might be using other Docker images that run as ROOT
- To prevent that, you can derive from the base image from that software - so you can do a FROM thatbaseimage
 - and then add the non-privileged user account in Dockerfile

Docker security: Lab

 **Develop**Intelligence

A PLURALSIGHT COMPANY

LAB - from ROOT to non-privileged user

- Earlier we created our own Docker image (from Apache/httpd) that runs as root
- Now, we want to modify it to run as a non-privileged user
- Go ahead and add the lines below to the end of your Dockerfile
- Rebuild another image (using docker build), name it "mysecureimage:v1"
- Then, bring up a container with a shell using "it" mode and you should see the shell has a "\$" instead of a "#"
 - That tells you its NOT running as ROOT anymore
- And of course it no longer says "root", but rather "runner"

```
3  RUN useradd --uid 10000 runner
4  USER 10000
```

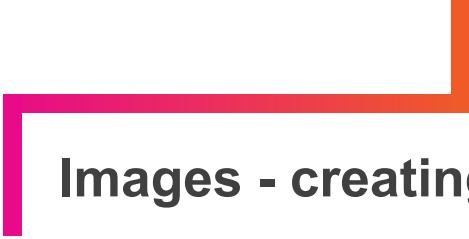
Advanced Images

 **Develop**Intelligence

A PLURALSIGHT COMPANY

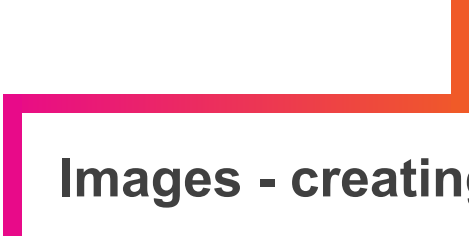
Images - creating from containers

- There's two ways to create a Docker image:
 - Dockerfile
 - create images from running container
 - This is like going backwards! Normally: Dockerfile -> image
-> container, but for now: running container -> image
- We can actually create Docker images from containers (***without*** using Dockerfiles), but can you think of a reason why we would **not** want to do this?
- Type answers in chat...



Images - creating from containers

- The primary reason not to do it this way is b/c we have then moved away from infrastructure as code (IaC)
 - b/c the Dockerfile is our infra as code - it can be version controlled and it tells us what base images we are using, what their respective versions are, what modifications have been made, etc..
 - when we create an image from a container, we can't look at the Dockerfile to tell us exactly what changes were made for that particular image
- However, doing this can be useful sometimes...



Images - creating from containers (not Dockerfile)

- This can be useful for small experiments and proof of concepts

Steps to create an image from a container...

The general procedure roughly looks like (docker run does both of these):

docker create ... (to create a new container)

docker start ... (this starts the container you just created)

NOTE THAT DOCKER RUN DOES BOTH CREATE AND START

Make changes in the container, install software, etc.

docker commit ... (this creates a new image without a tag)

- This command is new to us
- How we create an image from a container

LAB - create an image from a container...

Let's create an image from a container. We'll start with a base image from Docker Hub. Then customize it, and finally create an image from that.

1. First, create a new container using the Nginx image
 - `docker run -it nginx bash`
 - this will create a new container and also create a bash shell for you
2. Make some changes to the home page
 - The homepage is at `/usr/share/nginx/html/index.html`
 - How do you edit the file?
 - Try installing vim. This is a Debian OS, so use `apt-get install vim`. First update the package manager with `apt-get update`
 - Now update the headline
3. Then, create a new image from that container. You can name it "mynginx" AND give it a tag of v1

Use the **docker commit** command - read the docs:

<https://docs.docker.com/engine/reference/commandline/commit/>

4. Now, delete the old container.

5. Finally, create a new running container from your new image (mynginx:v1) and verify that it serves the home page you updated - which lives at `/usr/share/nginx/html/index.html`

Advanced Images multi-stage builds

 **Develop**Intelligence

A PLURALSIGHT COMPANY



Multi-stage builds

- We want to keep the size of our images down
 - Because if we have maybe 1,000 instances of our container, keeping size small as possible matters even more!
- Multi-stage builds help us do that..
- How does it help?
 - Because it makes it easier to get rid of any unnecessary build artifacts that may be created during the build process
 - sometimes intermediate artifacts like Java/GO/etc SDK and other things are leftover in the build
 - We do NOT want that stuff leftover in our final image, its not necessary and takes up space

Multi-stage builds - stages

- A Dockerfile is multi-stage if you see multiple FROM statements:
- This example below has 2 distinct stages
- The second stage references the first stage by using **COPY --from=0**
 - **0 is the first stage**
 - this says just grab what i need from the 0th (first) stage, then put it inside an Alpine OS

```
# syntax=docker/dockerfile:1
FROM golang:1.16
WORKDIR /go/src/github.com/alexellis/href-counter/
RUN go get -d -v golang.org/x/net/html
COPY app.go ./
RUN CGO_ENABLED=0 GOOS=linux go build -a -installsuffix cgo -o app .

FROM alpine:latest
RUN apk --no-cache add ca-certificates
WORKDIR /root/
COPY --from=0 /go/src/github.com/alexellis/href-counter/app ./
CMD ["/app"]
```

Multi-stage builds - naming stages

- We can instead name stages using “AS”
- Easier to read, and makes more sense especially if the instructions in your Dockerfile are re-ordered later, the COPY doesn't break:
- reference:<https://docs.docker.com/develop/develop-images/multistage-build/>

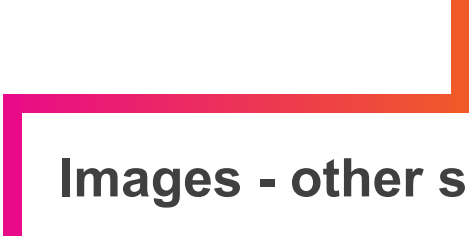
```
# syntax=docker/dockerfile:1
FROM golang:1.16 AS builder
WORKDIR /go/src/github.com/alexellis/href-counter/
RUN go get -d -v golang.org/x/net/html
COPY app.go ./
RUN CGO_ENABLED=0 GOOS=linux go build -a -installsuffix cgo -o app .

FROM alpine:latest
RUN apk --no-cache add ca-certificates
WORKDIR /root/
COPY --from=builder /go/src/github.com/alexellis/href-counter/app ./
CMD ["/app"]
```



Images - ALPINE

- A goal of image creation should be reduction of image size.
- Some images can be very large. Many images in Docker Hub exceed 1gb.
- Additionally, many standard distros contain security vulnerabilities. If you remove all unnecessary components, you reduce security flaws.
- Many Docker users choose to use Alpine Linux, which is an extremely stripped down Linux distribution. You get the bare bones Linux.
- Alpine is only 4-5 mb in size! Additionally, the runtime footprint is less.
- Due to the small size, this means deploying to a server is very fast with the small download size.
- Alpine is built to be secure. This does not however mean you should just assume it's good enough. Security teams should still scan it for vulnerabilities.
- You'll need to install any necessary software in your Dockerfile. You don't even get curl.



Images - other small images

- Newer images now exist for small images that bring the same benefits of Alpine, but in other possibly more friendly Linux distros. They offer a bit more balance of size and functionality than Alpine.
 - Ubuntu Minimal 18.04 is 29 mb
 - Debian Slim is around 88 mb

Docker Volumes

 **Develop**Intelligence

A PLURALSIGHT COMPANY

Docker Volumes - why

- In Docker, containers are stateless by default
 - Containers come and go - they are not meant to “stick around”
 - Imagine a scaling event - like a retail site on Black Friday
 - During such an event, many new containers are spun up
 - But then those containers are removed later on
 - They are like “seasonal workers” - they do their job for a short time, and when no longer needed they leave
- But often we need some persistent data for our applications
 - How do we get this?
- Docker **volumes** are the answer



Docker volumes - why

- A volume is basically some stateful data
- This data volume can be shared with multiple containers
- When we delete a container, that does not delete the volume



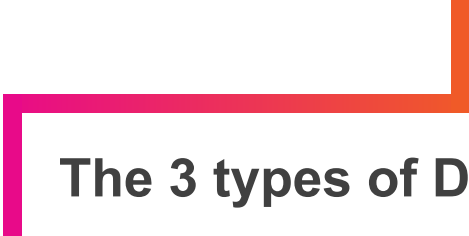
Docker Volumes: an example

- Suppose we have Jenkins (a popular build tool like Codebuild) running on Docker containers
- If our Jenkins containers come and go, how would the Jenkins data be persisted?
 - ie - data like our build jobs, our configurations in Jenkins
 - How does that data get saved if the containers running Jenkins can be deleted/restarted/etc. ?



Docker Volumes: when to use

- When the Docker host is not guaranteed to have a given directory or file structure.
 - Volumes help you decouple the configuration of the Docker host from the container runtime.
- When you want to store your container's data on a remote host or a cloud provider, rather than locally.
- When you need to back up, restore, or migrate data from one Docker host to another, volumes are a better choice.



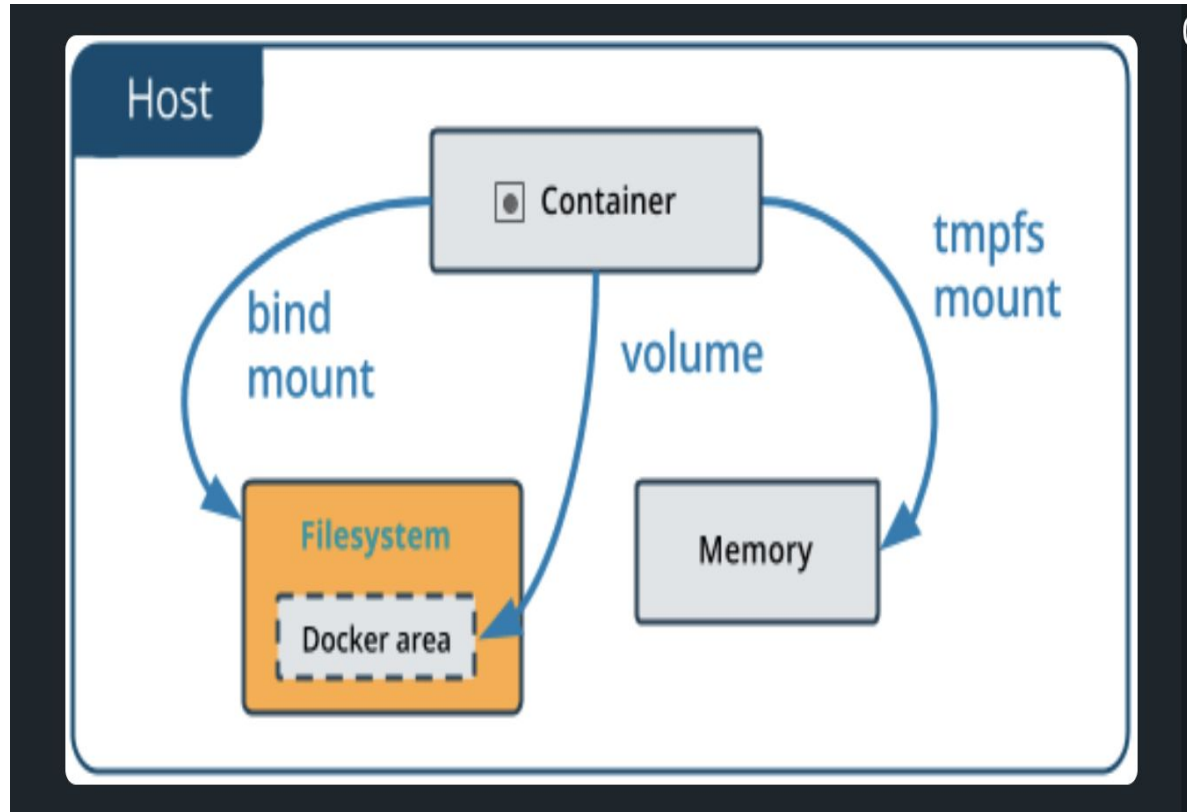
The 3 types of Docker volumes

- Host/Binding - aka bind mounts
- Named
- Anonymous (behave the same as “named” volumes it’s just that we don’t give them a name)

Bind mount versus volumes

- You may hear the term bind mounts
 - How do they compare to volumes?
- Volumes are managed by Docker
- Volumes - remember they put stuff inside the VM that Docker creates
- But bind mounts put stuff inside the actual host machine - NOT the VM created by Docker
 - in other words, if you are running Docker Desktop on a Mac, a bind mount will create a folder on your Mac OS - so you can actually see the bind mount in Finder (same for Windows)

Bind mount versus volumes





Bind mount versus volumes

- Per the docs: “***Bind mounts have limited functionality compared to volumes***”
- <https://docs.docker.com/storage/bind-mounts/>

Bind mounts - only good if consistent host environments

- Remember that Docker containers are meant to run in different environments
 - So, if you want to use a bind mount, best if you can guarantee the file structure
- imagine using a bind mount that points to folder1/xyz, which exists on one host machine but not on another
 - for that reason, bind mounts may not be the best solution
- Bind mount - creates a bind/connection between **host machine** and container **NOT** between the Linux VM created by Docker and the container
- <https://docs.docker.com/storage/>

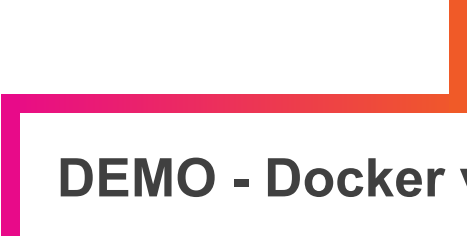
Docker (Named) Volume vs Bind Mount Syntax

- difference is very subtle
- with named volumes, we just specify a ***name***
- with bind mounts, we specify a ***path*** on the host
- backslash is to allow for new lines
- NAMED DOCKER VOLUMES:

```
docker run \  
  --name=nginxtest \  
  -v nginx-vol:/usr/share/nginx/html \  
  nginx:latest
```

- BIND MOUNTS:

```
docker run --name devtest \  
  -v "$(pwd)"/target:/app \  
  nginx:latest
```



DEMO - Docker volumes vs Bind Mounts



Docker Volume vs bind mount Syntax

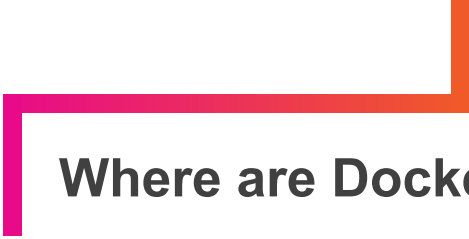
- The difference in syntax you can see is that with a bind mount we provide a path on our host machine - instead of the name of a volume

Docker Volume Syntax

- Can use the -v or --mount flag
 - --mount is more explicit/verbose
 - We will use the -v option
- We will use v flag to create volumes
- Can create a Volume outside context of container, ie:
 - **docker volume create myvolume**
- Docs here: <https://docs.docker.com/storage/volumes/>

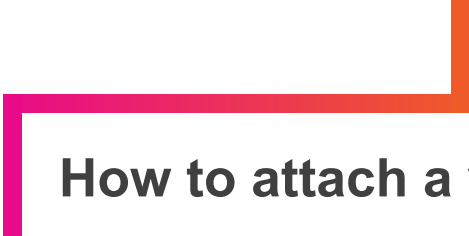
Docker Volume commands

- **docker volume ls** - to view the list of docker volumes on your host machine
- **docker volume inspect <volume name>** - to see the details of a specific volume
- You can also connect volumes to a remote host (like AWS EBS) using Docker volume plugins - see here:
https://docs.docker.com/engine/extend/plugins_volume/



Where are Docker Volumes stored?

- Remember that Docker Desktop creates a Linux Virtual machine
 - this is true whether you are on Windows or a Mac
- Volumes are stored inside that Linux Virtual Machine NOT on the “host”
 - in your case, if you’re on a Mac that is your “host” - the macOS
 - remember macOS is NOT Linux, and obviously neither is Windows
- So, **Docker is entirely responsible for managing the volume**



How to attach a volume to a running container?

- ***docker container inspect <container id or name>***
 - this will tell us if a volume is attached to a particular container or not
 -



Is a volume attached to a container?

- ***docker container inspect <container id or name>***
 - this will tell us if a volume is attached to a particular container or not
 -

Docker Volumes Lab



LAB - Jenkins and sharing data via volume

- First, setup a volume and attach it to a Jenkins image
- Make some changes in the Jenkins image...
- Then, create another Jenkins container instance and attach it to the same volume
- Be sure to change the ports, but keep the volume exactly the same
- In the second Jenkins instance, you should see the same changes you made in the first
 - This is because they share a volume



LAB PART 2 - Jenkins and sharing data via bind mount

- Now, do the same thing except with a bind mount instead of a volume
- Notice that a bind mount will automatically create a folder if one does not exist

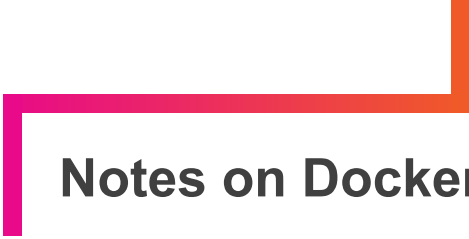
LAB PART 3 - replace with Docker Compose

- Finally, writing out these long commands is a lot of work
- Let's use Docker compose to get our Jenkins containers to share a volume
- Create 2 Jenkins containers in your docker compose file
- Have them share the same volume. Change stuff in one container and see those changes reflected in the other container
- Then, try using a .env file to store the path/name of the Docker volume
 - You may need to change Docker compose version to 3.4 or above
 - And grab that value from inside the Docker compose file
 - The “.env” file has to be in the same folder as the compose file
 - See : <https://docs.docker.com/compose/environment-variables/>
- Reminder: This is what attaching a volume to Jenkins looks like on a command line:
- ***docker run --name MyJenkinsContainer11 -v myVolume1:/var/jenkins_home -p 9092:8080 -p 40001:5000 jenkins/jenkins***



LAB PART 3 - replace with Docker Compose

- Take a look back at what we did earlier with Wordpress and Mysql - our Docker compose file from earlier
 - What if we want the database data and app-generated data to persist?
- We could use volumes in that file as well!
- The path to point your Wordpress volume to (inside the container) is `/var/www/html/wp-content`
- The path to point your MySQL volume to (inside the mysql container) is `/var/lib/mysql`
- We want to edit that file to get it to work with volumes now



Notes on Docker compose

- Note that with Docker compose if we do **docker compose down** it does not by default delete our volumes - just the containers and network
 - So, on the next **docker compose up** it will simply RE-use the volumes it created on the last run
- However, if we wanted to delete our volumes, we could do run **docker compose down --volumes** (double dash)
 - Use with caution - often we want our volumes to persist!

Kubernetes



What is Kubernetes?

- ❑ Container orchestration
- ❑ Derived from the Greek word for helmsman or pilot (hence the logo of a helm)
- ❑ Created by Google, open sourced in 2015

Kubernetes make containers *at scale* possible

K8s history



- ❑ 2003 - internal @ Google, they used a system called Borg to help with cluster management
- ❑ 2013 – Docker released, makes containers easier than they've ever been by providing a solid ecosystem – docker images
- ❑ 2015 – K8s version 1.0 is open sourced/released and also CNCF formed at same time
- ❑ Some of CNCF founding members include Google, Twitter, Docker, VMWare, IBM..
- ❑ Goal of CNCF to advance adoption of container technology
- ❑ Amazon's ECS released 6 days after K8s
- ❑ ECS not open source like K8s (EKS is how we use K8s in AWS)
- ❑ ECS – EC2 Container Service
- ❑ Meant to work with other Amazon services

Container orchestration - why?





K8s terminology: pod, node, cluster

- ❑ Docker Container/Image – wrapper around an application
- ❑ Pod (k8s) – wrapper around a container - a single copy of your application
- ❑ Node – collection of pods. Minikube runs a single node k8s cluster on your virtual machine. Think of your VM as a single 'node' right now
- ❑ Cluster – collection of nodes. Analogy: server rack
- ❑ Hierarchy is: your (Java, python, Node.js) application -> Docker image -> deploy to k8s (eks)-> pod (Docker container) -> node (EC2 instance)-> cluster

A note on Docker Swarm

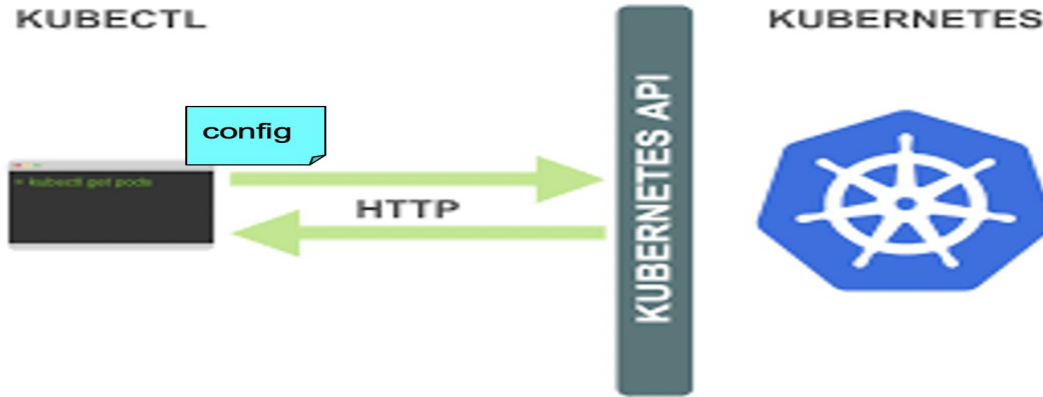


- ❑ Docker Swarm is also a container orchestrator. By Docker of course
- ❑ But k8s is far more popular and “enterprise-grade”
- ❑ Even Docker somewhat conceded by including k8s within Docker Desktop

Kubectl



- ❑ CLI for running commands against k8s clusters
- ❑ K8s exposes an API that we can call into



Kubernetes Architecture



Kubernetes can be divided into *planes*

- Control plane - the actual pods that the kubernetes core components runs on, exposing the api, etc...
- Data Plane - everything else meaning the nodes and pods which the application runs on



K8s Architecture - more

- ❑ In short, the ***control plane*** in k8s is what monitors the cluster, schedules the work, scaling the applications, makes changes, etc...
- ❑ Whereas the node are what actually do the real work, report back to the control plane and watch out for changes



K8s control plane components

- ❑ **etcd** - key-value store that stores config data/state of k8s cluster (something roughly like a .tfstate file)
- ❑ **api server** - provides REST interface for managing k8s cluster
 - ❑ allows us to connect to k8s via **kubectl**
- ❑ **scheduler** - schedules pods onto nodes



K8s non-control plane components

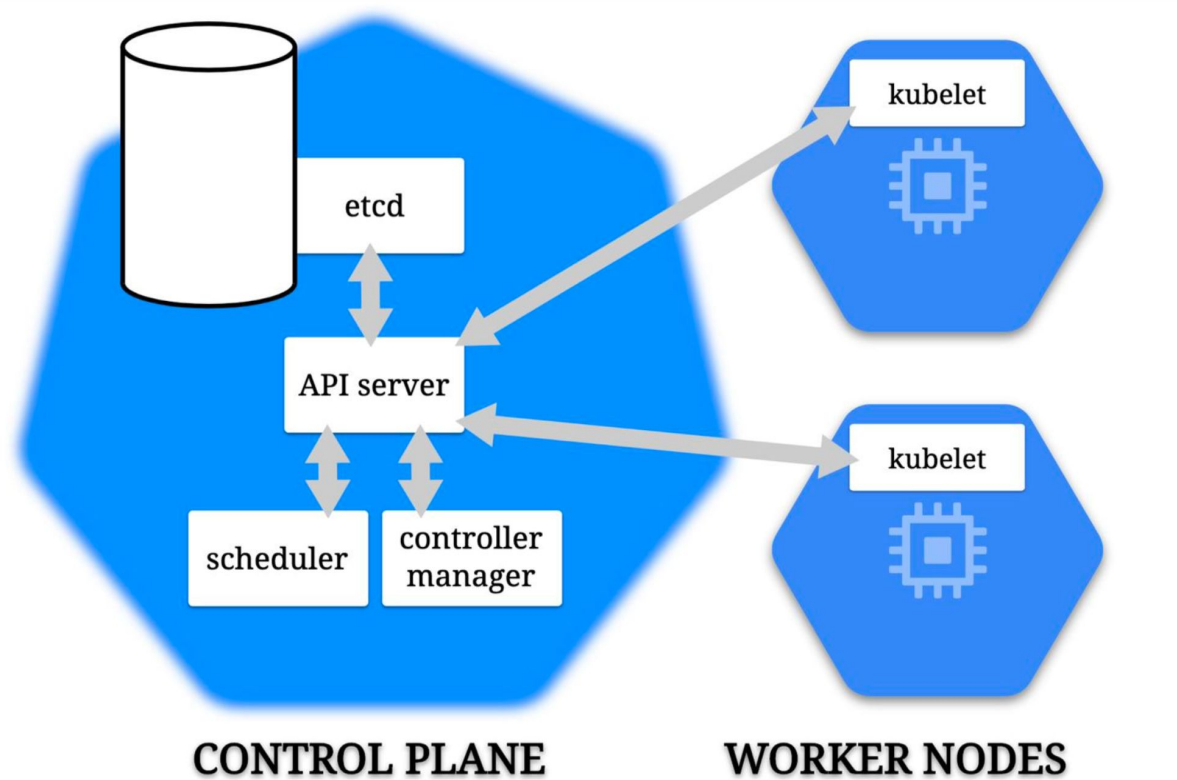
- ❑ **nodes** - worker nodes are the machines that run your applications
 - ❑ in EKS a node is an EC2 instance - so 1 node == 1 EC2 instance
 - ❑ on each node it's running the container runtime, Docker in our case
 - ❑ each pod lives inside a node
- ❑ **kubelet** - runs on each node in the cluster and is responsible for managing the pods on that node (think agent)

K8s cluster

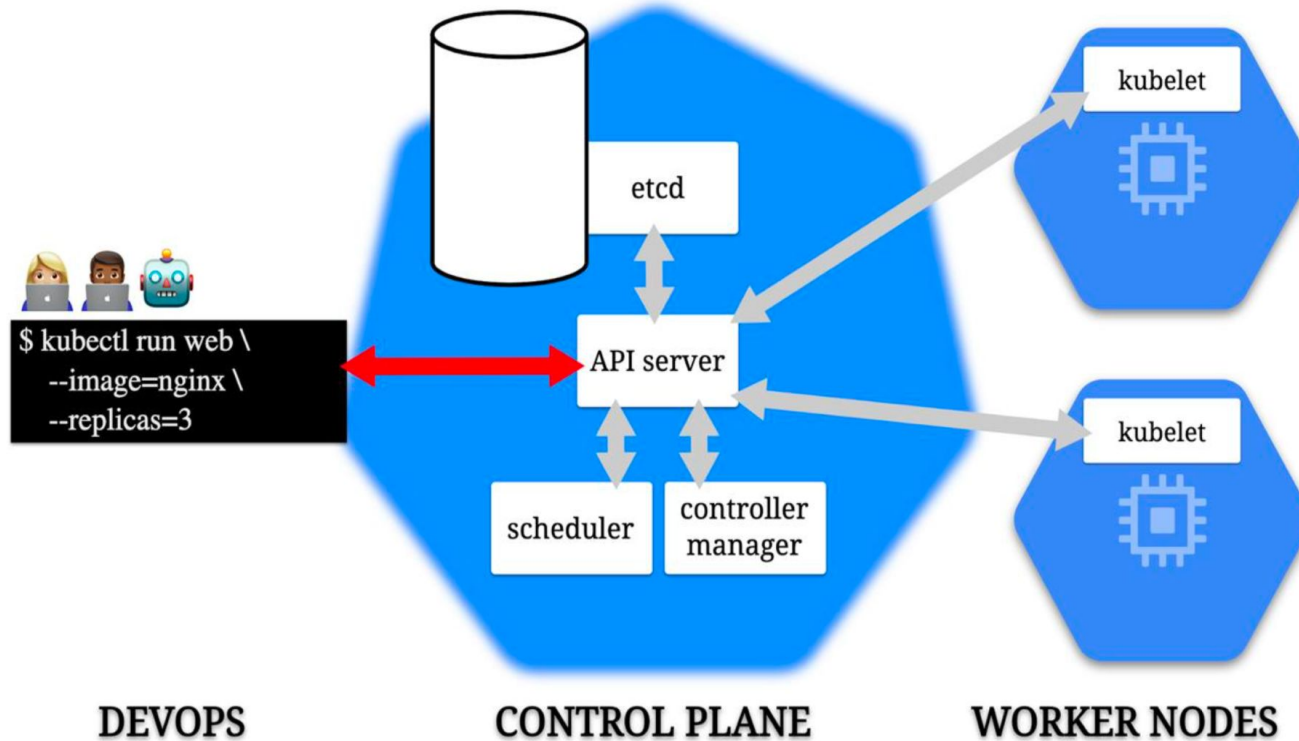


- ❑ **cluster ==** a collection of nodes
 - ❑ think: collection of EC2 instances if we are using EKS
 - ❑ how many nodes in a cluster?
 - that really depends on the size of your workloads
 - start small and scale up
 - ❑ k8s has support for clusters comprised of 5,000 nodes
- ❑ hierarchy in k8s is pod->node->cluster

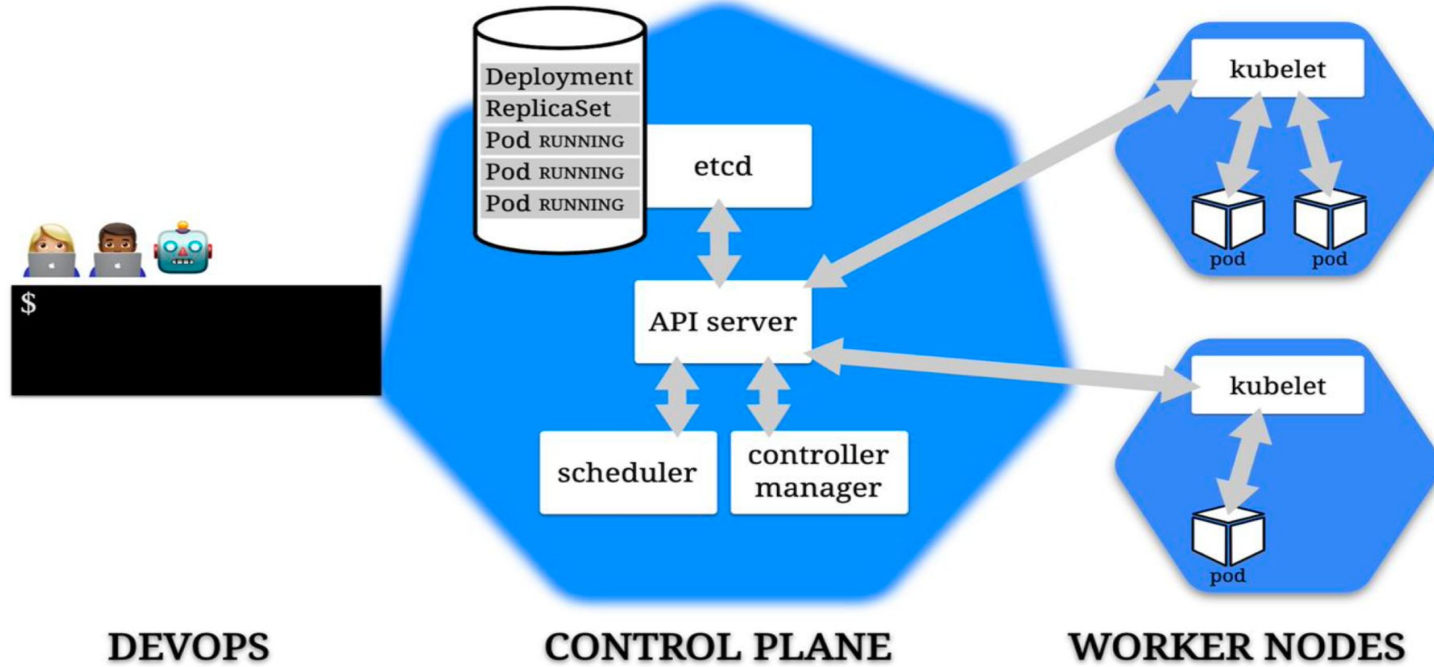
K8s Architecture



K8s Architecture



K8s Architecture





Kubectl commands

- ❑ Common Kubectl Commands:
- ❑ Apply – apply a template file to your cluster, its like create/update:
kubectl apply -f workloads.yaml
- ❑ Get<service, pod, etc> <name> - get information about the resource -
kubectl get pod mypod
- ❑ Get all – return a full page of details about your cluster's resources
Describe – get detailed information about a resource
 kubectl describe service myservice
Logs – return console output from a pod (or a container inside it)
 kubectl logs mypod
In production, you may also want to provide a namespace for any and all
kubectl commands:
 Kubectl -n <namespace> logs mypod



Kubectl – connect to a pod

- ❑ Common kubectl Commands
- ❑ Exec – launch a bash shell inside a pod:
 - ❑ `kubectl exec -it <pod-name> -- /bin/bash`
- ❑ To connect to a service (we'll talk about services shortly):
 - ❑ **`kubectl -n <namespace> get svc sample-service`**

Walmart case study



- ❑ You have an application - Java, python, whatever...
- ❑ CG says: containerize it and run it in k8s, b/c there's advantages
- ❑ What is the first step to getting it to work?
 - 1. Add a Dockerfile to create the Docker image...
 - 2. How do we go from Dockerfile to Docker image?
 - We run docker build to create the Docker image
 - The question is - who/what will run the docker build command?
 - build scripts
 - CG: uses bamboo for CI, some teams use Jenkins
 - Why don't we just manually run "docker build"
 - Can't keep track of build artifacts (like Docker image)
 - It's easier to automate versus manually running builds - especially if you run a lot of builds
 - Good for versioning, it's faster, you can automate unit tests - HUGE
 - vulnerability scanning of builds as well
 - At the end of all the build, we up with a Docker image assuming the build goes OK
 - Where does the Docker image go?
 - an artifact repository which stores images for maybe all applications within CG
 - Nexus, Bamboo provides one, Artefactory
 - 3. Ok, my docker image works and is in an artifact repository - so how do we get it to K8s?
 - We need something automated that will take the Docker image and deploy it to K8s
 - Hey k8s, here's an updated version of the Docker image
 - That can be a lot of different tools
 - Walmart uses something called Concord
 - basically something that will run these kubectl commands for you in an automated fashion...
 - Terraform
 - Cloud Formation



Kubectl – example - get all

- ❑ Let's run kubectl get all, what do we see...
- ❑ It shows me both pods and services I currently have running:

```
vn0silf@m-c02xr115jgh7 wcnp-deep-dive % kubectl get all
```

NAME	READY	STATUS	RESTARTS	AGE
pod/hello-pod	1/1	Running	8	26d
pod/nginx-pod	1/1	Running	14	61d

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service/kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	62d
service/nginx-service	NodePort	10.106.89.56	<none>	80:30080/TCP	60d

K8s pods



Pods are lowest/smallest unit of work within K8s

Containers at a high level are wrappers around your applications

Pods are another wrapper around your containers

Your application -> Docker container -> K8s pod

Typically just one container per pod

Pods are mortal

This is key, so remember this!

K8s – more on pods



So, K8s doesn't manage containers – it manages pods

Every pod gets a unique IP address

That IP address is not available outside the K8s cluster

So, by default, pods are not accessible outside the k8s cluster

In practical terms that means we can't directly access the application running in our container

For instance, if we are only running a pod with a Jenkins Docker image we can't open our browser and access an endpoint with our running instance of Jenkins

Lab 1: KUBECTL + pod exercise – apply

1. First, ensure that you have Kubernetes up and running via minikube - you can run `minikube start`
2. Copy the contents from the `pod.yaml` file here locally in a new folder (just name the folder `k8s`):
<https://github.com/varoonsahgal/k8s/blob/main/pod.yaml>
1. Use **`kubectl apply -f pod.yaml`** to apply that file – basically that creates a pod with the Docker image specified
2. Let's do a **`kubectl get all`** or **`kubectl get pods`** to view it
3. Use **`kubectl describe`** to get more info about the pod
4. Now, lets launch a shell inside our pod to play around:
 1. Run `kubectl exec -it my-nginx-pod -- /bin/bash`

Lab 1: KUBECTL + pod exercise – apply

5. Now, let's launch a shell inside our pod to play around:

1. Run `kubectl exec -it my-nginx-pod -- /bin/bash`
2. Once inside the pod, try running `curl http://localhost`
 1. But, it will complain that curl is not found!
 2. So, we can use apt – advanced package tool – which is used to install/remove software on various Linux distros
 3. Now, do **apt-get update**
 4. Then, run **apt-get install curl**
 5. Finally, try to **curl localhost** again
 6. You should get a response this time w/ the nginx default html!
 7. Type “exit” to leave the shell

6. Question: If I delete the pod, do another apply to create a new one, will the curl library still be there inside the new pod?

7. Takeaway from lab: we don't yet have a way of accessing the pod from our own browser – we haven't exposed it through a service. That's coming up!



KUBECTL example - describe

With describe we can get a lot more information about our pods

Much like "inspect" in the Docker CLI

Let's run kubectl describe on our new pod: `kubectl describe pod/my-nginx-pod`

Notice ClusterIP for the service - this is the IP address assigned to the pod available WITHIN the cluster not OUTSIDE of the cluster

In other words, you can't use this IP address to connect to the cluster from outside world

Pods can exist without services, but they will NOT be accessible, so let's see how to access one

And a service without a pod is just an empty endpoint - will give you a "host not found"



KUBECTL delete

Let's delete our pod

Kubectl delete -f pod.yaml

We can delete it by specifying the file used to create it via apply

We can also delete by name: **kubectl delete pods/pod-name**

Can we "stop" pods in K8s like we stopped containers in Docker?

Not really, as that concept doesn't transfer from Docker to K8s pods

Closest thing in K8s is to set replicas of a pod to 0 (we'll cover this soon)

Go ahead and apply the pod again as we will need it for next lab –

kubectl apply -f pod.yaml

K8s + Docker images

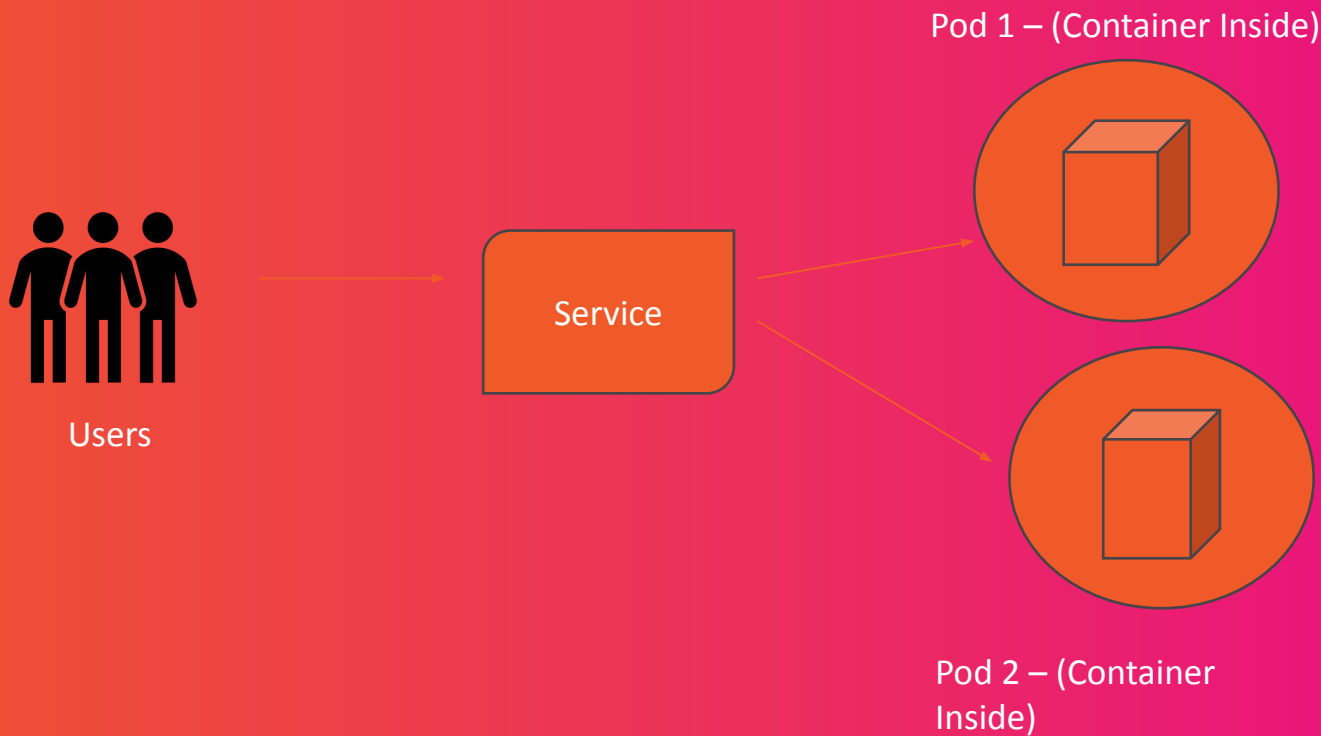
- Where does k8s look to find images?
 - It will first look at your machine to see if you have any images
 - If it doesn't find it there, then it will pull it from DockerHub
 - You can also specify the repo where it looks
 - Remember we have this in our pod.yaml file
 - (See containers section in image on right):

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
  labels:
    app: nginx
spec:
  containers:
  - name: nginx-container
    image: nginx:1.17.0
```

K8s – services

- So, how do we expose our pods to users? Through something called a **service** in K8s
- Why services? Why not just access pods directly?
- Because pods are mortal
 - K8s often needs to kill pods – an app might be over consuming resources
 - Or when we are scaling up dramatically, new pods are added and removed at a fast pace, which makes having a stable endpoint even more important
- So, **services** give us a stable, single point of entry into our pods...

K8s – services: expose pods



K8s - services

- Like pods, services are another K8s construct
- Services are what essentially expose our pods – which hold our applications - to the outside world
 - So services are a gateway to our applications in K8s
- Now, if pod dies, service will handle the IP address mapping between the endpoint that the user hits and the new pod replacing it
- You can think of services as load balancers for your pods, which distribute traffic across pods

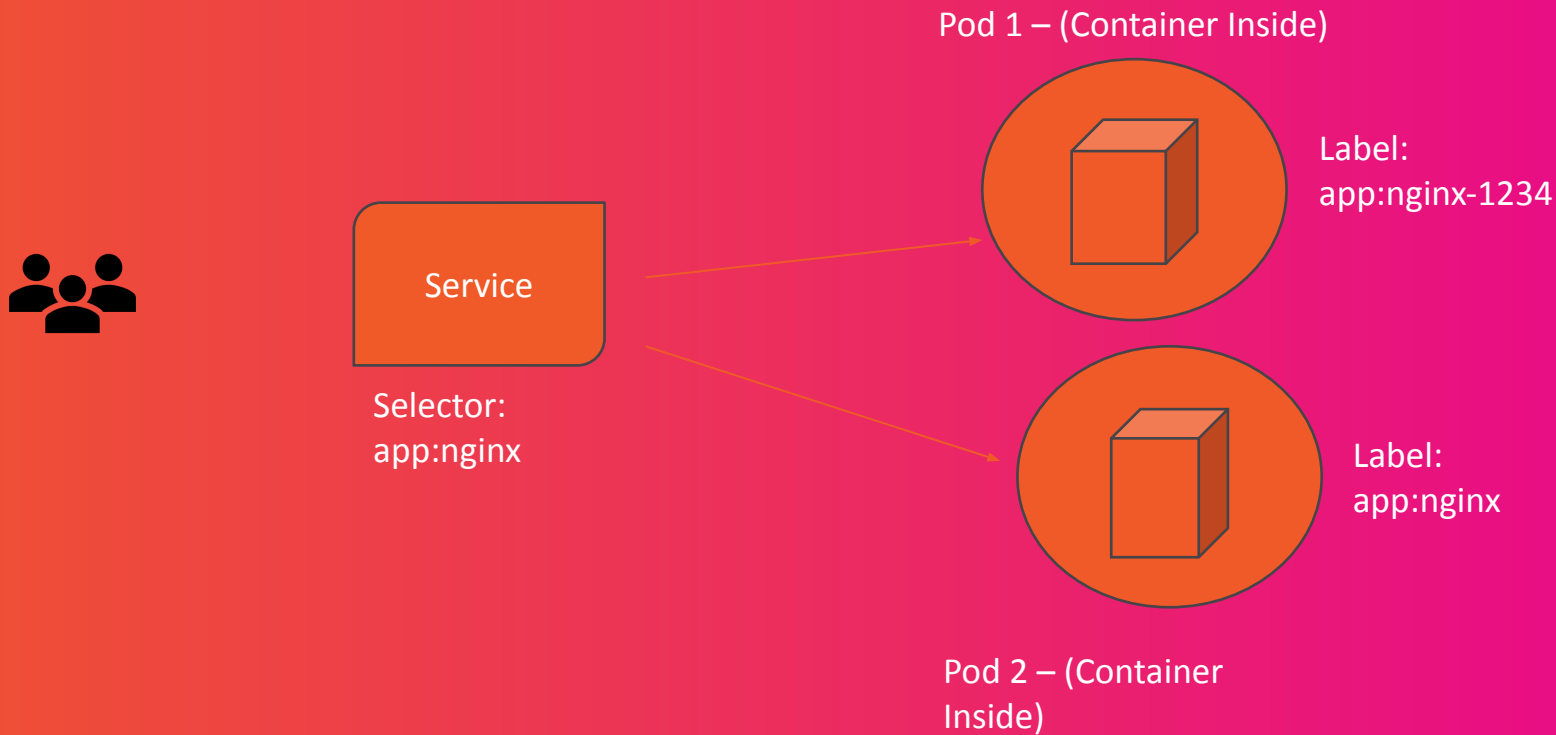
K8s – service types

- ClusterIP – default option. Provides access inside the cluster. Good for services that should remain internal to the application like a database.
- NodePort – Provides a port outside of the cluster. Makes your service public.
 - A ClusterIP Service, to which the NodePort Service routes, is also automatically created when you use a NodePort.
 - Note: the port you expose on the cluster is only valid from 30000-32767 –
 - **We will be using this service type in our lab to expose a pod that we create!**
- LoadBalancer – Exposes the Service externally using a cloud provider's load balancer.
 - Only valid on cloud implementations, and creates a load balancer and attaches it to your service.

K8s labels– services + pods

- How do we tell a service to forward traffic to a pod?
- Through labels!
 - Labels are in key:value format
 - Pods can have multiple labels
- So, a service would have a label selector and then a given pod will have a label which would match that label selector
 - Which in turn means that the service will forward traffic to that particular pod!
- Basically, the service says “all pods on the cluster with these labels are mine and i will load balance them!”

K8s – labels in services + pods



K8s – Service.yaml

```
! service.yaml
1  kind: Service
2  apiVersion: v1
3  metadata:
4    name: my-nginx-service
5  spec:
6    selector:
7      app: nginx
8    ports:
9      - protocol: TCP
10        port: 80
11        nodePort: 30001
12    type: NodePort
```

- Notice the selector: **app:nginx** – in our pod.yaml we also defined a label saying **app:nginx**
 - Because the selector matches our pod.yaml label, K8s will know to put our pod behind this service!
 - NodePort is the port which we can use to access this service from browser
 - Remember port mapping in Docker? This is same idea except with K8s – nginx is running on port 80, and we are exposing it on 30001

Lab 2: K8s services

1. Our goal here is to apply a service locally and connect it to our nginx pod. That way, we can access nginx through our browser
 1. Remember – pods are not directly accessible outside of K8s cluster, we must expose them through a service!
2. Look at the service.yaml file here:
<https://github.com/varoonsahgal/k8s/blob/main/service.yaml>
 1. Make a copy of that file locally in the same directory as your pod.yaml file
3. Now, apply the file to create the service in K8s! Run **kubectl apply -f service.yaml**

Lab 2: K8s services pt 2

4. You should now be able to view nginx running from your browser – go to <http://localhost:30001>
 - So, our service is exposing our pod, which in turn is running the nginx Docker container! Beautiful!
5. Try describing the service via kubectl to get more info – **kubectl describe service/my-nginx-service**
6. Try deleting the pod – **kubectl delete -f pod.yaml**, and now try accessing the service in the browser – you should see nothing!
7. Go ahead and reapply the pod

K8s – sidecar

- Typically you would only put one container per pod
- In rare cases, a pod can have multiple (>1) containers if those containers are tightly coupled
 - The sidecar pattern is one example of when we would have multiple containers in a single pod
 - In this pattern a secondary container's only job is to support the primary container
 - For example, you may have a secondary container that just sends logs from the primary container to a central logging system

K8s – declarative model

- Example of a pod.yaml file:

10 lines (10 stmt) | 148 Bytes

```
1  apiVersion: v1
2  kind: Pod
3  metadata:
4    name: nginx-pod
5    labels:
6      app: nginx
7  spec:
8    containers:
9      - name: nginx-container
10      image: nginx:1.17.0
```

K8s – declarative model

- K8s uses infrastructure as code concepts
 - *Infrastructure as code (IaC) means managing your IT infrastructure using configuration files.*
- Declarative vs imperative:
 - Declarative is like asking someone to bake a cake and giving them the ingredients without a recipe – the baker will figure it out
 - Imperative is like giving ingredients + recipe - telling baker exactly what to do..ie here are the exact steps you need to take to bake a cake

K8s Replicaset

- What is self healing? When k8s automatically spins up a new pod when one goes down
- Replicaset and deployments are the two ways to achieve self-healing within K8s
- When creating a Replicaset, we don't need to define the pods as a separate resource – ie as a separate pod.yaml file.
 - Pods are created within the replicaset and are “owned” by that replicaset

Lab - K8s Replicasets

- Grab the replicaset file and apply it locally:
<https://github.com/varoonsahgal/k8s/blob/main/replicaset.yaml>
 - - **kubectl apply -f replicaset.yaml**
- You can see the pods created by the replicaset if you do a **kubectl get all**
- Try deleting a pod and see what happens – it should self heal and bring it back up (kubectl delete pod/pod-name)
- Look at desired, current and ready states!
- Your service should still be running
- Note that this replicaset is still attaching to our service through the label selector
- so if service is still running you should be able to see the nginx response in the browser by going to <http://localhost:30001/>
- Try deleting the service, do you still see the nginx homepage?
- Everything under template is specific to the pods its creating
 - Note that we don't explicitly use the word pod in the replicaset.yaml file, it's implied!

K8s Deployments – the what

- Deployments are another K8s construct (like pods, services, replicaset)
- Deployments actually create replicaset
- Why do we need deployments if they just use replicaset under the hood?
- Because deployments **allow us to rollback to prior application states** if something went wrong with our rollout
 - For instance the new version of our application does not work as planned, so we want to go back to the earlier working version. Very easy in K8s through deployments!
 - Plus we still get self healing because we are using replicaset(s) created

K8s - Triggering Deployments

- A new deployment will trigger a new replicaset to be created
- How do we trigger a new deployment rollout? By updating the pod template in the deployment.yaml file (like the labels or image)
 - We would then need to reapply the deployment.yaml as well
 - Note that if we update the number of replicas that does not trigger a “new” deployment, just an update to the number of pods
- If we have 1000's of replicas, it will take longer to rollout the new updates —so some users may see the older version of your application post-deploy until all pods have been updated

K8s Deployments – Rolling updates

- If we go from v1 to v2 of our image (our application), then there will be an intermediate phase where some pods are running on v1 and some on v2!
- This is what's known as a **rolling update**, and ensures that we don't experience any downtime!
- For our lab exercise, let's pretend we are “upgrading” our application from an Apache server to a Nginx server – so we will modify the Docker image being used in the deployment that we will create....
 - So, we will go from Apache -> Nginx

K8s – Kubectl Deployment commands

- You can rollback with: **kubectl rollout undo deployment apache-deployment**
- You can see rollout status with: **kubectl rollout status apache-deployment**
- You can see rollout history with: **kubectl rollout history deployment apache-deployment**
- You can pause and resume rollouts
- More details in the docs:
<https://kubernetes.io/docs/concepts/workloads/controllers/deployment/>

Lab - K8s Deployments

- Delete any replicaset/pods that you might have running via **kubectl delete** (but leave the service)
- Grab the deployment file here and apply it locally:
<https://github.com/varoonsahgal/k8s/blob/main/deployment.yaml>
 - - **kubectl apply -f deployment.yaml**
- Do a **kubectl get all** and notice that it creates a replicaset, 10 pods, and also a “deployment”
- Try deleting a pod and see what happens – it should self heal and bring it back up
- Look at desired, current and ready states!
- Your service should still be running
- Note that the **deployment** is still attaching to our service through the label selector – we still need a **service** to expose our pods!
- If service is still running you should be able to see the Apache response in the browser by going to <http://localhost:30001/>

Lab - K8s Deployments – pt 2

- Now, let's update our deployment.yaml file to use nginx image instead of httpd inside the deployment.yaml file. Go ahead and reapply the deployment file via **kubectl apply -f deployment.yaml**
- **This is the equivalent of doing a new release for our application!**
- Now, notice that a new replicaset has been created and the old one still remains
 - The old one is there to allow us to roll back if we need to!
- Go to <http://localhost:30001/> and you should see that it's updated to use the nginx homepage!
- Notice that you never experienced any downtime – because K8s is gradually terminating the old Apache pods and replacing them with the new nginx pods – it's not killing all 10 pods at once and then bringing up 10 new ones
- Also, notice that the old replicaset is still maintained – this is key for rollbacks!
- So, what if we want to rollback to our earlier deployment with the Apache container image?
 - Simple!
- Just run **kubectl rollout undo deployment apache-deployment**



K8s Best practice - using probes

K8s – Why do we need probes?

- Pods and their respective containers can misbehave.
- If a pod goes down (maybe due to a network issue), a deployment/replicaset can and will bring it back up to maintain a certain count
 - The replicas count if you recall from your deployment.yaml files
- However, what if our application is deadlocked? To K8s, the pod will still seem healthy and it will keep running unless we do something about that...
- So, by default, K8s has no way of knowing if our application (containers) are misbehaving
- But, we can fix that with probes! This is considered a **best practice** with k8s to ensure the “robustness” of your pods

K8s –probe types

- We will focus on 2 primary probe types: liveness and readiness
- **Liveness** probes basically are a check that runs periodically (you define how often) and asks your application are you alive, are you alive, etc?
 - If the answer is "no" k8s will **restart** your pod since that should bring your container/application pod back to life
 - If the answer is "yes" nothing will happen since your pod is healthy
- **Readiness** probes are similar except they are basically checks to see if your pod is ready to accept traffic
 - Sometimes our applications need to load large data or configuration files during startup, or depend on external services after startup.
 - In such cases, you don't want to kill/restart the application, but you don't want to send it requests either
 - So, readiness probes will prevent traffic from being sent to pods that are not ready to accept it.

K8s –combining readiness + liveness probes

- Note that you can (and often should) combine both liveness and readiness probes for a robust pod

K8s – Parameters for liveness/readiness probes

Both readiness and liveness probes have additional parameters

They are the same exact parameters for both probes...

And if you don't specify these parameters they do have default values as well:

initialDelaySeconds: Probes start running after initialDelaySeconds after container is started (default: 0)

periodSeconds: How often probe should run (default: 10)

timeoutSeconds: Probe timeout (default: 1)

successThreshold: Required number of successful probes to mark container healthy/ready (default: 1)

failureThreshold: When a probe fails, it will try failureThreshold times before deeming unhealthy/not ready (default: 3)

Liveness probe lab

- Go ahead and open this file : <https://github.com/varoonsahgal/k8s/blob/main/probe.yaml>

You can see that this Docker image is what's being used
<https://hub.docker.com/r/varoon10/teachingaid>

- teachingaid is a simple Docker image using nginx
- It is setup so that it will first sleep for 30 seconds, and then deletes the homepage automatically
- By deleting the homepage, the httpGet to the homepage on lines 12-14 will fail
- This triggers the livenessprobe
- The liveness probe will then automatically restart the teachingaid nginx pod/container

Liveness probe lab

- Now, copy the file locally and apply it using kubectl:
<https://github.com/varoonsahgal/k8s/blob/main/probe.yaml>
 - Just do a kubectl apply -f probe.yaml
- You should be able to access the service at port 30080
- Do a kubectl get all and pay attention to the pod's restart column
 - You can see that after 30 seconds the pod will be restarted by the liveness probe
- Do a kubectl describe liveness-http-nginx and you should see the restarts
- What would happen if we did NOT have a liveness probe? Try it out!

K8s – Liveness probe lab

Our example is a bit contrived in that we auto-delete the homepage in the teachingaid Docker image

But, imagine if a container/application is deadlocked – the pod is still “running” but not doing anything productive

With liveness probes we can check for that sort of thing and have K8s automatically restart it when need be!

Container Based Solutions in AWS

ECS, EKS, EC2, Kubernetes?



- ❑ Let's make sense of all these acronyms - and discuss the differences!

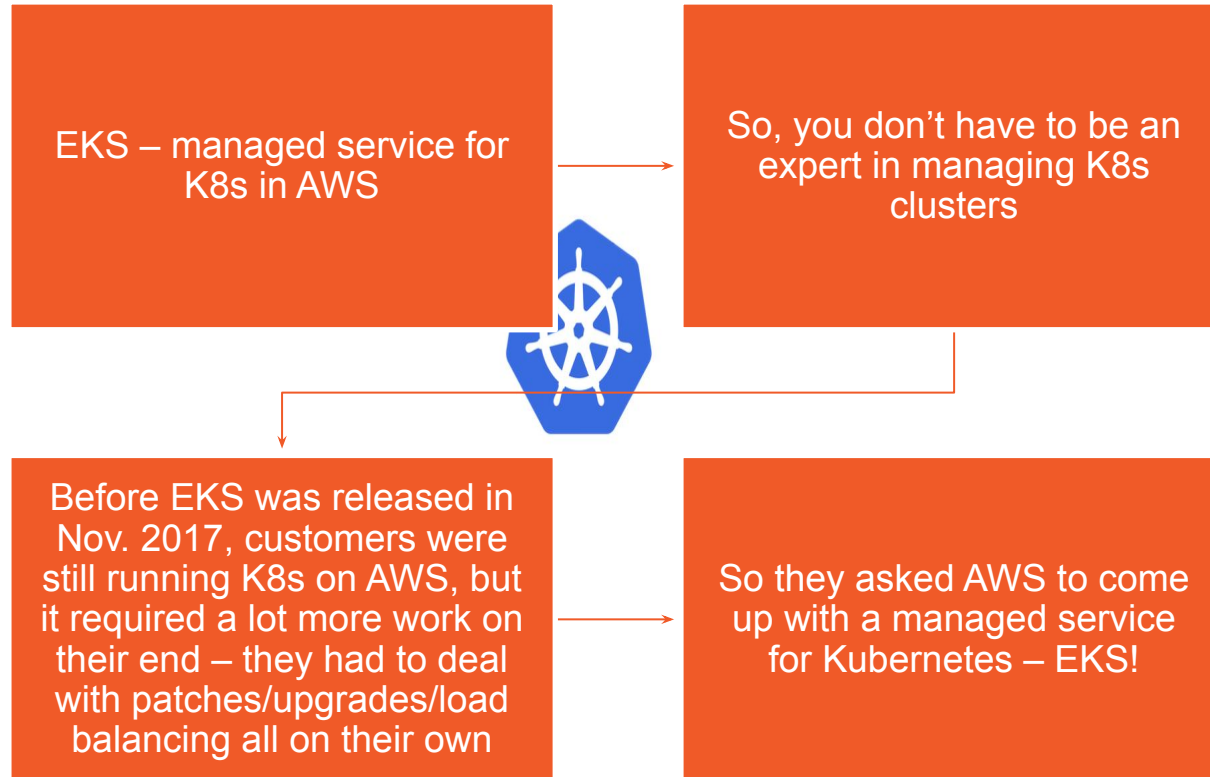


ECS vs K8s



- ☐ ECS (Elastic Container Service), like Kubernetes, is container orchestration (ie Docker containers)
- ☐ However, ECS is not open source – it's AWS proprietary
- ☐ Kubernetes *is* open-source
- ☐ Kubernetes was open-sourced July 21st, 2015
- ☐ ECS was released as a service just 6 days later July 27th, 2015
- ☐ Fargate was released in Nov. 2017

EKS – Elastic Kubernetes Service



ECS vs EKS



- ❑ ECS is generally easier to use than EKS
- ❑ ECS - simplest way to deploy containers in AWS
- ❑ ECS provides a higher level of abstraction than EKS
- ❑ EKS - lower level of abstraction than ECS
- ❑ For that reason, EKS offers more flexibility
- ❑ Both ECS and EKS can use Fargate

Role of EC2 with ECS and EKS



- ❑ Both ECS and EKS use EC2 instances behind the scenes
- ❑ EKS, since it's k8s, uses *nodes*
- ❑ Each node in EKS is equivalent to an EC2 instance
- ❑ Cluster == collection of servers (nodes == servers in the k8s terminology)

Fargate – ECS + EKS



- ❑ Fargate == serverless (does NOT mean there are no servers, rather the overhead of managing servers is taken care by AWS and abstracted away from you)
- ❑ Fargate allows you to run containers without having to manage servers or clusters of EC2 instances...
- ❑ Fargate can be used with both ECS and EKS
- ❑ Fargate is not a separate AWS service, has to be used with ECS or EKS

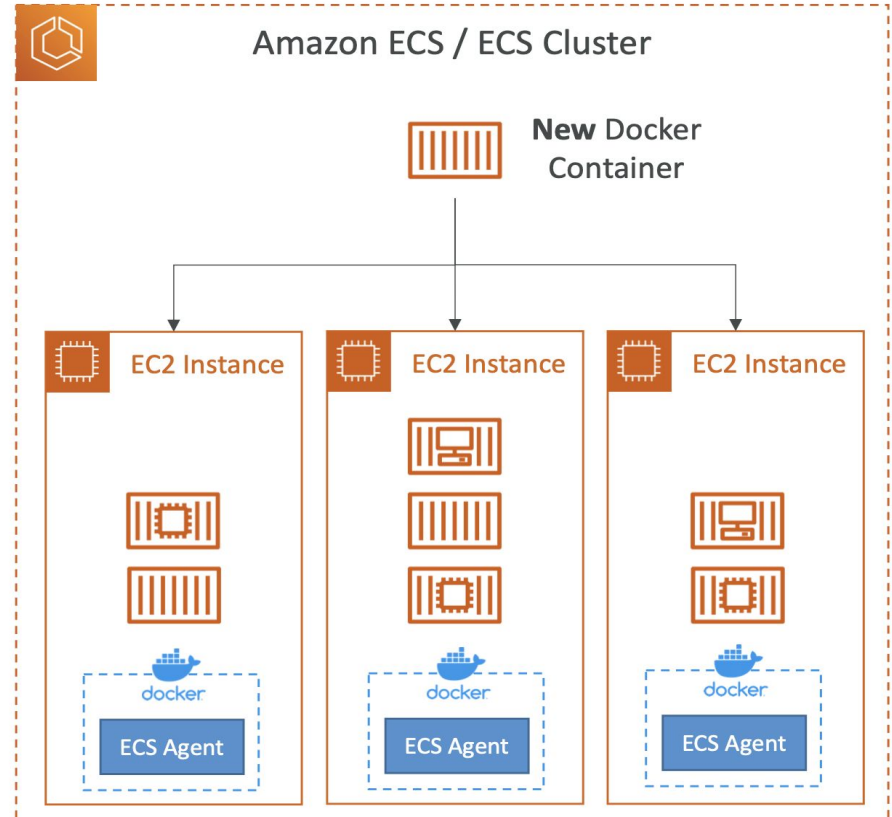
ECS – launch types



- ❑ There's essentially 2 ways of using ECS
- ❑ These are known as *launch types*
- ❑ 1. ECS + EC2
- ❑ 2. ECS + Fargate + EC2
- ❑ Let's explore and understand both options starting with ECS + EC2...

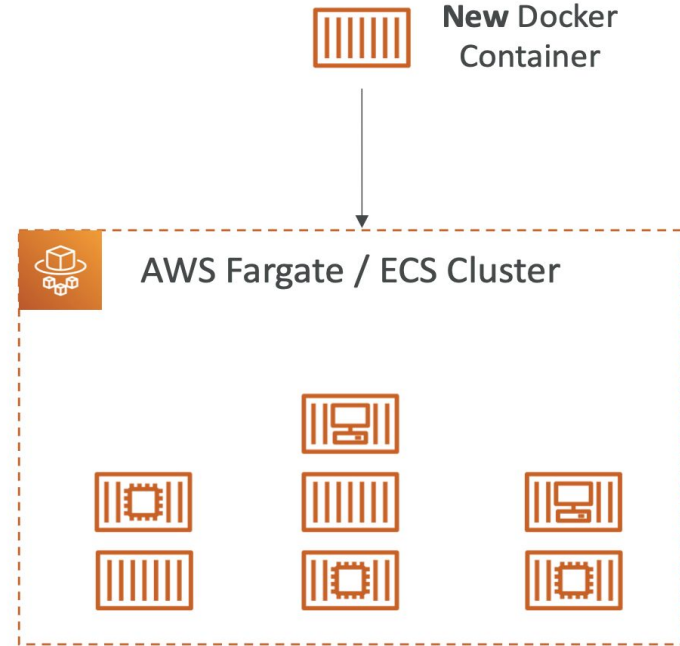
ECS + EC2 – option 1

- ❑ Your Docker containers will live inside EC2 instances (EC2 = ~ IaaS virtual machines)
- ❑ With ECS + EC2 – you have to provision the EC2 instances in **advance** – remember EC2 is IaaS – you maintain them as well
- ❑ ECS however does provide container orchestration for us – it starts and stops containers for us
- ❑ Note that each EC2 instance has an ECS Agent which basically says “this EC2 Instance is part of the ECS Cluster”



ECS + Fargate – option 2

- ❑ Fargate is considered *serverless*
- ❑ With ECS + Fargate, no EC2 instances
- ❑ So, you do NOT need to provision the infrastructure in advance – no EC2 instances to deal with. This is PaaS
- ❑ Then, to scale, you would just need to create what's known as task definitions
- ❑ Then, ECS will run those tasks for you
- ❑ Fargate is easier than ECS + EC2 launch type



ECS Labs – perform in order



- ❑ <https://docs.aws.amazon.com/AmazonECS/latest/developerguide/get-set-up-for-amazon-ecs.html>
- ❑ <https://docs.aws.amazon.com/AmazonECS/latest/developerguide/create-cluster-console-v2.html>
- ❑ https://docs.aws.amazon.com/AmazonECS/latest/developerguide/ecs_cwets.html
- ❑ <https://docs.aws.amazon.com/AmazonECS/latest/developerguide/tutorial-cluster-auto-scaling-console.html>

ECS Tasks – analogy to K8s



- ❑ Remember our deployment.yaml from the k8s lab?
- ❑ Tasks in ECS are actually quite similar to deployments in K8s
- ❑ Because in a task, like a deployment resource in K8s, we specify things like the Docker image to use, CPU/memory usage, etc..
- ❑ Remember that both ECS and K8s are container orchestrators – same goal, just different implementations

EKS – Key Points



- ❑ Also K8s is cloud-agnostic and supported by all major cloud providers like Azure/GCP - GKE, which is Google's equivalent of EKS in AWS/Amazon
- ❑ So using EKS instead of ECS gives an extra level of "safety" in case you want to change cloud providers
- ❑ Because again, ECS is AWS proprietary technology – other cloud providers can't use it unlike K8s

EKS vs ECS



- ❑ ECS is container orchestration, not open source
- ❑ EKS is K8s which is open source
- ❑ Both provide container orchestration...
- ❑ So, which one should we use?
- ❑ Sometimes migrating K8s from one provider to another then EKS may be best bet
- ❑ EKS good if you already running K8s before EKS introduced

EKS Hands On



- ❑ Execute the tutorial here:
- ❑ <https://docs.aws.amazon.com/eks/latest/userguide/getting-started-console.html>
- ❑ And another one here:
- ❑ <https://github.com/varoonsahgal/cg-cloud-foundations/wiki/EKS-Lab>

ECS – clusters



- ❑ ECS uses the term clusters just like Kubernetes
- ❑ To launch Docker containers using ECS, we use ECS tasks

Argo CD

 **Develop**Intelligence

A PLURALSIGHT COMPANY



Argo

- Automates deployment of infra to a k8s cluster
- Based on Gitops methodology
 - All deployment info stored in Git
 - Cluster is auto-synchronized with desired state in Git
- Consists of several components for Continuous Delivery



Argo

- API server is central component of the Argo CD architecture
 - Processes API requests from users, interacting with K8s server to deploy and manage application
 - also monitors state of the server, built on top of K8s API

Argo CD Lab

 **Develop**Intelligence

A PLURALSIGHT COMPANY

Argo CD Lab

- Go ahead and do this lab to get a feel for ArgoCD:
 - <https://github.com/varoonsahgal/argo-examples/wiki/Argo-CD-Lab>
 -

JVM Internals

Let's explore the JVM Internals!



JVM Internals!



What is -Xmx in relation to the Java heap?

- Xmx is the max heap size for the application
- Think of the mx as ***max***
- ***This is an argument we provide to the JVM***



JVM heap size - Xmx

- Xmx is the max heap size for the application
- Think of the mx as max
- If i set the xmx size to 2 GB then will my Java process ever exceed this size?
- The answer is yes: your Java process will consume more than the xmx limit of 2 GB



Does Java process memory utilization go beyond -Xmx?

- If i set the xmx size to 2 GB then will my Java process ever exceed this size??
- The answer is yes: your Java process will consume more than the xmx limit of 2 GB
- So, why even bother setting that xmx limit in the first place?
- To understand, let's look at the internal JVM regions of memory...

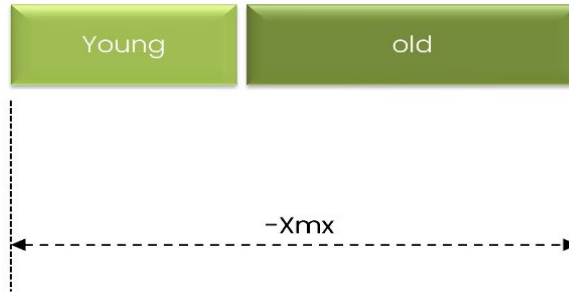
JVM memory regions - young and old generation

- There are 2 primary regions in the heap
- They are called the young and old generations
- If we create an object like “Car” or “Person” it will start out in the young area of the heap and when we’ve had it for some time it will be promoted to the “old” area
- Long lived objects go to old area of memory
- Long lived == they survived one or more minor Garbage collection



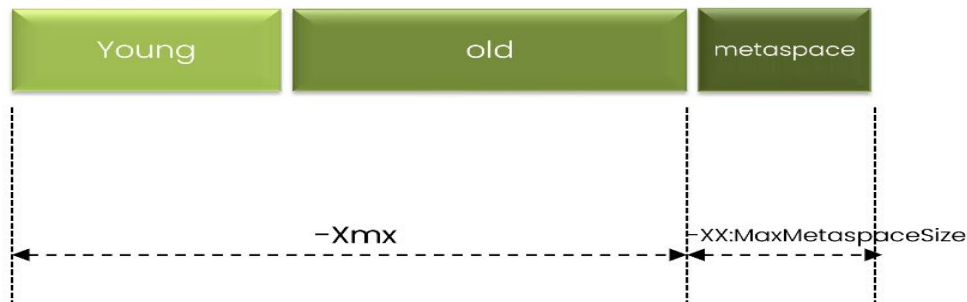
Xmx == Young + Old

- Xmx is a JVM parameter - used to specify the size of the heap for our application when we run it..
- These 2 regions together are the size we set when we configure the -Xmx
- Are there other regions?
- ***java -Xmx=250Mb MainApplication.java*** - this runs program and sets the max heap size to 250 Mb...Someclass x = new Someclass();



The metaspace region

- In addition we have the **Metaspace** region as well
- This is not part of the heap (which is Young + Old)
- This is where the class and method definitions used to execute your program live
- Note that the metaspace is outside of the -Xmx region:



Young: Newly created Objects

Old: New objects that survived one or more minor GC promoted here

Metaspace: Classes, Methods, metadata

Metaspace region

- If we set the Xmx equal to 2 GB, then the metaspace will take up additional space outside of that 2 GB
- Unlike the heap and the stack, the metaspace does not have a fixed size and can grow dynamically
- However, it is still essential to monitor its usage to avoid memory leaks and potential OutOfMemoryError exceptions.



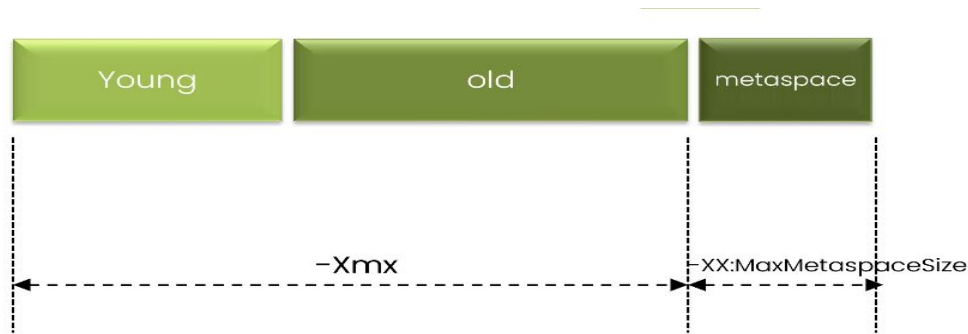
Young: Newly created Objects

Old: New objects that survived one or more minor GC promoted here

Metaspace: Classes, Methods, metadata

Where are threads stored?

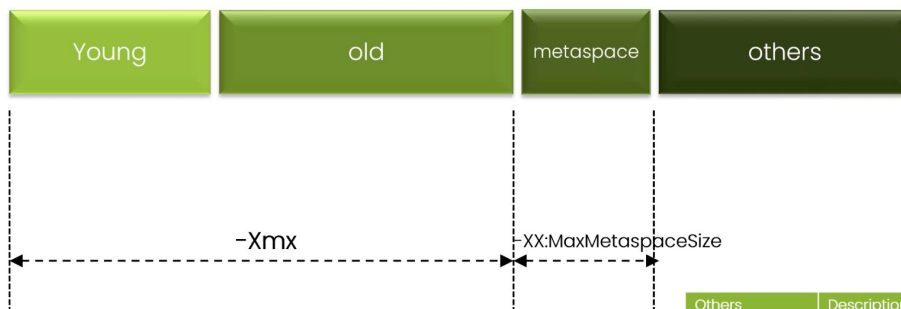
- Are threads stored in any of these 3 regions? Type your answers in chat...



Young: Newly created Objects
Old: New objects that survived one or more minor GC promoted here
Metaspace: Classes, Methods, metadata

“Others” region

- There is another regions that has no official name
- We will call it “Others”
- ***This is where threads are stored:***



Young: Newly created Objects

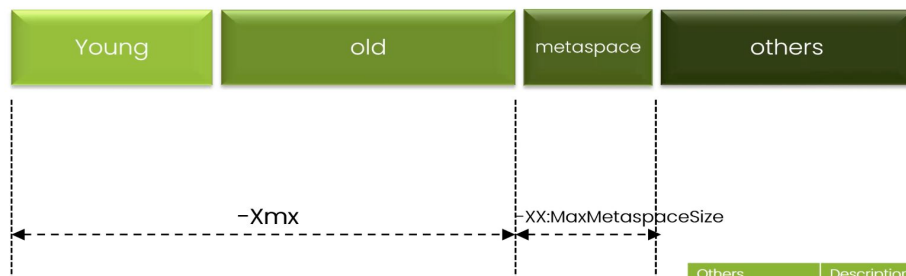
Old: New objects that survived one or more minor GC promoted here

Metaspace: Classes, Methods, metadata

Others	Description
Thread Stacks	Each thread has a separate memory space. Controlled by -Xss
Garbage Collection	Threads, Memory to store GC info
Code Generation	Converting bytecode to native code
Socket Buffers	TCP Connections (Receive buffer ~37k, Send Buffer ~2.5k)
JNI	JNI program also allocate memory

Garbage collection

- Garbage collector (GC) in Java belongs to the JVM
- GC frees up memory to prevent memory leaks and improve performance
- The GC itself needs memory to run
- Where does it get that memory?
- From the “others” region

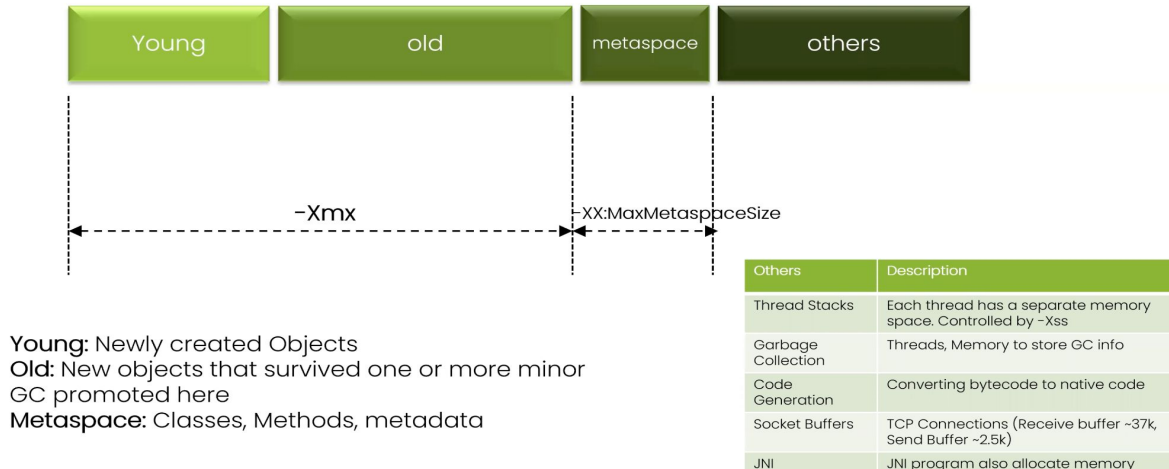


Young: Newly created Objects
Old: New objects that survived one or more minor GC promoted here
Metaspace: Classes, Methods, metadata

Others	Description
Thread Stacks	Each thread has a separate memory space. Controlled by -Xss
Garbage Collection	Threads, Memory to store GC info
Code Generation	Converting bytecode to native code
Socket Buffers	TCP Connections (Receive buffer ~37k, Send Buffer ~2.5k)
JNI	JNI program also allocate memory

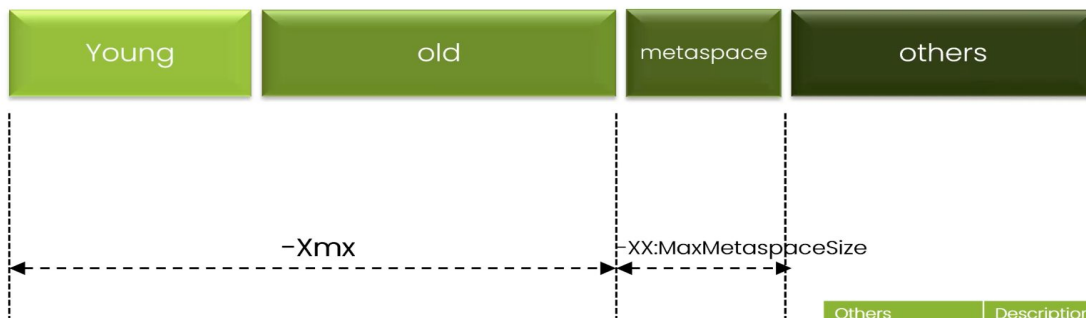
JVM optimizations - where do they run?

- Remember that the code you write as a dev is not what gets executed by the JVM
- JVM does a lot of optimizations
- Like Inlining, hotspotting (identifying and optimizing the most frequently used parts of a java program)
- In order to do that the JVM needs memory
- That memory comes from the others region.



Socket buffer

- When a customer connects to your application, you need connections
- And when your app needs to talk to a backend system you also need connections
- So, where do these connections/socket buffers come from?
- They also come from this **others** region



Young: Newly created Objects

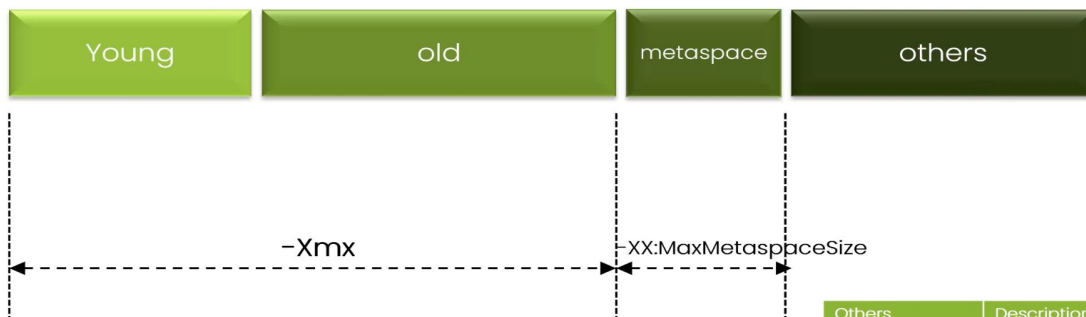
Old: New objects that survived one or more minor GC promoted here

Metaspace: Classes, Methods, metadata

Others	Description
Thread Stacks	Each thread has a separate memory space. Controlled by -Xss
Garbage Collection	Threads, Memory to store GC info
Code Generation	Converting bytecode to native code
Socket Buffers	TCP Connections (Receive buffer ~37k, Send Buffer ~2.5k)
JNI	JNI program also allocate memory

Takeaways

- Your Java application WILL consume more than the -Xmx limit
- This is just the limit for Young+Old which is your heap size

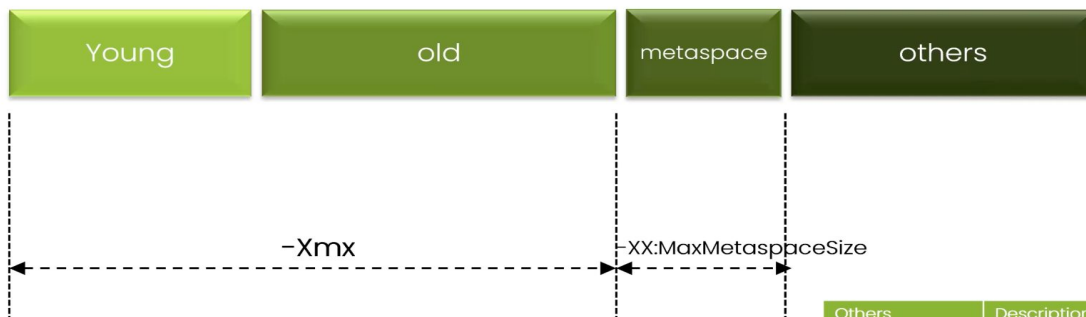


Young: Newly created Objects
Old: New objects that survived one or more minor GC promoted here
Metaspace: Classes, Methods, metadata

Others	Description
Thread Stacks	Each thread has a separate memory space. Controlled by <code>-Xss</code>
Garbage Collection	Threads, Memory to store GC info
Code Generation	Converting bytecode to native code
Socket Buffers	TCP Connections (Receive buffer ~37k, Send Buffer ~2.5k)
JNI	JNI program also allocate memory

Is there a global setting?

- Natural follow up question: is there a global setting that would allow me to control the entire size of the memory by used the JVM?
- Answer: No

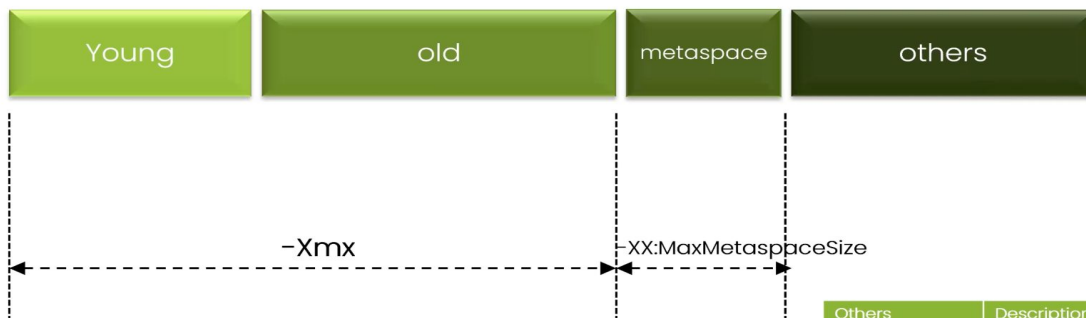


Young: Newly created Objects
Old: New objects that survived one or more minor GC promoted here
Metaspace: Classes, Methods, metadata

Others	Description
Thread Stacks	Each thread has a separate memory space. Controlled by -Xss
Garbage Collection	Threads, Memory to store GC info
Code Generation	Converting bytecode to native code
Socket Buffers	TCP Connections (Receive buffer ~37k, Send Buffer ~2.5k)
JNI	JNI program also allocate memory

Is there a global setting?

- Natural follow up question: is there a global setting that would allow me to control the entire memory size used by Java?
- Answer: No
- What you see here is the total memory used by Java process: Young + old + metaspace + others..



Young: Newly created Objects

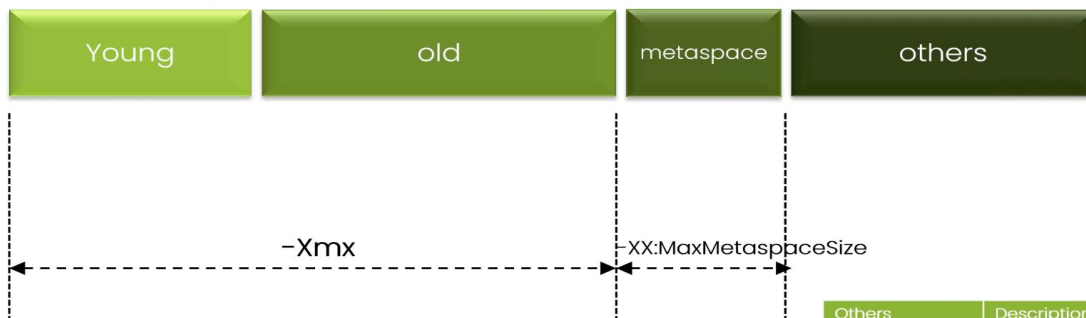
Old: New objects that survived one or more minor GC promoted here

Metaspace: Classes, Methods, metadata

Others	Description
Thread Stacks	Each thread has a separate memory space. Controlled by <code>-Xss</code>
Garbage Collection	Threads, Memory to store GC info
Code Generation	Converting bytecode to native code
Socket Buffers	TCP Connections (Receive buffer ~37k, Send Buffer ~2.5k)
JNI	JNI program also allocate memory

What about multiple threads?

- If you have say, 100 or 200 threads, **each** thread will have a thread stack
- And the thread stack will be stored in the “others” section...



Young: Newly created Objects

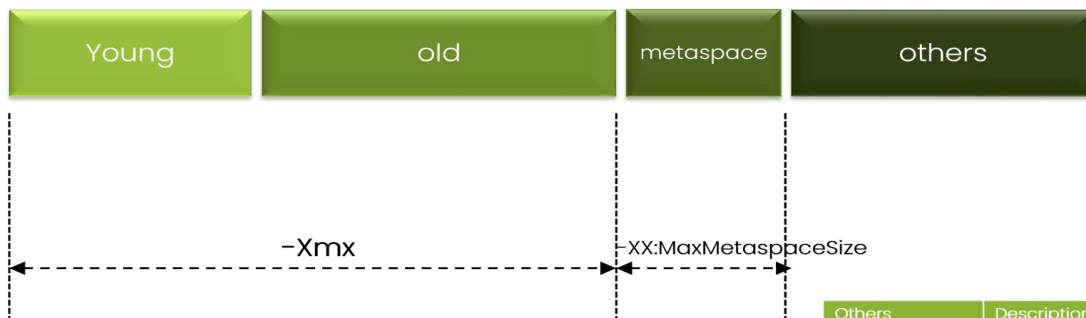
Old: New objects that survived one or more minor GC promoted here

Metaspace: Classes, Methods, metadata

Others	Description
Thread Stacks	Each thread has a separate memory space. Controlled by -Xss
Garbage Collection	Threads, Memory to store GC info
Code Generation	Converting bytecode to native code
Socket Buffers	TCP Connections (Receive buffer ~37k, Send Buffer ~2.5k)
JNI	JNI program also allocate memory

Which part of this is the heap?

- When we use the term heap, it's really just the Young + Old portions below.
- So, Heap == Young + Old



Young: Newly created Objects

Old: New objects that survived one or more minor GC promoted here

Metaspace: Classes, Methods, metadata

Others	Description
Thread Stacks	Each thread has a separate memory space. Controlled by -Xss
Garbage Collection	Threads, Memory to store GC info
Code Generation	Converting bytecode to native code
Socket Buffers	TCP Connections (Receive buffer ~37k, Send Buffer ~2.5k)
JNI	JNI program also allocate memory

When we run a performance test....

- What are some things that we set when run a perf test?
- Intuit uses Gatling, also JMeter is common
- Input parameters: throughput, what kind SLA - response time/latency == GOAL that you are setting
- If im perf testing turbotax.com, i expect a certain response time from the website - so let's say it's something like .3 ms
- What the performance testing tool does is output the resource recommendations
- More aggressive goal in a perf test == more resource recommendations (more memory)

How is this Java program executed?

- Take a look at this sample code - let's step through what happens when it's executed:

```
public class SimpleExample {  
  
    public static void main(String args[]) {  
  
        a();  
    }  
  
    public static void a() {  
  
        int x = 1;  
        b();  
    }  
  
    public static void b() {  
  
        Car y = new Car();  
        c();  
    }  
  
    public static void c() {  
  
        float z = 2.11f;  
    }  
}
```


How is this Java program executed?

- A is invoked, which sets a primitive variable to 1 and then calls b and then c

```
public class SimpleExample {  
  
    public static void main(String args[]) {  
  
        a();  
    }  
  
    public static void a() {  
  
        int x = 1;  
        b();  
    }  
  
    public static void b() {  
  
        Car y = new Car();  
        c();  
    }  
  
    public static void c() {  
  
        float z = 2.11f;  
    }  
}
```

How is this Java program executed?

- B instantiates a complex object which then invokes method c, which initializes a float

```
public class SimpleExample {  
  
    public static void main(String args[]) {  
  
        a();  
    }  
  
    public static void a() {  
  
        int x = 1;  
        b();  
    }  
  
    public static void b() {  
  
        Car y = new Car();  
        c();  
    }  
  
    public static void c() {  
  
        float z = 2.11f;  
    }  
}
```

How is this Java program executed?

- What happens behind the scenes here when JVM executes this code?

```
public class SimpleExample {  
    public static void main(String args[]) {  
        a();  
    }  
    public static void a() {  
        int x = 1;  
        b();  
    }  
    public static void b() {  
        Car y = new Car();  
        c();  
    }  
    public static void c() {  
        float z = 2.11f;  
    }  
}
```

How is this Java program executed?

- Let's step through a sample invocation of this program...
- When we invoke this application it is using a thread

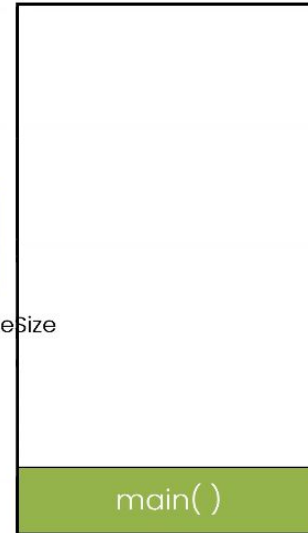
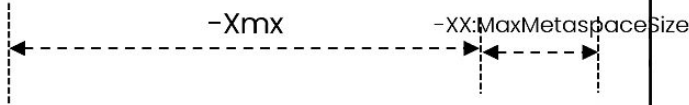
```
public class SimpleExample {  
  
    public static void main(String args[]) {  
  
        a();  
    }  
  
    public static void a() {  
  
        int x = 1;  
        b();  
    }  
  
    public static void b() {  
  
        Car y = new Car();  
        c();  
    }  
  
    public static void c() {  
  
        float z = 2.11f;  

```

How is this Java program executed?

- The moment the thread is created it will create a thread stack in the Others region:
- When main is invoked it's added to the thread's stack frame...

```
public class SimpleExample {  
  
    public static void main(String args[]) {  
        a();  
    }  
  
    public static void a() {  
        int x = 1;  
        b();  
    }  
  
    public static void b() {  
        Car y = new Car();  
        c();  
    }  
  
    public static void c() {  
        float z = 2.11f;  
    }  
}
```



Thread's stack

How is this Java program executed?

- Then the thread moves to execute method a
- What happens now? A is added to the thread stack frame:

```
public class SimpleExample {
```

```
    public static void main(String args[]) {
```

```
        a();
    }
```

```
    public static void a() {
```

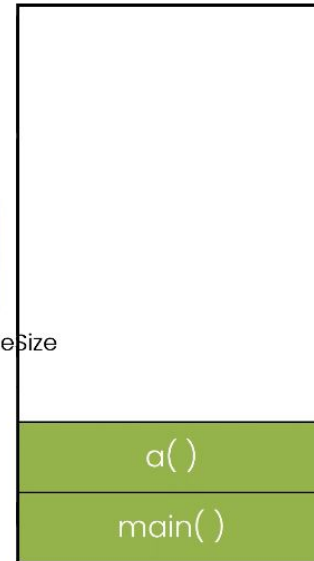
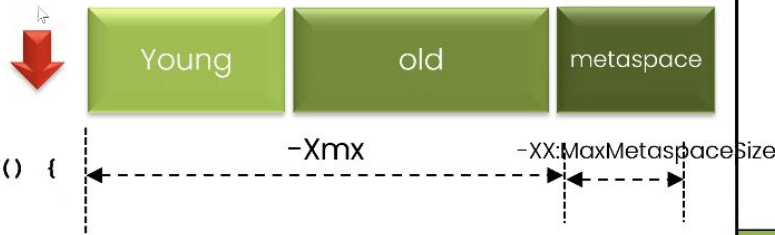
```
        int x = 1;
        b();
    }
```

```
    public static void b() {
```

```
        Car y = new Car();
        c();
    }
```

```
    public static void c() {
```

```
        float z = 2.11f;
    }
```



Thread's stack

How is this Java program executed?

- Because x is a primitive data type it will be stored in the **thread stack**
- X is NOT an object/complex data type so it will not be put on the heap

```
public class SimpleExample {
```

```
    public static void main(String args[]) {
```

```
        a();
    }
```

```
    public static void a() {
```

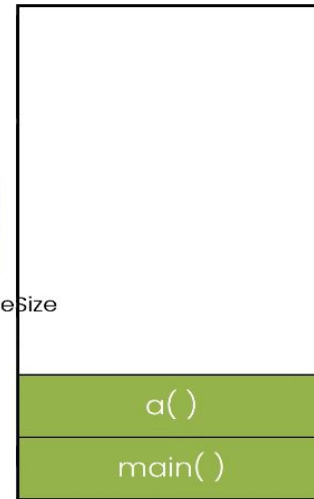
```
        int x = 1;
        b();
    }
```

```
    public static void b() {
```

```
        Car y = new Car();
        c();
    }
```

```
    public static void c() {
```

```
        float z = 2.11f;
    }
```

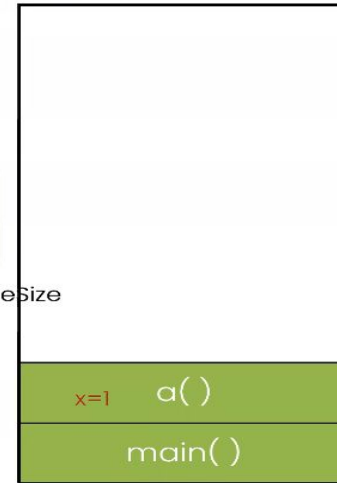


Thread's stack

How is this Java program executed?

- Because x is a primitive data type it will be stored in the **thread stack**
- X is NOT an object/complex data type so it will NOT be put on the heap

```
public class SimpleExample {  
    public static void main(String args[]) {  
        a();  
    }  
  
    public static void a() {  
        int x = 1;  
        b();  
    }  
  
    public static void b() {  
        Car y = new Car();  
        c();  
    }  
  
    public static void c() {  
        float z = 2.11f;  
    }  
}
```

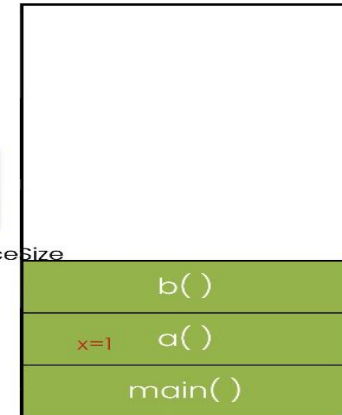


Thread's stack

How is this Java program executed?

- Next up, we have a call to the method b
- In method b we can see that here it creates a Car object
- Every class has some metadata - like what are the methods, what instance variables are inside it?
- The metaspace is where that metadata will be created:

```
public class SimpleExample {  
    public static void main(String args[]) {  
        a();  
    }  
  
    public static void a() {  
        int x = 1;  
        b();  
    }  
  
    public static void b() {  
        Car y = new Car();  
        c();  
    }  
  
    public static void c() {  
        float z = 2.11f;  
    }  
}
```

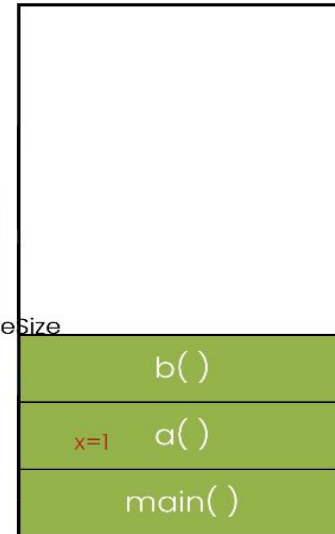
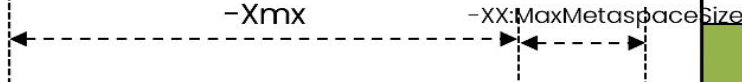


Thread's stack

How is this Java program executed?

- Now, a question: where will the car object be stored? And where will the variable y be stored??

```
public class SimpleExample {  
  
    public static void main(String args[]) {  
  
        a();  
    }  
  
    public static void a() {  
  
        int x = 1;  
        b();  
    }  
  
    public static void b() {  
  
        Car y = new Car();  
        c();  
    }  
  
    public static void c() {  
  
        float z = 2.11f;  
    }  
}
```

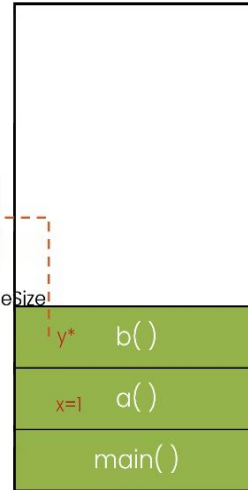
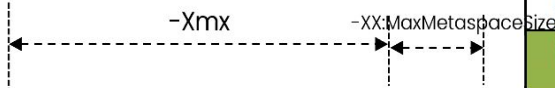


Thread's stack

How is this Java program executed?

- Car is a complex data type (NOT a primitive like “x”)
- And because it's a new object it will be stored in the Young generation of memory as you can see here:
- And the variable y will be stored in the thread stack as you can see:

```
public class SimpleExample {  
  
    public static void main(String args[]) {  
  
        a();  
    }  
  
    public static void a() {  
  
        int x = 1;  
        b();  
    }  
  
    public static void b() {  
  
        Car y = new Car();  
        c();  
    }  
  
    public static void c() {  
  
        float z = 2.11f;  
    }  
}
```

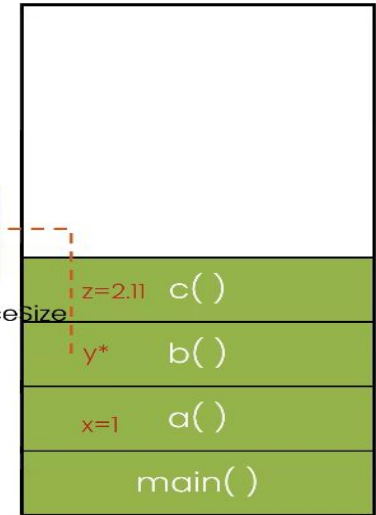
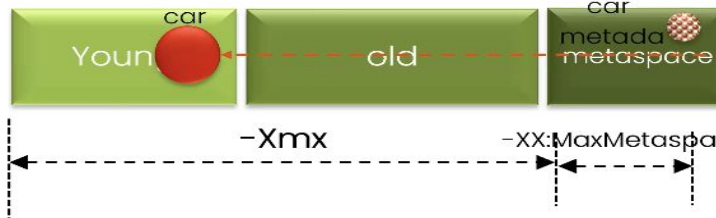


Thread's stack

How is this Java program executed?

- Finally we move to invoking method C:
- And then C is moved to thread stack and z is a local variable added to the thread stack as well:

```
public class SimpleExample {  
  
    public static void main(String args[]) {  
  
        a();  
    }  
  
    public static void a() {  
  
        int x = 1;  
        b();  
    }  
  
    public static void b() {  
  
        Car y = new Car();  
        c();  
    }  
  
    public static void c() {  
  
        float z = 2.11f;  
    }  
}
```



Thread's stack

How is this Java program executed?

- And because c is the last method, the thread will start exiting from each method

```
public class SimpleExample {
```

```
    public static void main(String args[]) {
```

```
        a();
    }
```

```
    public static void a() {
```

```
        int x = 1;
        b();
    }
```

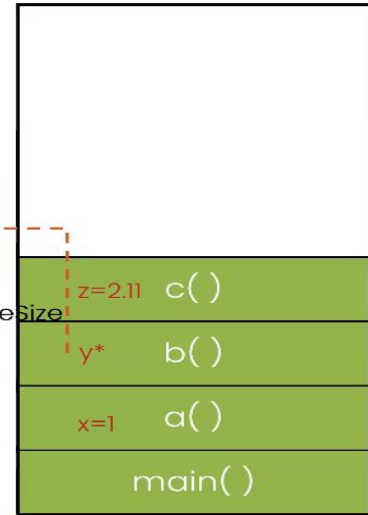
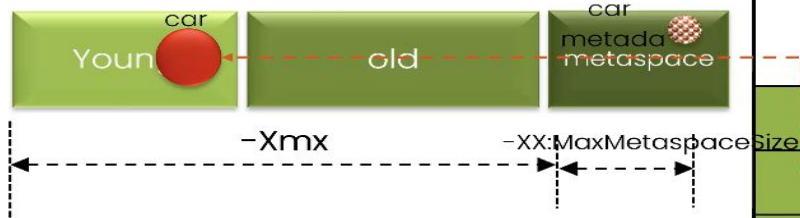
```
    public static void b() {
```

```
        Car y = new Car();
        c();
    }
```

```
    public static void c() {
```

```
        float z = 2.11f;
```

```
    }
```



Thread's stack

Thread stack size: -Xss JVM argument

- The thread stack size is set by the JVM argument Xss:

```
public class SimpleExample {
```

```
    public static void main(String args[]) {
```

```
        a();
    }
```

```
    public static void a() {
```

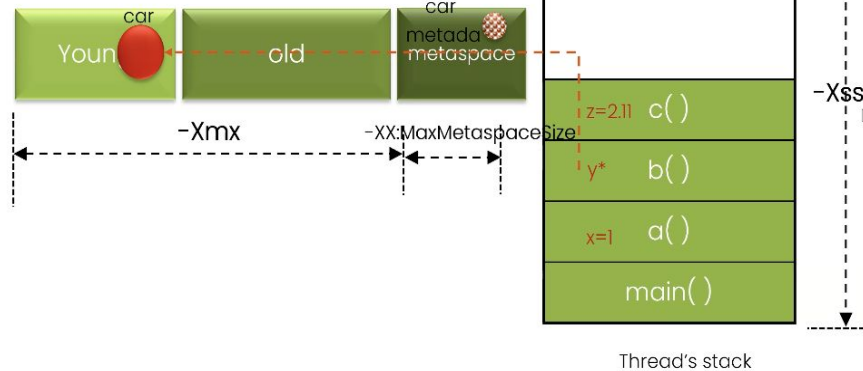
```
        int x = 1;
        b();
    }
```

```
    public static void b() {
```

```
        Car y = new Car();
        c();
    }
```

```
    public static void c() {
```

```
        float z = 2.11f;
    }
```



Thread stack size: -Xss JVM argument

- So if i set the Xss to 1 MB size, it means 1 MB will be allocated for each thread in my application
- So if i have an app with 1,000 threads, then $1,000 * 1 \text{ MB} == 1 \text{ GB}$ of additional memory is required!
- So, you will need 1 GB in addition to whatever you set your Xmx to

```
public class SimpleExample {
```

```
    public static void main(String args[]) {
```

```
        a();
    }
```

```
    public static void a() {
```

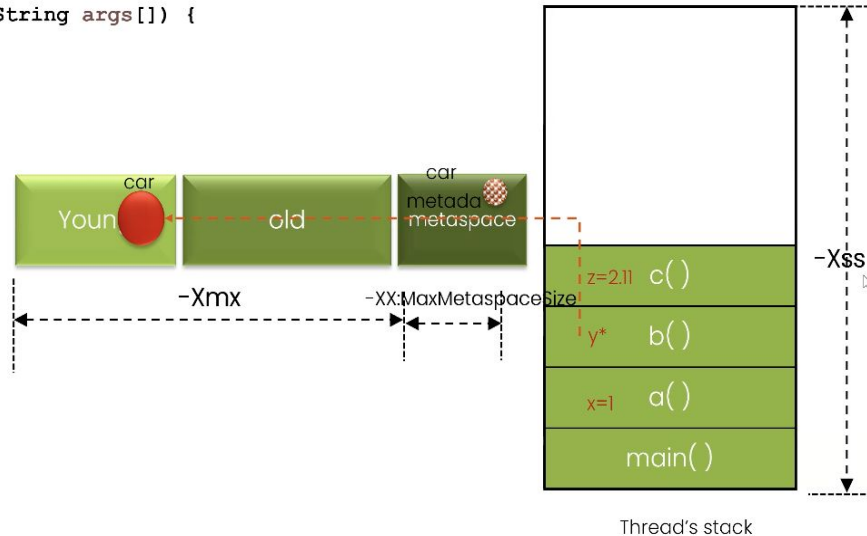
```
        int x = 1;
        b();
    }
```

```
    public static void b() {
```

```
        Car y = new Car();
        c();
    }
```

```
    public static void c() {
```

```
        float z = 2.11f;
```



Capacity planning

- When you do any capacity planning you want to take these things into consideration
- If you're using an AWS EC2 instance to host your application
- For example, if your Xmx size is set to 2 GB, you don't want to get an AWS instance that's only 2.5 GB - you need to have space for metaspace, threads and kernel process
- Based on that you want to choose the sizing of your EC2 instance...

```
public class SimpleExample {
```

```
    public static void main(String args[]) {
```

```
        a();
    }
```

```
    public static void a() {
```

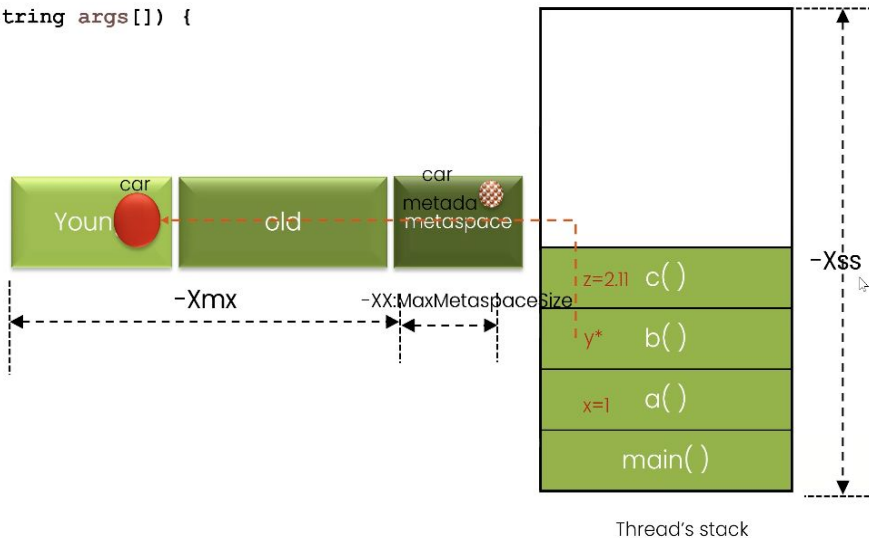
```
        int x = 1;
        b();
    }
```

```
    public static void b() {
```

```
        Car y = new Car();
        c();
    }
```

```
    public static void c() {
```

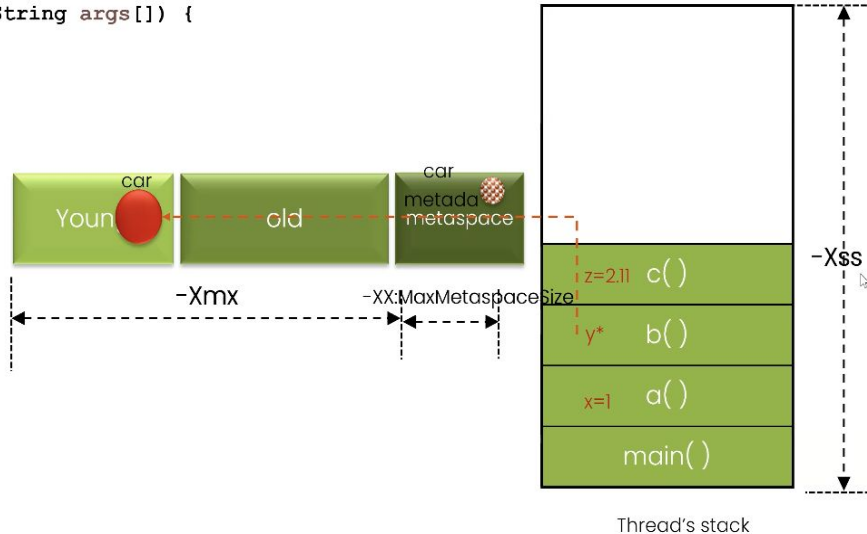
```
        float z = 2.11f;
```



Stack overflow error

- You get the stack overflow error when you go past the -Xss limit..
- Think: Xss - the “ss” is for stack size
- Think of Xss as the upper bound
- Because you no longer have any memory left in the stack

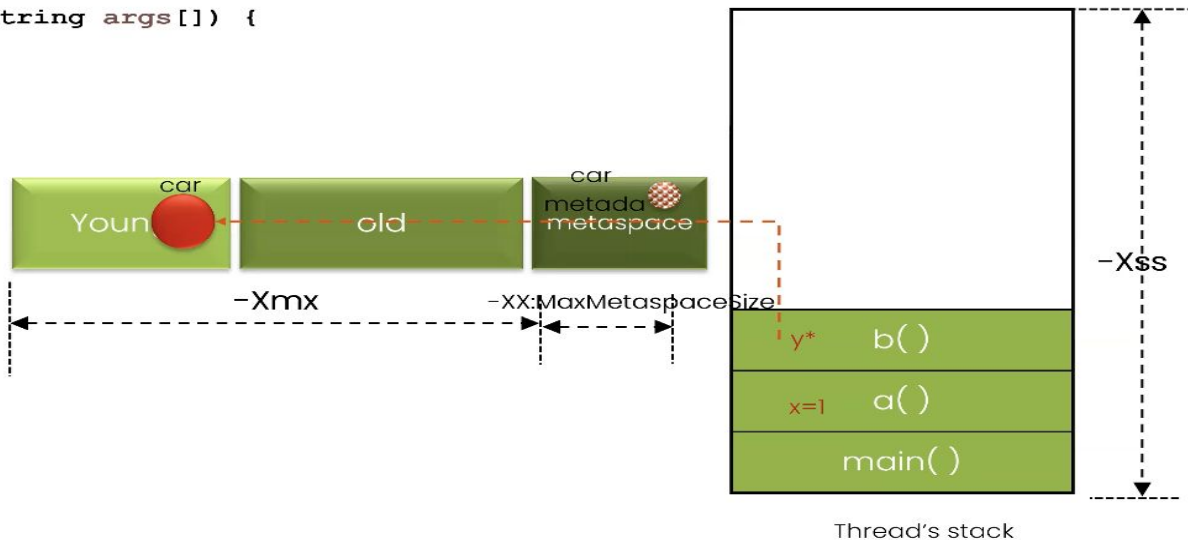
```
public class SimpleExample {  
  
    public static void main(String args[]) {  
  
        a();  
    }  
  
    public static void a() {  
  
        int x = 1;  
        b();  
    }  
  
    public static void b() {  
  
        Car y = new Car();  
        c();  
    }  
  
    public static void c() {  
  
        float z = 2.11f;  
        }  
}
```



Next step - stack pops

- Now there are no more methods to execute
- The stack will pop off the “c” method stack frame
- Remember a stack is LIFO structure

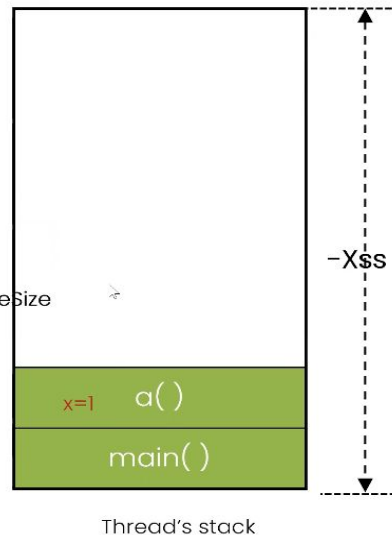
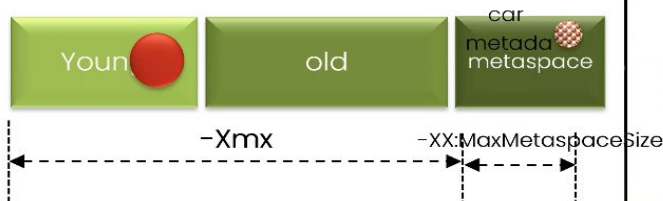
```
public class SimpleExample {  
  
    public static void main(String args[]) {  
  
        a();  
    }  
  
    public static void a() {  
  
        int x = 1;  
        b();  
    }  
  
    public static void b() {  
  
        Car y = new Car();  
        c();  
    }  
  
    public static void c() {  
  
        float z = 2.11f;  
    }  
}
```



Next step - stack pops

- Now the thread will exit method “b” since there’s nothing more to execute there
- Notice how the variable “y” is removed from the thread stack:
- But you can see the red dot which is the car object, that still resides in memory!
- The reference “y” was the reference but was deleted

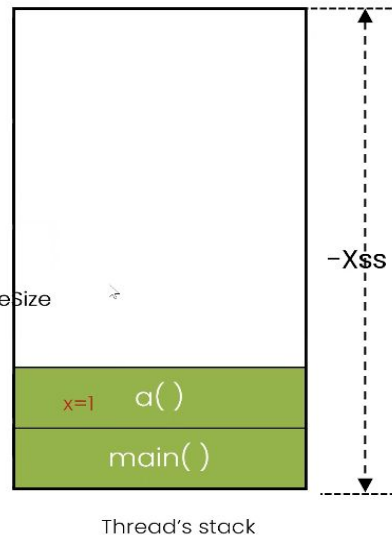
```
public class SimpleExample {  
  
    public static void main(String args[]) {  
  
        a();  
    }  
  
    public static void a() {  
  
        ↑ int x = 1;  
        b();  
    }  
  
    public static void b() {  
  
        Car y = new Car();  
        c();  
    }  
  
    public static void c() {  
  
        float z = 2.11f;  
    }  
}
```



Next step - stack pops

- How long does the Car object stay in memory?
- Basically until the next cycle/event of the GC - the next time the GC runs
- Then the GC will see that there is no reference to the object and it will evict the object from memory

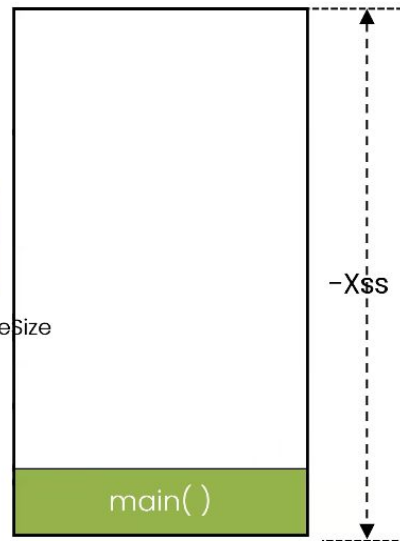
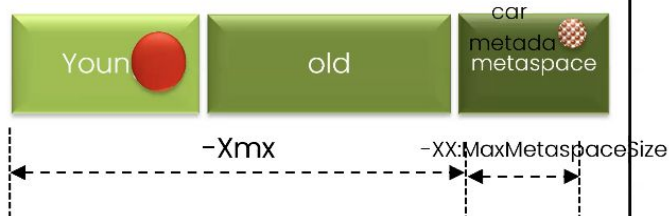
```
public class SimpleExample {  
  
    public static void main(String args[]) {  
  
        a();  
    }  
  
    public static void a() {  
  
        ↑ int x = 1;  
        b();  
    }  
  
    public static void b() {  
  
        Car y = new Car();  
        c();  
    }  
  
    public static void c() {  
  
        float z = 2.11f;  
    }  
}
```



Next step - stack pops

- Finally, a will pop and the only thing left is the main method

```
public class SimpleExample {  
    public static void main(String args[]) {  
        a();  
    }  
  
    public static void a() {  
        int x = 1;  
        b();  
    }  
  
    public static void b() {  
        Car y = new Car();  
        c();  
    }  
  
    public static void c() {  
        float z = 2.11f;  
    }  
}
```



Thread's stack

Next step - stack is now empty

- And the very last thing is main method will exit and the stack frame will be completely empty
- There is no more stack frames but the thread is still there
- So, when the next request comes in the thread will handle it and repeat the cycle...

```
public class SimpleExample {
```

```
    public static void main(String args[]) {
```

```
        a();
    }
```

```
    public static void a() {
```

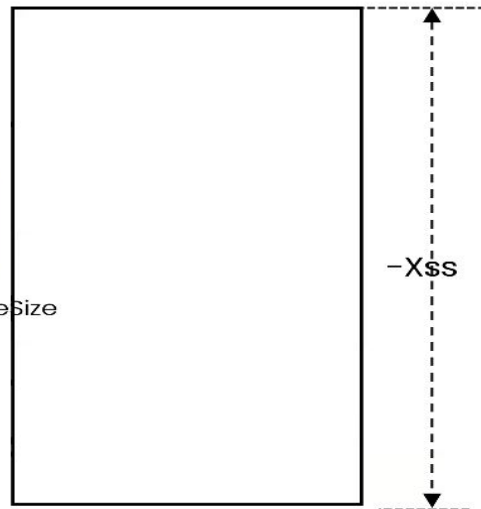
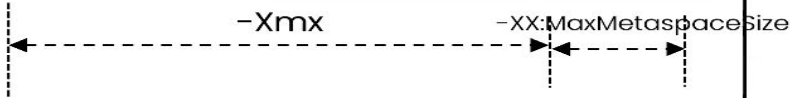
```
        int x = 1;
        b();
    }
```

```
    public static void b() {
```

```
        Car y = new Car();
        c();
    }
```

```
    public static void c() {
```

```
        float z = 2.11f;
    }
```



Thread's stack

Demo - stackoverflow error

- What happens if we run some code that will give us a stackoverflow error?

```
public class SimpleExample {
```

```
    public static void main(String args[]) {
```

```
        a();
    }
```

```
    public static void a() {
```

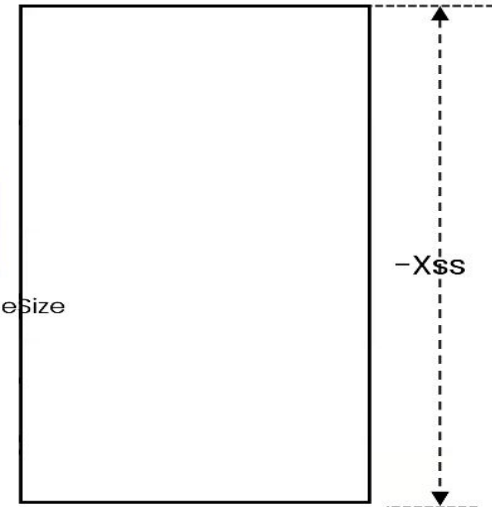
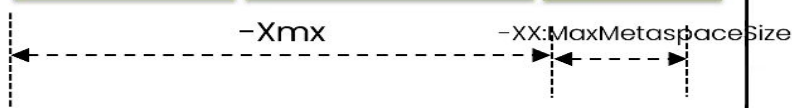
```
        int x = 1;
        b();
    }
```

```
    public static void b() {
```

```
        Car y = new Car();
        c();
    }
```

```
    public static void c() {
```

```
        float z = 2.11f;
    }
```



Thread's stack

Default of -Xss value?

- What happens if we don't have this value configured?
- The default value varies based on operating system and what version of Java you use



JVM version	Thread stack size
Sparc 32-bit JVM	512k
Sparc 64-bit JVM	1024k
x86 Solaris/Linux 32-bit JVM	320K
x86 Solaris/Linux 64-bit JVM	1024K
Windows 32-bit JVM	320K
Windows 64-bit JVM	1024K



Are Java threads – memory efficient?

Best practice recommendation: ***set your -Xss***

- You should set a limit for the -Xss
- You do not want to be influenced by your underlying architecture



JVM version	Thread stack size
Sparc 32-bit JVM	512k
Sparc 64-bit JVM	1024k
x86 Solaris/Linux 32-bit JVM	320K
x86 Solaris/Linux 64-bit JVM	1024K
Windows 32-bit JVM	320K
Windows 64-bit JVM	1024K



Are Java threads – memory efficient?



Xss - more

- What if we set the Xss to some value, like 1024K - 1 MB
- What would happen if our thread only needs 500 Kb (half of 1 MB, half of the Xss value)
- Will the thread memory occupy the 500kb or 1 MB (the Xss value)?

Xss answer

- The answer to that question is: the thread will allocate the memory specified by the Xss value
- You can read more about the study here:
<https://blog.gceasy.io/2021/02/24/java-threads-may-not-be-memory-efficient/>
- Basically, 2 different applications - one with a huge stack frame and one almost empty were compared
- But they were showed to have the same memory consumption

Xss answer

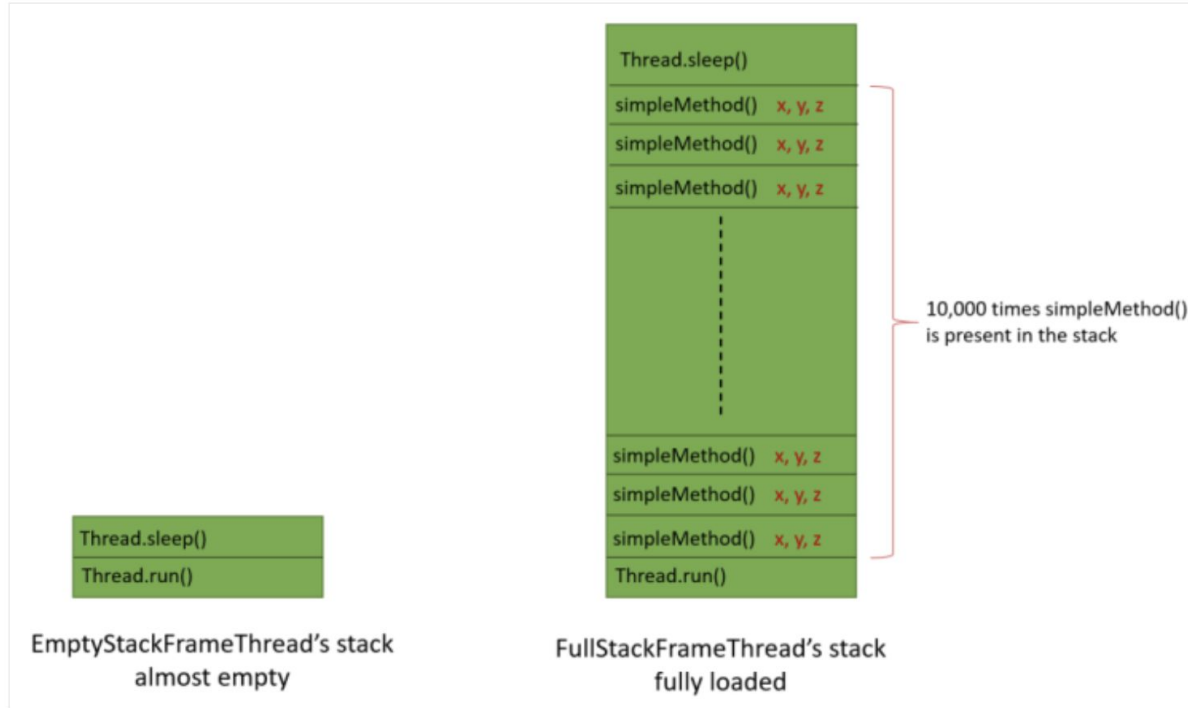


Fig: EmptyStackFrameThread and FullStackFrameThread stack

Xss answer

Below you can see the program's memory consumption reported by the system monitoring tool – 'top'.

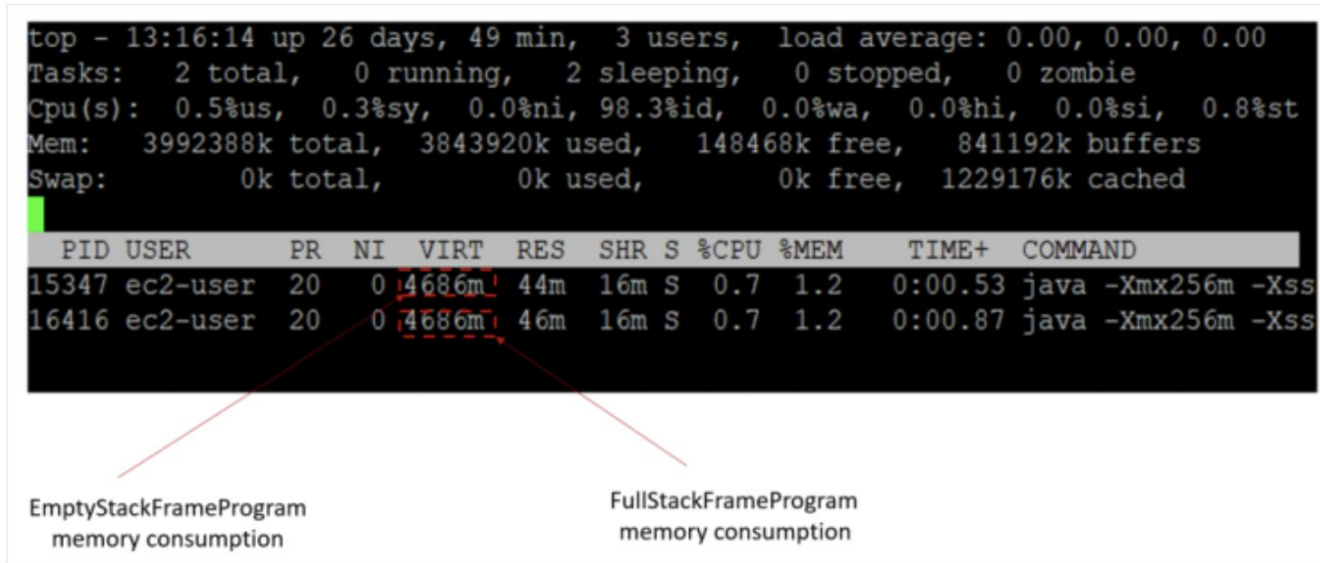


Fig: Memory consumption of both programs are same



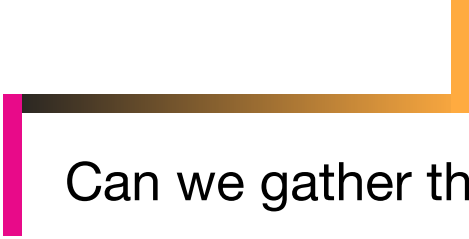
Different threads

- What if you have different threads each with their own memory requirement?
- Like some threads need to do more analysis versus others...
- Is it possible to set the xss value individually - the thread stack size on per thread basis?
- Answer: no it's not possible!



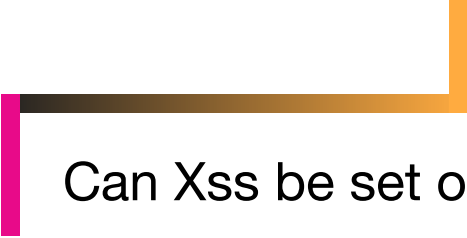
How would we determine what Xss should be set to?

- Through a performance/stress test
- You will want to monitor your logs to see if you are getting a stackoverflow error
- If you get that error, then you will want to keep increasing the Xss limit
- It's a bit of trial and error - no magnifying glass here...
- Keep in mind that for capacity planning it's not just the Xss - it's also the Xmx, metaspace and the network connections



Can we gather the values for xmx, metaspace from a tool?

- Answer: yes!
- You can feed your logs into a tool like YCrash and then get recommendations for those values
- But for threads and Xss, the tool can not do that!



Can Xss be set on Android as well?

- Android is still running on a modified version of JVM
- So, it should allow you



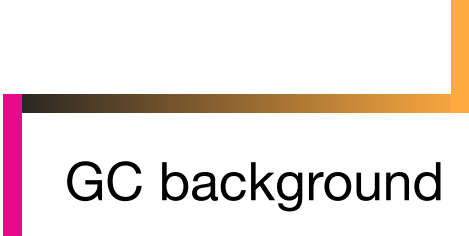
Is there a range of values for Xss

- Yes, you can read more here: <https://www.baeldung.com/jvm-configure-stack-sizes>
-
-



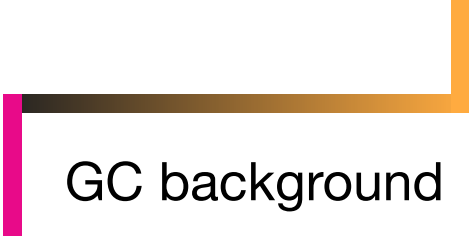
Different threads...

GC Background and Patterns in JVM



GC background

- The GC implementation lives in the JVM
- GC is automatic (but with some parameters we can configure)
- GC will identify unused objects and delete that memory for use by other objects...
- Keep in mind the GC only frees up space on the **heap** (objects) not on the stack (local variables)
- The JVM will free up space on the **stack**



GC background

- There are several different GC's available in the JVM, each with their own strengths and weaknesses...

Why GC? A case study

- <https://blog.gceasy.io/2022/08/25/automobile-company-optimizes-performance-using-gceasy/>
- Just by tuning the GC arguments for the JVM they were able to reduce their application response time by 49%!!
- They went from 1.88 seconds to .95 seconds
- Amazing results. They did not have to refactor any code, they just adjusted their GC parameters...
- Pause for a second and think: how many times have you seen such a huge improvement?



Why GC? A case study

- Typically such huge improvements are caused by refactoring some code or moving to some expensive hardware
- Whereas for this you do not need to touch a line of code
- Just by modifying the JVM arguments for GC you can do this!



GC Throughput - a key GC metric

- One key KPI (Key performance indicator) with GC
- Suppose our GC throughput is 98% - what does that mean to you?
- Does that sound like a good number to you?



GC Throughput - a key GC metric

- A throughput of 98% means our application is spending 98% of it's time in processing customer transactions
- When GC runs, ***it pauses your application*** to see which objects are active/inactive and which objects do not have any references
- If object(s) does not have a reference it has to evict them from memory
- Then the GC has to ***compact*** the memory as well



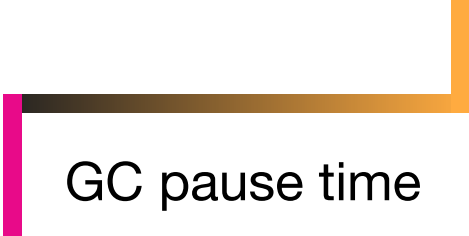
What does compact memory mean in GC? Brief aside

- When the GC compact memory, it means that the GC often deletes objects in memory that are NOT contiguous
- Contiguous == next to each other
- This means that there can be **gaps** in memory - basically small gaps/blocks that can be used for other purposes...
- So, the process of compacting memory means moving memory so that it is contiguous
- This process can free up memory and make it easier for the GC to allocate memory in the future...



GC Throughput - the math

- A throughput of 98% means our application is spending 98% of it's time in processing customer transactions
- In a single day that means there are 1440 minutes, and 2% of that is 28.8 minutes
- So for 28.8 minutes, the application is in GC pause time
- That time is not continuous of course, but rather a few milliseconds here and there which add up during the day...
- Imagine if you buy a car and it runs for a few miles and then pauses - not acceptable
- This is what the JVM does behind the scenes...

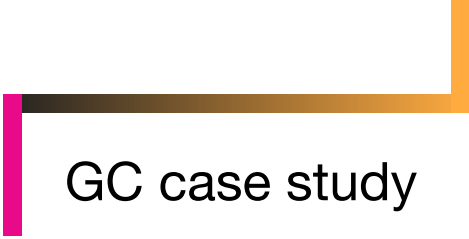


GC pause time

- The length of the pause can vary: it can be 10 ms, we've seen pause times of 10 seconds (continuous - not total)
- Typically for a given pause time it's a few milliseconds or a few seconds
- A 98% throughput is actually quite good

GC case study

- <https://blog.gceasy.io/2021/07/08/how-many-millions-of-dollars-enterprises-waste-due-to-garbage-collection/>
- Scroll down to Dollars wasted section
- You can see a table calculating dollars wasted for mid to large to x-large companies...
- 7 days of your application sleeping (roughly) per instance per year
- BUT, your cloud provider (AWS, GCP, etc..) will STILL be charging you for that time regardless
- For t2.large it can cost roughly 50 cents per hour
- For huge orgs can be 25 million lost!! Sad story: no one notices this and no one factors this price in.
<https://blog.gceasy.io/2021/07/08/how-many-millions-of-dollars-enterprises-waste-due-to-garbage-collection/>



GC case study

- Even embedded Java applications suffer from this problem
- For example, if you're running a Java app in a printer, every time GC happens you may see some whitespace being printed by the printer
- People have seen this in the past!
- Also, this is not a problem limited to the JVM - a lot of modern applications (like Python, .NET apps which have a GC) run into this problem!
- Once automatic GC came into existence, this became a common problem
- But other languages often do not have the same detailed logging as Java - so fine tuning the GC is often harder in other languages



GC case study

- A lot of times we focus more on the unpleasant customer experience during GC
- IE the response time
- But the dollar cost of the GC is often overlooked as we discussed...

GC patterns

- Here we can see a healthy saw-tooth pattern of GC:
- <https://blog.gceasy.io/2021/10/15/interesting-garbage-collection-patterns/>
- The heap usage will go up and the GC event will be triggered
- You can see the slope (dotted black line) at 0 degrees
- Scroll down to #3 - an acute memory leak
- Requests come in leading to more heap usage, then the GC event runs to free up heap memory
- The slope here in #3 is roughly 18 degrees

GC patterns - memory leak

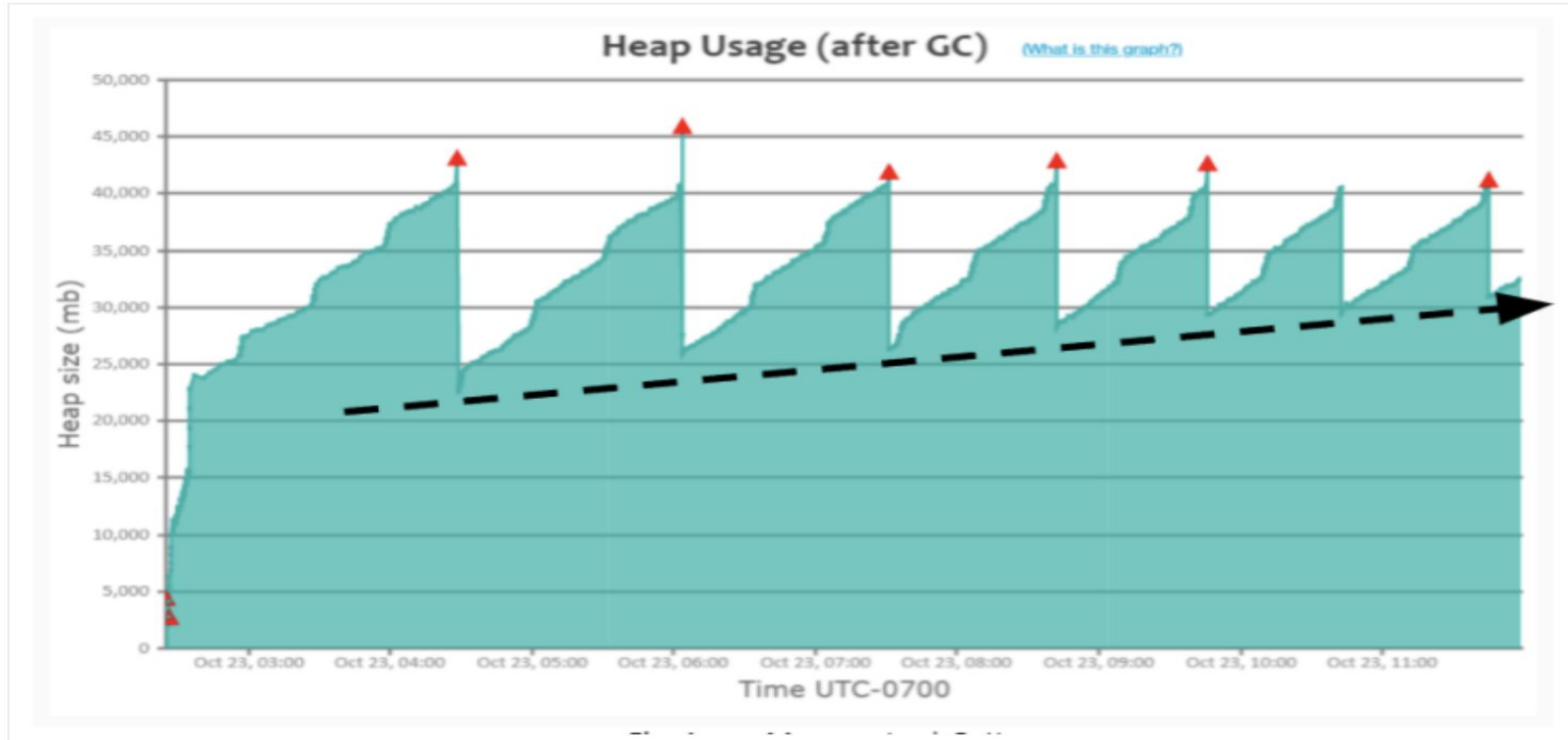
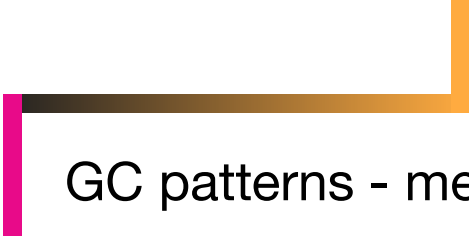


Fig 3: Acute memory leak GC pattern



GC patterns - memory leak

- You can see where this is trending...
- You can take some corrective action several days/hours ahead...
- The data being shown in the graph is coming from the logs
- The GCeasy tool can help with the analysis
- And then if you go to graph 5 you can see a memory leak pattern
- The red indicates the GC running over and over again - it's hit the max limit of memory and it's trying to reclaim memory but there is none to reclaim
- And then your CPU may be running at 98% (not to be confused with GC throughput)

GC patterns - memory leak

5. Memory Leak Pattern

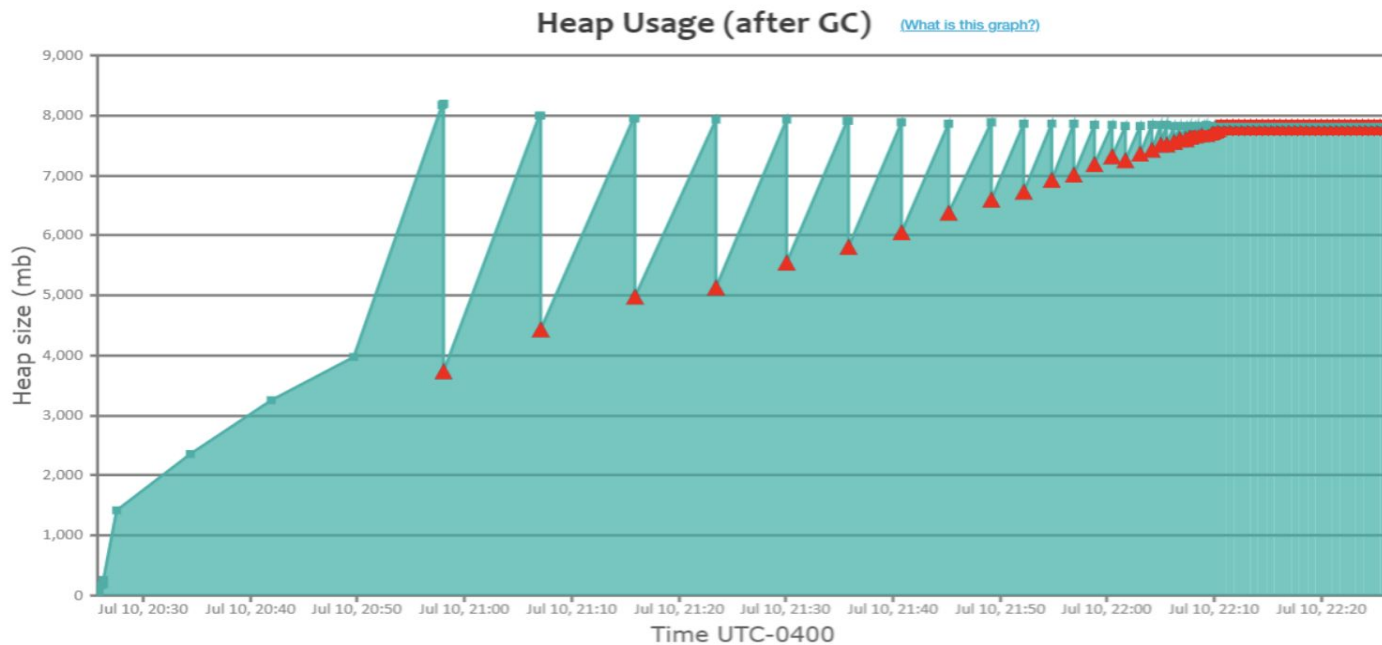
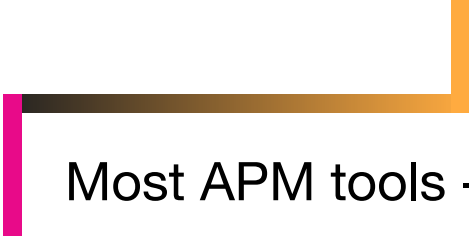


Fig 5: Memory Leak GC pattern



Key takeaway - GC CPU consumption is huge

- GC consumes ENORMOUS amounts of CPU cycles
- Modern applications in order to service a simple request they create a ton of objects
- And to keep track of all those objects - see who's referencing them, who's not referencing them, to sweep them out of memory - all of these activities require a lot of CPU cycles...
- When you see a lot of red dots together - it's indicative that your CPU will be running at a very high level - like 99%
- And then your OOM (Out of memory) error will be at the top end
- And your GC throughput will degrade as you go further up in the graph closer to the concentration of red dots (from 95 to 85 to 70, etc..)



Most APM tools - Application Performance Monitoring

- If you're monitoring GC Throughput, it can predict outages for you
- This is what we call a **micro** metric - this can **predict outages**
- Most APM tools are only looking at **macro** metrics like response time, CPU and memory - this is more of a **reactive** monitoring
- But a micro metric like GC Throughput is much better for helping us predict outages
- GC throughput is essentially a lead indicator of outages - a measurement that provides insight into future performance

Other interesting GC patterns - heavy caching pattern

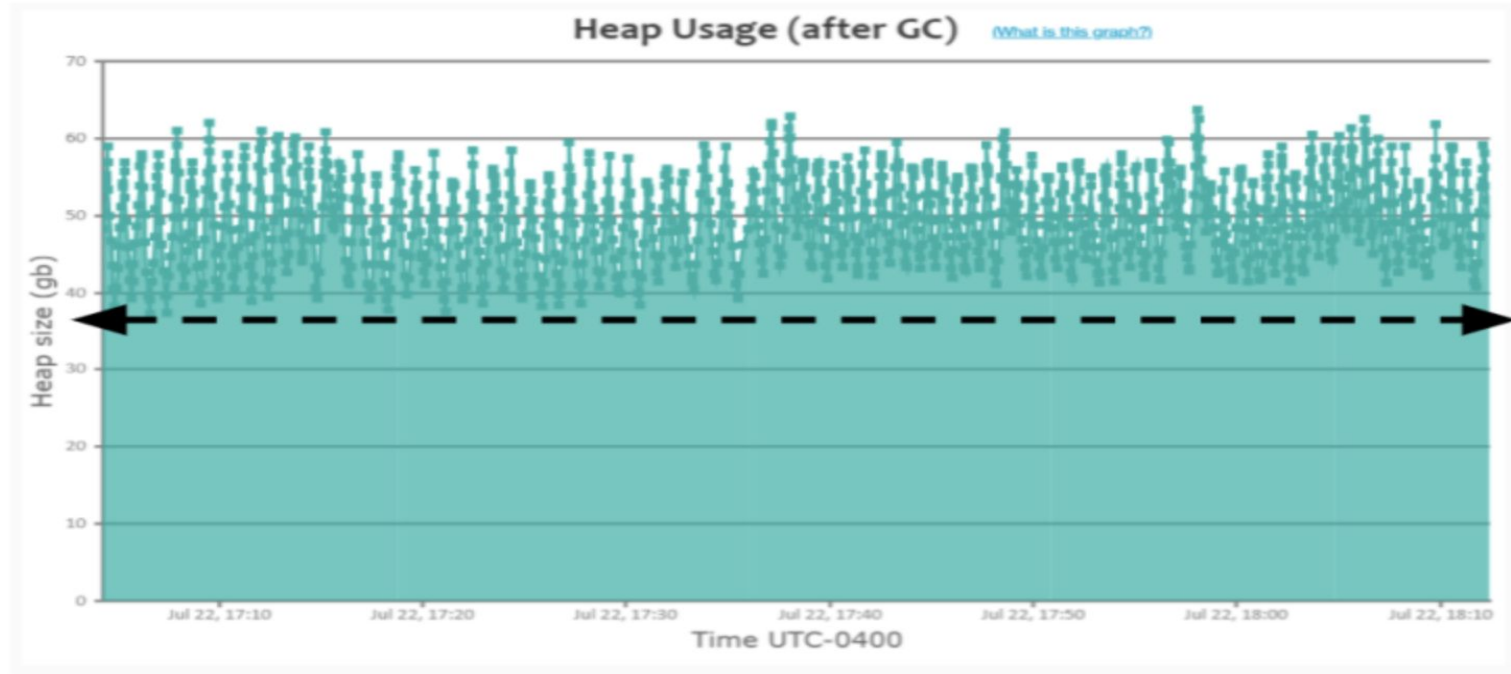


Fig 2: Heavy caching GC pattern



Heavy caching pattern

- With the heavy caching pattern, we can see that the heap size does not drop below a certain amount here
- The graph shows that here we are caching 40 GB of data
- If you show this to the developers who work on this application, even they may be surprised by the fact that they're caching 40 GB of data



Pattern 4

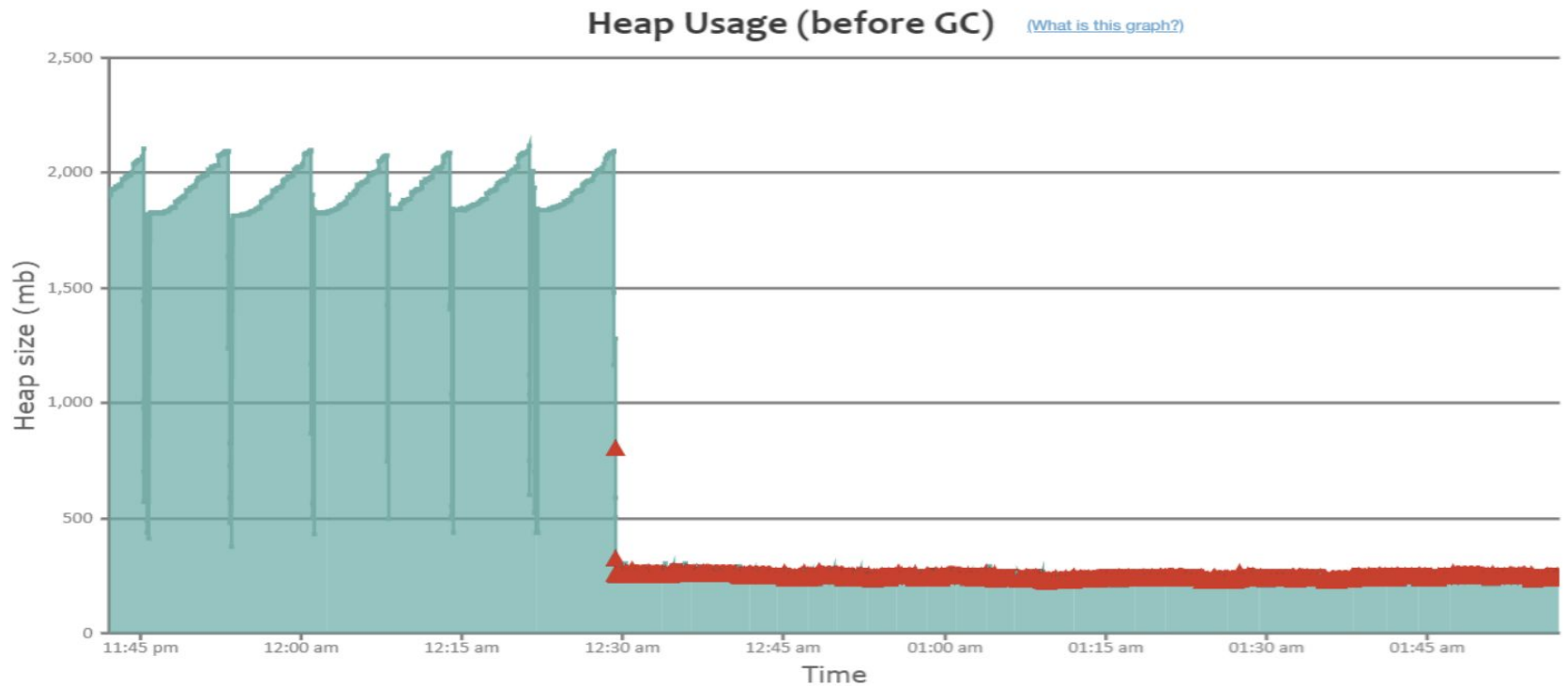
- In pattern 4, we can see what would happen if we under-allocate memory



Pattern 6

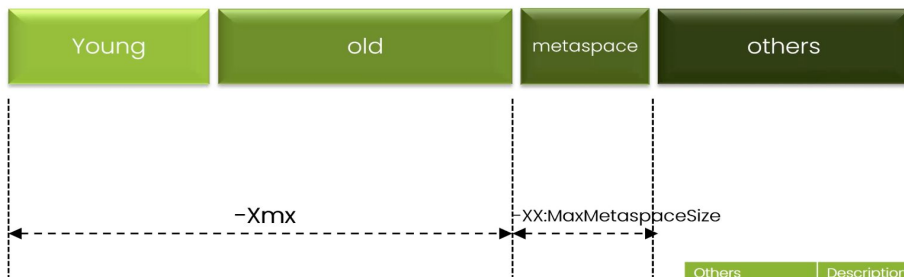
- Before getting into pattern 6, let's take a step back to understand some basics...

Pattern 6



Pattern 6

- In pattern 6, we can see what would happen if our metaspace size is underallocated - recall metaspace here:

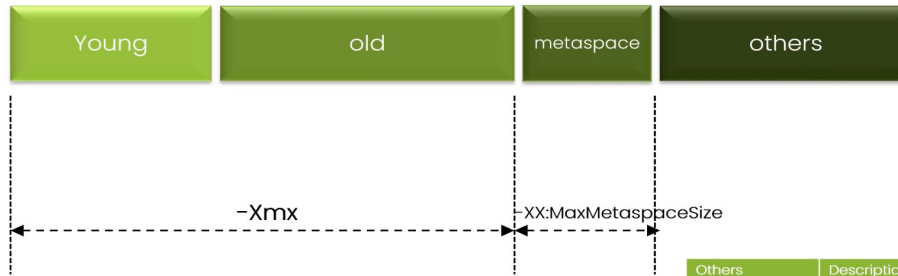


Young: Newly created Objects
Old: New objects that survived one or more minor GC promoted here
Metaspace: Classes, Methods, metadata

Others	Description
Thread Stacks	Each thread has a separate memory space. Controlled by <code>-Xss</code>
Garbage Collection	Threads, Memory to store GC info
Code Generation	Converting bytecode to native code
Socket Buffers	TCP Connections (Receive buffer ~37k, Send Buffer ~2.5k)
JNI	JNI program also allocate memory

Full GC vs Young GC

- Young GC - only runs on the Young generation
- Full GC runs on all regions - Young, Old, Metaspace



Young: Newly created Objects
Old: New objects that survived one or more minor GC promoted here
Metaspace: Classes, Methods, metadata

Others	Description
Thread Stacks	Each thread has a separate memory space. Controlled by -Xss
Garbage Collection	Threads, Memory to store GC info
Code Generation	Converting bytecode to native code
Socket Buffers	TCP Connections (Receive buffer ~37k, Send Buffer ~2.5k)
JNI	JNI program also allocate memory



Pattern 6

- In pattern 6, this was a problem faced by a very large financial company
- They were using Kotlin
- And they were also using some 3rd party library
- The library was connecting to the database and creating a new class definition file for every DB connection
- As a result the metaspace was growing (with more and more class definition files)



Pattern 6

- When a memory dump was taken it became quite visible that the metaspace was growing
- The way that they were interacting with the 3rd party library was incorrect - they were creating a new instance each time they connected
- But they needed to simply re-use the same instance every time instead!
- With no knowledge of Kotlin - just troubleshooting at the **JVM level** and doing some heap dump analysis- you can figure out the root problem
- A similar problem was observed with another company that was using Groovy scripts, where they were creating new class files for every request coming in

Pattern 6

- You can go here:
<https://gceasy.io/diamondgc-report.jsp?p=c2hhcmVkJzlwMjlvMDcvMTYvbWV0YXNwYWNIW9vbWVycm9yLmdjLS0xOC0xNy0zMg==&channel=WEB&s=t>
- Then go to Interactive Graphs -> Metaspace to see that the Metaspace was maxed out



When will the GC be invoked?

- Is it based off of a JVM arguments that we feed it?
- Or is it at the discretion of the JVM entirely?



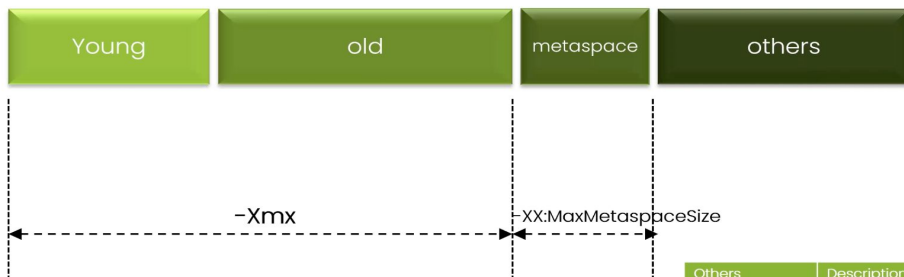
When will the GC be invoked?

- Answer: it's really based on 2 things:
- 1. The GC algorithm
- 2. The arguments that you pass the JVM

You can say: trigger the GC when the heap hits 70% or 40% - you can configure these args

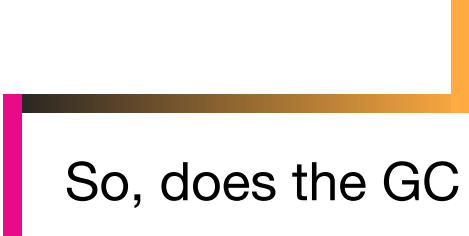
Is metaspace part of the heap?

- No, recall that the metaspace is not part of the heap - it's used to store class/methods metadata
- Heap is Young + Old generations of memory (not including metaspace)
- Recall that the heap size does not include the metaspace in the JVM



Young: Newly created Objects
Old: New objects that survived one or more minor GC promoted here
Metaspace: Classes, Methods, metadata

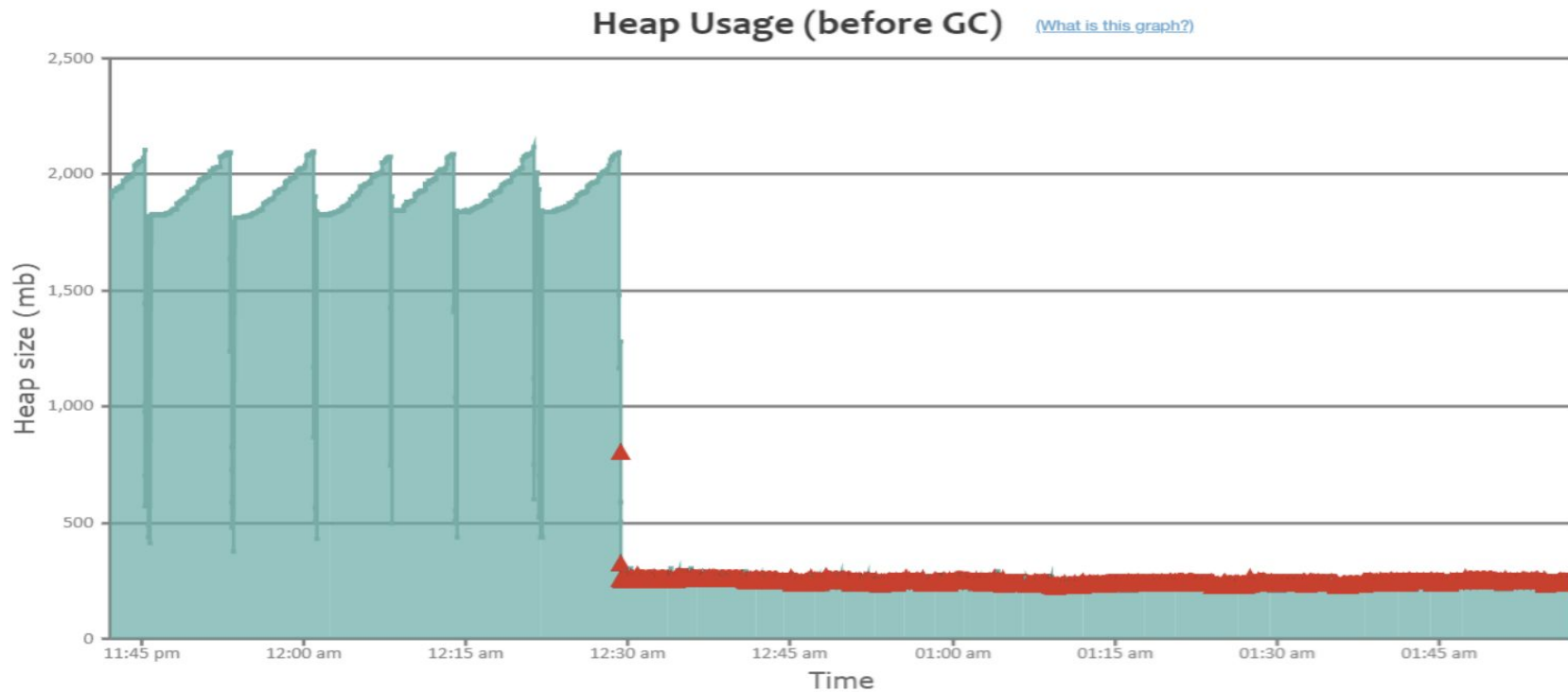
Others	Description
Thread Stacks	Each thread has a separate memory space. Controlled by -Xss
Garbage Collection	Threads, Memory to store GC info
Code Generation	Converting bytecode to native code
Socket Buffers	TCP Connections (Receive buffer ~37k, Send Buffer ~2.5k)
JNI	JNI program also allocate memory



So, does the GC run on the metaspace?

- Yes, garbage collection also runs on the metaspace even though it is not part of the heap!

Pattern 6





Pattern 6

- So, why would the FULL GC event run when heap memory usage is so low?
- Because the metaspace is actually the problem
- The metaspace is saturated!
- It has exceeded the max value set for the metaspace and that triggers the garbage collector to run
- In other words - the garbage collector runs across all 3 regions - young, old and metaspace



GC background

- The GC implementation lives in the JVM
- GC is automatic (but with some parameters we can configure)
- GC will identify unused objects and delete those for use by other objects...
- Keep in mind the GC only frees up space on the **heap** (objects) + metaspace + others, not on the stack (local variables)
- The JVM will free up space on the **stack**



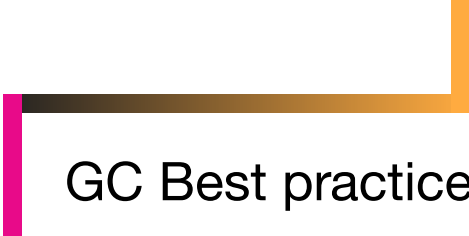
Why is Garbage Collection tuning so important?

- When the GC runs the application pauses for a few milliseconds
-



Why do our applications pause during garbage collection?

- When the GC runs the application pauses for some time
- This pause time is for the GC to run recover space that is no longer used
- We can try to limit the max pause time by setting a goal time
- But keep in mind that goal time is just a **hint** to the GC, not a hard requirement
- Also see:
<https://stackoverflow.com/questions/33803788/java-garbage-collection-fine-tuning-maximum-pause-time-goal>
- By default there is no max pause time



GC Best practices

- When studying GC usage - it's best to study it over the course of 24 hours
- This way you see the “high tide” and “low tide” of traffic
- Also you only want to study the GC Logs from your production environment - because it's impossible to replicate the production traffic in a dev/staging environment
- The GC Logs do not add any overhead
- What you can do is enable GC logs, and try tweaking the GC on ***just one instance*** (to minimize risk) to see if you get an improvement in performance



Is there any way to predict the metaspace size?

- Best thing to do is run a performance test and add a bit of extra space to what the logs tell you the max metaspace size is
- Metaspace size is just a function of the # of classes being loaded

JVM Arguments



JVM Arguments

- There are 600+ arguments that you can pass to JVM just around Garbage collection and memory.
- It's way too many arguments for anyone to digest and comprehend.
- Let's focus on some of these that can have a significant impact on your application performance

How would we specify xmx for docker containers?

- You have a few options to do this
- If your application is running as a Docker container, you can specify the xmx arguments as described here:
<https://blog.ycrash.io/2020/11/23/best-practices-java-memory-arguments-for-containers/>
- Note that fraction is deprecated but takes a portion of the overall memory assigned to your container (a value of 2 means half of the container memory)
- You can also use **XX:MaxRAMPercentage**, **-XX:MinRAMPercentage** - this would specify a percentage of the container size
-



How would we specify xmx for docker containers?

- Alternatively, you can specify **-xmx**, just as we have been doing beforehand
- So, which is the recommended approach?



How would we specify xmx for docker containers?

- We recommend just going with -xmx b/c we do not think it makes sense for the container size to dictate the heap size
- Containers are essentially wrappers around your application
- Think of it as this: the JVM is the body and the container is the shirt - so we don't want the shirt size to dictate the body size
- It should be the other way around - the body dictates the shirt size!
- We should say: "My JVM is this size, based on that, go ahead and figure out the container size!"
- V: what about kubernetes?? When and



Xmx vs Xms

- Xmx == max heap size (mx - think MAX)
- Xms == initial heap size, when my JVM starts how much memory the heap is set to (ms - think MIN SIZE)



Recommendation set $X_{mx} ==$ to X_{ms}

- It's recommended to set the X_{mx} value equal to the X_{ms} value
- There's an advantage- why?



Recommendation set Xmx == to Xms

- Why?
- Because there can be an improvement in the application's availability
- See here:
<https://blog.gceasy.io/2022/03/22/benefits-of-setting-initial-and-maximum-memory-size-to-the-same-value/>

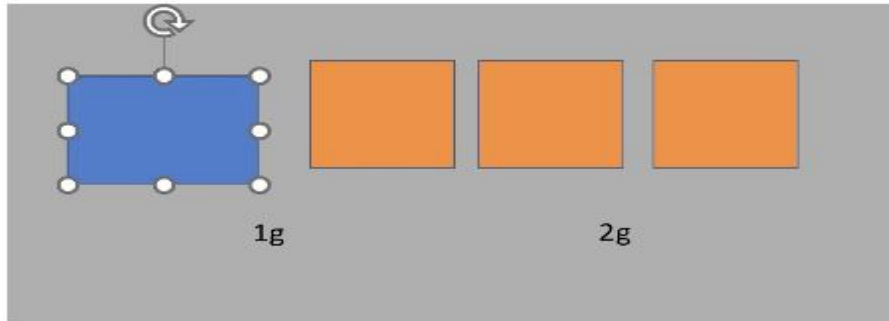


Recommendation set Xmx == to Xms

- Suppose our xmx is 2 GB (our max) and xms is 1 GB (our min)
- Our app will grow and shrink between that range of 1 GB and 2 GB
- If the app runs in container or some other machine
- On that machine, suppose it's running some cron job or maybe a Splunk script and those things are starting to take more memory
- Now, if your application was at say 1.2 GB of memory usage and it needs more memory, your application will crash because there is no more memory left!

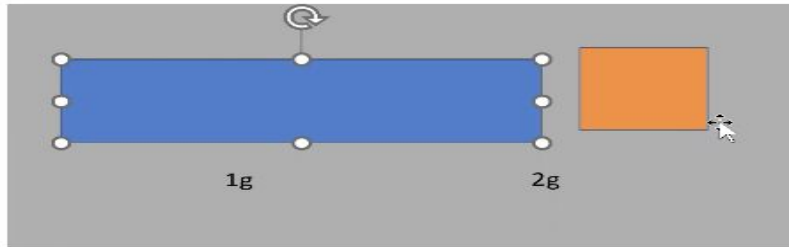
Recommendation set $X_{mx} ==$ to X_{ms}

- Suppose your app is blue and other apps are orange
- If your app is at 1 GB currently, there's no space for it to grow even though the x_{mx} is set to 2 GB
- And that is when it will crash!



Recommendation set $X_{mx} == X_{ms}$

- Now, suppose you actually set x_{mx} to x_{ms} (say to 2 GB)
- Then, your application will lay claim to the 2 GB because it starts at 2 GB
- What will happen is the other applications will crash instead!
- ***Whoever lays claim first to the memory gets to hold it!***
- Blue below represents your application:





Recommendation set Xmx == to Xms

- The other benefit of doing this would be performance - point 2 in this blog post:
<https://blog.gceasy.io/2022/03/22/benefits-of-setting-initial-and-maximum-memory-size-to-the-same-value/>



Recommendation set Xmx == to Xms

- When xmx is != not equal to xms, there is a constant negotiation that happens between the Operating system and the JVM for memory.
- When an app needs more memory it has to ask the OS
- Conversely, when an app needs less memory the OS will take it away
- This constant negotiation - back and forth – eats up resources

Recommendation set Xmx == to Xms

- In this study, first the app xmx was set to 24GB and xms set to 2 GB:
- This is the JVM's allocated and deallocated memory:

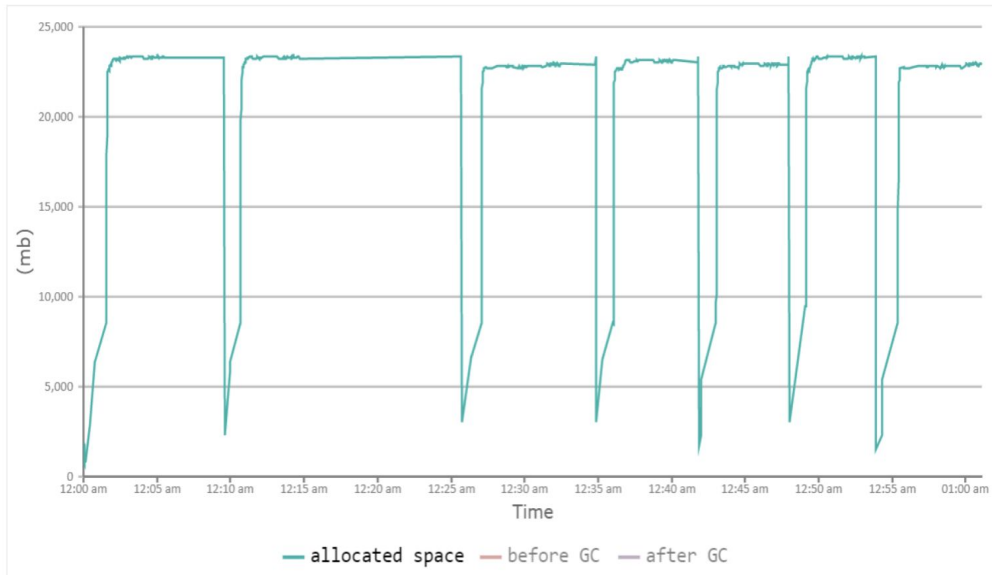


Fig: Scenario 1 memory allocation fluctuating (graph by [GGeasy](#))

Recommendation set Xmx == to Xms

- Then when xmx was set equal to xms
- ***Key takeaway: less negotiation between OS and JVM (because $xmx == xms$) translated to a 5% improvement in response time of the application***

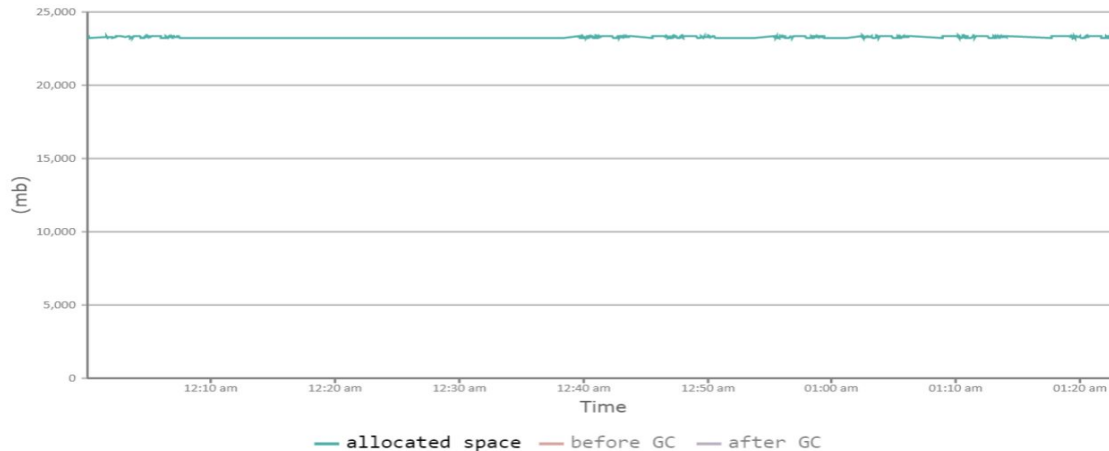


Fig: Scenario 2 memory allocation constant (graph by [GCeasy](#))



Why less negotiation is better

- Basically because Garbage Collection would have to run more often when `xmx` is not equal to `xms` - there's more overhead
- The measured GC throughput when `xmx == xms` was also 1% better (which is significant)

Impact on application start time

- We will also see an improvement in application **startup** time as well
- Remember that the JVM triggers garbage collection when it runs out of space on the heap
- The point of the GC is to free up space on the heap by cleaning up objects that are no longer being referenced
- From the Oracle docs: *If the initial heap is too small, the Java application startup becomes slow as the JVM is forced to perform garbage collection frequently until the heap grows to a more reasonable size. For optimal startup performance, set the initial heap size to the same as the maximum heap size.*
- https://docs.oracle.com/cd/E15289_01/JRPTG/tune_startup.htm



Impact on cost

- Your cloud instances (EC2 for example) will run at a fixed hourly cost
- You are typically choosing the size based on your max usage (xmx)
- So, if you set xmx equal to xms your costs on the cloud would be exactly the same
- You are paying the same amount whether you use the full memory allocated in the EC2 or not

GC Tuning - Key Metrics

What is garbage?



“What is garbage?”



How many garbage collector algorithms are there?

How many garbage collector algorithms are there?

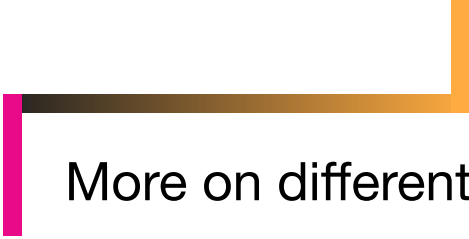
Java 11+:
How many Garbage
Collectors?

HotSpot

1. Serial
2. Parallel
3. CMS
4. G1
5. Shenandoah
6. Z GC
7. Epsilon

OpenJ9

1. Balanced
2. Concurrent
3. Mentrionome
4. Pause Time
5. Throughput
6. Epsilon



More on different JVM's

- Hotspot - you could be using either the Oracle version or OpenJDK version
- Open J9 is a high performance JVM they have their own 6 algorithms
- If you use the Azul JVM they have their own algorithms as well...



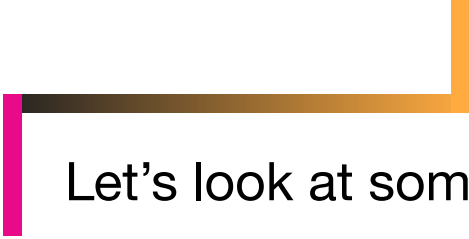
How many GC related JVM arguments are there?

**How many GC/Memory related
JVM arguments are available?**



How many GC related JVM arguments are there? Only for GC!

- Answer: over 600!
- This can be overwhelming, but you are not alone.
- GC is a very deep subject.
- Typically people are experts in certain areas of GC - like a G1 algorithm architect would know that specific algorithm very well



Let's look at some KPIs - Key Performance Indicators



Latency

- ***If it can be measured it can be improved!***
- How much time did it take for a GC event to complete?
- You want to understand your max pause times
- And you also want to understand the distribution as well

Latency

Avg Pause GC Time ?	34.7 ms
Max Pause GC Time ?	990 ms

Duration (ms)		No. of GCs	Percentage
200	ms		
Change			
0 - 200		769	99.23%
200 - 400		1	0.13%
400 - 600		2	0.26%
600 - 800		1	0.13%
800 - 1,000		2	0.26%

1. Latency

GC Event's Pause Time

Other metrics: GC Throughput and Memory/CPU/Response Time

Avg Pause GC Time 	34.7 ms
Max Pause GC Time 	990 ms

Duration (ms)		No. of GCs	Percentage
200	ms 		
			
0 - 200		769	99.23%
200 - 400		1	0.13%
400 - 600		2	0.26%
600 - 800		1	0.13%
800 - 1,000		2	0.26%

99.994%

Memory: 2GB
CPU: 45%
Response Time: 1.2 secs

1. Latency

GC Event's Pause Time

2. Throughput

Percentage of time spent in processing customer transactions vs time spent in GC activity, i.e. productive work vs non-productive work

3. Macrometrics

Memory, CPU, Response Time



Gathering these metrics

- Most of these metrics can be gathered with GC tooling
- Typically what folks do is use a tool, upload GC logs, modify GC algorithms and then measure to see the results
- Best to try this in production (perhaps on a single instance to minimize risk)
- Note: response time/CPU are not metrics you would get from your GC logs
- Adding GC logging does not add any overhead to your application
 - Enabling debug mode will add some overhead however

Average creation rate

- How do we ensure consistency across different tests of our application?
- Average creation rate is a good indicator (size of objects + metaspace data you are creating per sec)
- If it's consistent then indicates you are running the app the same way

⚙️ Object Stats ?

Total created bytes ?	3.67 tb
Total promoted bytes ?	121.33 gb
Avg creation rate ?	111.79 mb/sec
Avg promotion rate ?	3.6 mb/sec

How to source these KPIs?



Till Java 8

`-XX:+PrintGCDetails -XX:+PrintGCDateStamps -Xloggc:<file-path>`

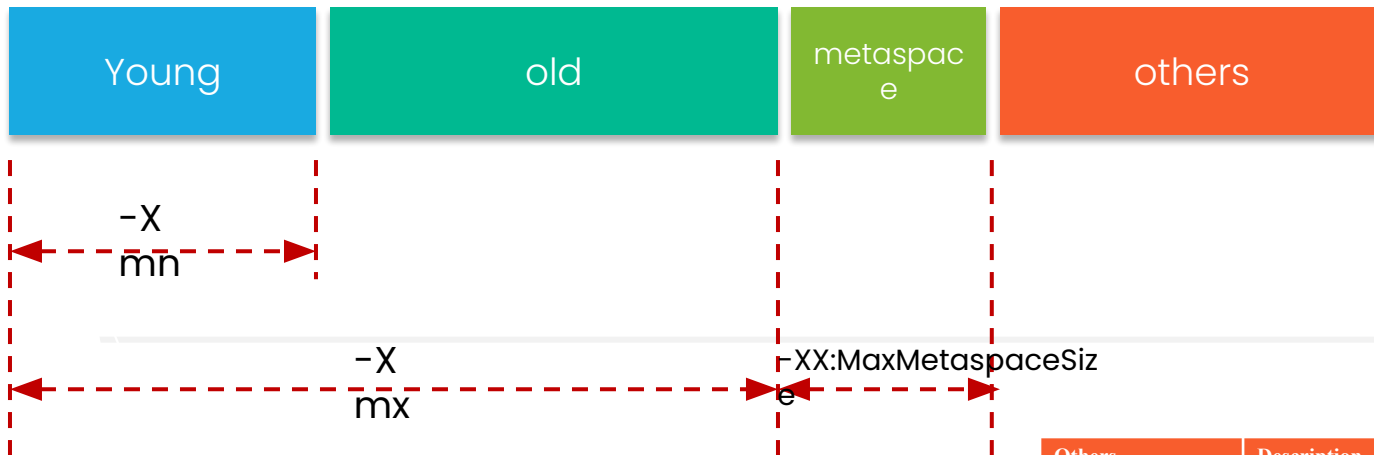
Enable GC logs (always)



From Java 9

`-Xlog:gc*:file=<file-path>`

Does Java process memory utilization go beyond -Xmx?

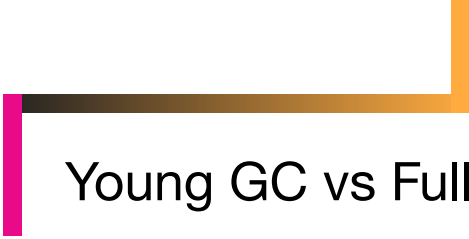


Young: Newly created Objects
Old: New objects that survived one or more minor GC promoted here
Metaspace: Classes, Methods, metadata

Others	Description
Thread Stacks	Each thread has a separate memory space. Controlled by -Xss
Garbage Collection	Threads, Memory to store GC info
Code Generation	Converting bytecode to native code
Socket Buffers	TCP Connections (Receive buffer ~37k, Send Buffer ~2.5k)
JNI	JNI program also allocate memory

The 2 types of JVM GC activities

- There are 2 different types of GC events within the JVM
- For our discussion we will call them Young GC (aka minor GC events) and Full GC
- Young GC - it only runs on the Young generation (region of memory)
 - Statistically it's been proven that 80% of objects are ***short lived and then they die down immediately***
 - When an object is first created it's put in the Young generation portion of memory
- When a new request comes in to your application it creates a bunch of new objects
- The Young GC/Minor GC event will run on the Young generation portion of memory



Young GC vs Full GC

- The Young GC will delete objects from memory very quickly
- Full GC events, however, will not just run on the young portion of memory - but on the young + old + metaspace
- So, typically the Full GC pause times are higher
- And the Young GC pause times are lower
- People also call the Young GC events as “Minor GC” as well
- There is a misconception in the industry that Young GC events do not pause the application - that is NOT true
- Young GC also pauses the application
- But since the young region is smaller, which means that there's less objects to perform garbage collection on, the pause time of young GC is smaller relative to FULL GC
- Again, full GC runs on larger region of memory (heap + metaspace), hence more cleanup to do (think: it's a bigger house to clean so more time needed)



GC Logs

- In 2nd event in logs: more objects were deleted from the young generation portion of memory than were promoted. So the net is that the heap size went down
- Generally, you would use a tool to help you analyze GC logs
- But it helps to look at one to understand it further...
- If you enable the GC logs this is how they would look (next slide):

Vanilla Format

2019-08-31T01:09:19.397+0000: 1.606: [GC (Metadata GC Threshold) [PSYoungGen: 545393K->18495K(2446848K)] 545393K->18519K(8039424K), 0.0189376 secs] [Times: user=0.15 sys=0.01, real=0.02 secs]

2019-08-31T01:09:19.416+0000: 1.625: [Full GC (Metadata GC Threshold) [PSYoungGen: 18495K->0K(2446848K)] [ParOldGen: 24K->17366K(5592576K)] 18519K->17366K(8039424K), [Metaspace: 20781K->20781K(1067008K)], 0.0416162 secs] [Times: user=0.38 sys=0.03, real=0.04 secs]

2019-08-31T01:18:19.288+0000: 541.497: [GC (Metadata GC Threshold) [PSYoungGen: 1391495K->18847K(2446848K)] 1408861K->36230K(8039424K), 0.0568365 secs] [Times: user=0.31 sys=0.75, real=0.06 secs]

2019-08-31T01:18:19.345+0000: 541.554: [Full GC (Metadata GC Threshold) [PSYoungGen: 18847K->0K(2446848K)] [ParOldGen: 17382K->25397K(5592576K)] 36230K->25397K(8039424K), [Metaspace: 34865K->34865K(1079296K)], 0.0467640 secs] [Times: user=0.31 sys=0.08, real=0.04 secs]

2019-08-31T02:33:20.326+0000: 5042.536: [GC (Allocation Failure) [PSYoungGen: 2097664K->11337K(2446848K)] 2123061K->36742K(8039424K), 0.3298985 secs] [Times: user=0.00 sys=9.20, real=0.33 secs]

2019-08-31T03:40:11.749+0000: 9053.959: [GC (Allocation Failure) [PSYoungGen: 2109001K->15776K(2446848K)] 2134406K->41189K(8039424K), 0.0517517 secs] [Times: user=0.00 sys=1.22, real=0.05 secs]

2019-08-31T05:11:46.869+0000: 14549.079: [GC (Allocation Failure) [PSYoungGen: 2113440K->24832K(2446848K)] 2138853K->50253K(8039424K), 0.0392831 secs] [Times: user=0.02 sys=0.79, real=0.04 secs]

2019-08-31T06:26:10.376+0000: 19012.586: [GC (Allocation Failure) [PSYoungGen: 2122496K->25600K(2756096K)] 2147917K->58149K(8348672K), 0.0371416 secs] [Times: user=0.01 sys=0.75, real=0.04 secs]

2019-08-31T07:50:03.442+0000: 24045.652: [GC (Allocation Failure) [PSYoungGen: 2756096K->32768K(2763264K)] 2788645K->72397K(8355840K), 0.0709641 secs] [Times: user=0.16 sys=1.39, real=0.07 secs]

2019-08-31T09:04:21.406+0000: 28503.616: [GC (Allocation Failure) [PSYoungGen: 2763264K->32768K(2733568K)] 2802893K->83469K(8326144K), 0.0789178 secs] [Times: user=0.12 sys=1.59, real=0.08 secs]

GC Logs

- For every GC event one line is generated in the GC logs
- First thing we see is the timestamp - it ran on Aug. 31st at midnight
- Then it shows 1.606 - that means after 1.606 seconds of the JVM starting this GC event was triggered
- If it says FULL GC it was a full GC event, but if no “full” it ran on the young generation
- YoungGen size is 545 MB before the GC event ran, once it ran the young generation size dropped from 545 to 18 MB
- It has reclaimed the unreferenced objects from the memory - the young generation
- Then in parentheses we see the overall young gen size (the max size)
- And we also see after that the overall heap size (500 MB before 18.5 MB after) - and also the XMX == heap max of 8 GB
- Generally you only want to look at the real time of .02 seconds - wall clock time...
- We will talk about user and sys time later...

GC Log Format varies

JVM Vendor
Oracle
HP
IBM
Azul
OpenJDK
...

Java Version
1.4
5
6
7
8
9
10
11
12
13
14

GC algorithm
Serial
Parallel
CMS
G1
Shennandoh
Z GC
Epsilon

Arguments
-XX:+PrintGC
-XX:+PrintGCDateStamps
-XX:+PrintGCDetails
-XX:+PrintGCTimeStamps
-XX:+PrintPromotionFailure
-XX:+PrintGCApplicationStoppe dTime
-XX:+PrintClassHistogram
-XX:PrintFLSStatistics=1
-XX:+PrintCodeCache

G1 GC Format

2015-09-14T11:58:55.131-0700: 0.519: [GC pause (G1 Evacuation Pause) (young), 0.0096438 secs]

[Parallel Time: 7.9 ms, GC Workers: 8]

[GC Worker Start (ms): Min: 519.4, Avg: 519.6, Max: 520.6, Diff: 1.3]

[Ext Root Scanning (ms): Min: 0.0, Avg: 2.9, Max: 7.3, Diff: 7.3, Sum: 23.4]

[Update RS (ms): Min: 0.0, Avg: 0.0, Max: 0.0, Diff: 0.0, Sum: 0.0]

[Processed Buffers: Min: 0, Avg: 0.0, Max: 0, Diff: 0, Sum: 0]

[Scan RS (ms): Min: 0.0, Avg: 0.0, Max: 0.0, Diff: 0.0, Sum: 0.0]

[Code Root Scanning (ms): Min: 0.0, Avg: 0.0, Max: 0.1, Diff: 0.1, Sum: 0.1]

[Object Copy (ms): Min: 0.0, Avg: 4.2, Max: 7.2, Diff: 7.2, Sum: 34.0]

[Termination (ms): Min: 0.0, Avg: 0.2, Max: 0.4, Diff: 0.4, Sum: 1.7]

[Termination Attempts: Min: 1, Avg: 7.9, Max: 18, Diff: 17, Sum: 63]

[GC Worker Other (ms): Min: 0.0, Avg: 0.0, Max: 0.1, Diff: 0.1, Sum: 0.4]

[GC Worker Total (ms): Min: 6.4, Avg: 7.4, Max: 7.7, Diff: 1.3, Sum: 59.6]

[GC Worker End (ms): Min: 527.0, Avg: 527.1, Max: 527.1, Diff: 0.1]

[Code Root Fixup: 0.0 ms]

[Code Root Purge: 0.0 ms]

[Clear CT: 0.5 ms]

[Other: 1.3 ms]

[Choose CSet: 0.0 ms]

[Ref Proc: 0.7 ms]

[Ref Enq: 0.0 ms]

[Redirty Cards: 0.3 ms]

[Humongous Register: 0.0 ms]

[Humongous Reclaim: 0.0 ms]

[Free CSet: 0.0 ms]

[Eden: 24.0M(24.0M)->0.0B(34.0M) Survivors: 0.0B->3072.0K Heap: 24.0M(252.0M)->3338.0K(252.0M)]

CMS GC Format

Before GC:

Statistics for BinaryTreeDictionary:

Total Free Space: 2524251

Max Chunk Size: 2519552

Number of Blocks: 13

Av. Block Size: 194173

Tree Height: 8

2016-05-03T04:27:37.503+0000: 30282.678: [ParNew

Desired survivor size 214728704 bytes, new threshold 1 (max 1)

- age 1: 85782640 bytes, 85782640 total

: 3510063K->100856K(3774912K), 0.0516290 secs] 9371816K->6022161K(14260672K)After GC:

Statistics for BinaryTreeDictionary:

Total Free Space: 530579346

Max Chunk Size: 332576815

Number of Blocks: 7178

Av. Block Size: 73917

Tree Height: 44

After GC:

Statistics for BinaryTreeDictionary:

Total Free Space: 2524251

Max Chunk Size: 2519552

Number of Blocks: 13

Av. Block Size: 194173

Tree Height: 8

IBM GC Format

```
<af type="tenured" id="4" timestamp="Jun 16 11:28:22 2016" intervals="5633.039">
  <minimum requested_bytes="56" />
  <time exclusiveaccessms="0.010" meanexclusiveaccessms="0.010" threads="0" lastthreadtid="0xF6B1C400" />
  <refs soft="7232" weak="3502" phantom="9" dynamicSoftReferenceThreshold="30" maxSoftReferenceThreshold="32" />
  <tenured freebytes="42949632" totalbytes="1073741824" percent="3" >
    <soa freebytes="0" totalbytes="1030792192" percent="0" />
    <loa freebytes="42949632" totalbytes="42949632" percent="100" />
  </tenured>
  <pending-finalizers finalizable="0" reference="0" classloader="0" />
  <gc type="global" id="6" totalid="6" intervals="3342.687">
    <classunloading classloaders="0" classes="0" timevmquiescems="0.000" timetakenms="1.200" />
    <finalization objectsqueued="75" />
    <timesms mark="28.886" sweep="1.414" compact="0.000" total="31.571" />
    <tenured freebytes="1014673616" totalbytes="1073741824" percent="94" >
      <soa freebytes="982461648" totalbytes="1041529856" percent="94" />
      <loa freebytes="32211968" totalbytes="32211968" percent="100" />
    </tenured>
  </gc>
  <tenured freebytes="1014608080" totalbytes="1073741824" percent="94" >
    <soa freebytes="982396112" totalbytes="1041529856" percent="94" />
    <loa freebytes="32211968" totalbytes="32211968" percent="100" />
  </tenured>
  <refs soft="7020" weak="2886" phantom="9" dynamicSoftReferenceThreshold="30" maxSoftReferenceThreshold="32" />
  <pending-finalizers finalizable="75" reference="15" classloader="0" />
  <time totalms="33.852" />
</af>
```

IBM GC another Format

```
<gc-op id="139" type="scavenge" timems="335.610" contextid="136"
timestamp="2016-06-15T15:51:10.128">
  <scavenger-info tenureage="4" tenuremask="7ff0" tiltratio="58" />
  <memory-copied type="nursery" objects="11071048" bytes="448013400"
bytesdiscarded="88016" />
  <memory-copied type="tenure" objects="286673" bytes="9771936"
bytesdiscarded="320608" />
  <copy-failed type="nursery" objects="286673" bytes="9771936" />
  <finalization candidates="112" enqueued="16" />
  <ownableSynchronizers candidates="8111" cleared="11" />
  <references type="soft" candidates="1256" cleared="0" enqueued="0"
dynamicThreshold="32" maxThreshold="32" />
  <references type="weak" candidates="2953" cleared="0" enqueued="0"
/>
  <references type="phantom" candidates="142406" cleared="142335"
enqueued="142335" />
</gc-op>
```

Android Dalvik GC Format

07-01 15:56:20.035: I/Choreographer(30615): Skipped 141 frames! The application may be doing too much work on its main thread.

07-01 15:56:20.275: D/dalvikvm(30615): GC_FOR_ALLOC freed 4774K, 45% free 49801K/89052K, paused 168ms, total 168ms

07-01 15:56:20.295: I/dalvikvm-heap(30615): Grow heap (frag case) to 56.900MB for 4665616-byte allocation

07-01 15:56:21.315: D/dalvikvm(30615): GC_FOR_ALLOC freed 1359K, 42% free 55045K/93612K, paused 95ms, total 95ms

07-01 15:56:21.965: D/dalvikvm(30615): GC_CONCURRENT freed 6376K, 40% free 56861K/93612K, paused 16ms+8ms, total 126ms

07-01 15:56:21.965: D/dalvikvm(30615): WAIT_FOR_CONCURRENT_GC blocked 111ms

07-01 15:56:21.965: D/dalvikvm(30615): WAIT_FOR_CONCURRENT_GC blocked 97ms

Android ART GC Format

07-01 16:00:44.690: I/art(801): Explicit concurrent mark sweep GC freed 65595(3MB) AllocSpace objects, 9(4MB) LOS objects, 810% free, 38MB/58MB, paused 1.195ms total 87.219ms

07-01 16:00:46.517: I/art(29197): Background partial concurrent mark sweep GC freed 74626(3MB) AllocSpace objects, 39(4MB) LOS objects, 1496% free, 25MB/32MB, paused 4.422ms total 1.371747s

07-01 16:00:48.534: I/Choreographer(29197): Skipped 30 frames! The application may be doing too much work on its main thread.

07-01 16:00:48.566: I/art(29197): Background sticky concurrent mark sweep GC freed 70319(3MB) AllocSpace objects, 59(5MB) LOS objects, 825% free, 49MB/56MB, paused 6.139ms total 52.868ms

07-01 16:00:49.282: I/Choreographer(29197): Skipped 33 frames! The application may be doing too much work on its main thread.

'Free' GC Log analysis tools

Freely available Garbage collection log analysis tools

01

GCeasy

<https://gceasy.io/>

02

GC Viewer

<https://github.com/chewiebug/GCViewer>

03

IBM GC & Memory visualizer

<https://developer.ibm.com/javasdk/tools/>

04

HP Jmeter

<https://h20392.www2.hp.com/portal/swdepot/displayProductInfo.do?productNumber=HPJMeter>

05

Google Garbage cat (cms)

<https://code.google.com/archive/a/eclipse-labs.org/project/garbagecat>



GC tuning tips & tricks

<https://blog.gceasy.io/2016/11/22/reduce-long-gc-pauses/>

1. Start from scratch

Remove all GC related JVM arguments



Student of my training program

Presented in DevOps/Jenkins world
conference
<https://tinyurl.com/jwperformance>

Ryan Smith
Senior Developer Support Engineer, CloudBees

Real world data from Big Bank

Key Performance Indicators

(Important section of the report. To learn more about KPIs, [click here](#))

1 Throughput : 92.078%

2 Latency:

Avg Pause GC Time

265 ms

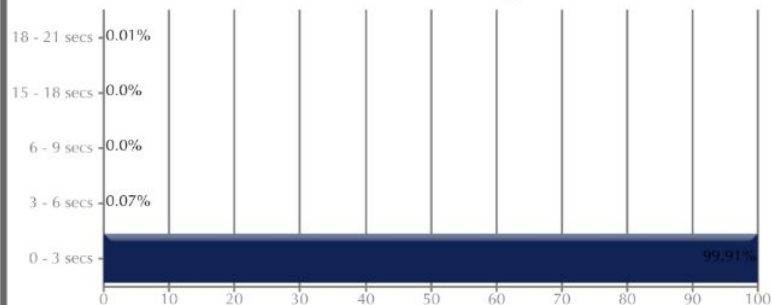
Max Pause GC Time

20 sec 600 ms

GC Pause Duration Time Range

Duration (secs)	No. of GCs	Percentage
0 - 3	41742	99.911%
3 - 6	31	99.986%
6 - 9	1	99.988%
15 - 18	2	99.993%
18 - 21	3	100.0%

GC Duration Time Range



~~-XX:-UseAdaptiveSizePolicy
-XX:G1NewSizePercent=20
-XX:MaxMetaspaceExpansion=64M
-XX:G1SummarizeSetStatsPeriod=1
-XX:G1HeapRegionSize=4m
-XX:MetaspaceSize=1024m
-XX:MaxMetaspaceSize=2048m~~

After removing all GC parameters

RESOLUTION: From ~92% to 99% Application Throughput

Key Performance Indicators

(Important section of the report. To learn more about KPIs, [click here](#))

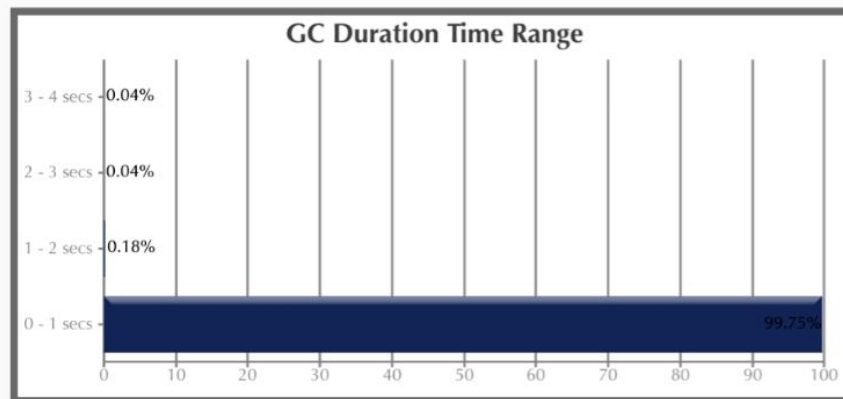
1 Throughput : 99.468%

2 Latency:

Avg Pause GC Time	127 ms
Max Pause GC Time	3 sec 250 ms

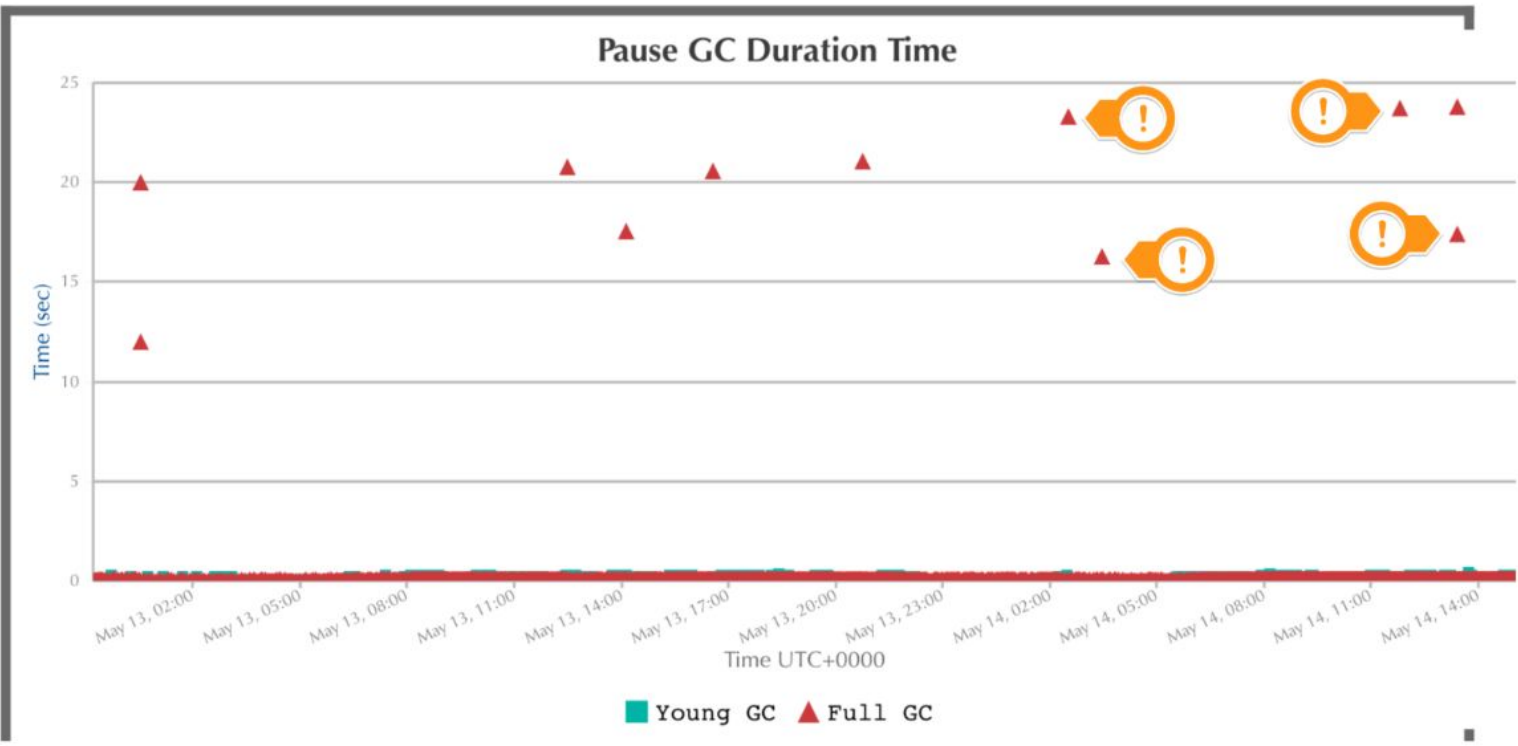
GC Pause Duration Time Range

Duration (secs)	No. of GC	Percentage
0 - 1	2839	99.754%
1 - 2	5	99.93%
2 - 3	1	99.965%
3 - 4	1	100.0%



Real World Data from BIG SHIPPING COMPANY

Scenario: It was reported that HA Failover was occurring daily, leading to multiple production outages and downtime for Jenkins users. Looking at the GC Data, the following was observed:

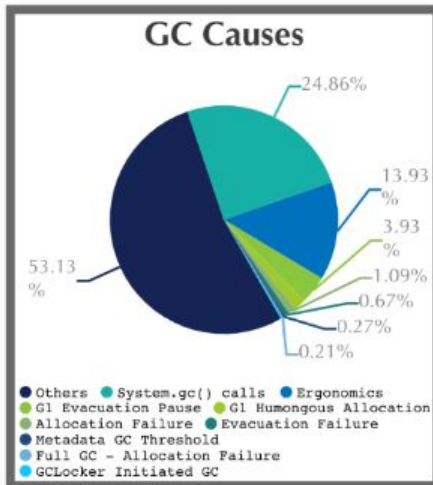


2. Study GC Causes

? GC Causes ?

(What events caused the GCs, how much time it consumed?)

Cause	Count	Avg Time	Max Time	Total Time	Time %
Others	n/a	n/a	n/a	24 min 41 sec 829 ms	53.14%
System.gc() calls	104	6 sec 666 ms	11 sec 176 ms	11 min 33 sec 284 ms	24.86%
Ergonomics	12609	30.8 ms	5 sec 846 ms	6 min 28 sec 324 ms	13.93%
G1 Evacuation Pause	5896	18.6 ms	1 sec 837 ms	1 min 49 sec 667 ms	3.93%
G1 Humongous Allocation	3642	13.3 ms	2 sec 908 ms	48 sec 417 ms	1.74%
Allocation Failure	27	1 sec 124 ms	5 sec 846 ms	30 sec 354 ms	1.09%
Evacuation Failure	25	744 ms	1 sec 837 ms	18 sec 596 ms	0.67%
Metadata GC Threshold	438	16.9 ms	160 ms	7 sec 417 ms	0.27%
Full GC - Allocation Failure	1	5 sec 846 ms	5 sec 846 ms	5 sec 846 ms	0.21%
GCLocker Initiated GC	217	21.8 ms	1 sec 105 ms	4 sec 721 ms	0.17%
Total	22959	n/a	n/a	46 min 28 sec 455 ms	100.01%



3. Remove System.gc()



`System.gc()` or `Runtime.getRuntime().gc()`

Always causes stop-the-world Full GCs. Try to avoid it. Because you are intervening with GC's ergonomics



May not always originate from your source code

- 3rd party libraries, frameworks, sometimes even application servers
- External tools (like VisualVM) through use of JMX.
- RMI.



`-XX:+DisableExplicitGC`

This JVM argument disables `System.gc()` globally in your JVM.

Real World Data from BIG SHIPPING COMPANY

RESOLUTION: 3500% PERFORMANCE INCREASE

- 99% Throughput
- From 12-23 second pauses to 660ms
- 89ms avg

🔍 Key Performance Indicators

(Important section of the report. To learn more about KPIs, [click here](#))

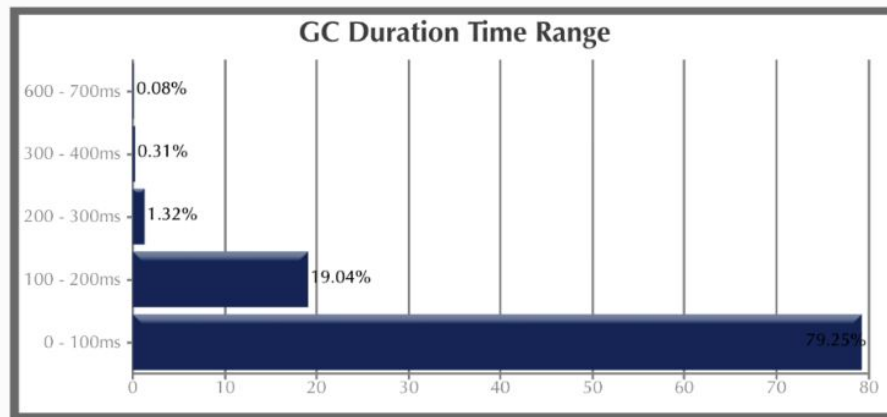
1 Throughput 📈 : 99.834%

2 Latency:

Avg Pause GC Time 📈	89.1 ms
Max Pause GC Time 📈	660 ms

GC Pause Duration Time Range 📈:

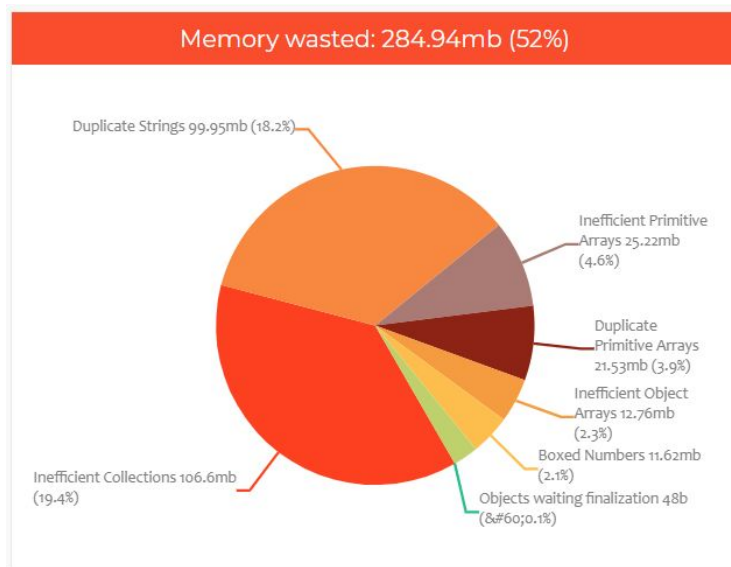
Duration (ms)	No. of GCs	Percentage
100 ms ⬆️ Change		
0 - 100	1020	79.25%
100 - 200	245	19.04%
200 - 300	17	1.32%
300 - 400	4	0.31%
600 - 700	1	0.08%



4. High object creation rate

30 – 90% memory wasted
due to inefficient programming

Total created bytes ?	298.29 gb
Total promoted bytes ?	70.44 gb
Avg creation rate ?	618.81 mb/sec
Avg promotion rate ?	146.13 mb/sec

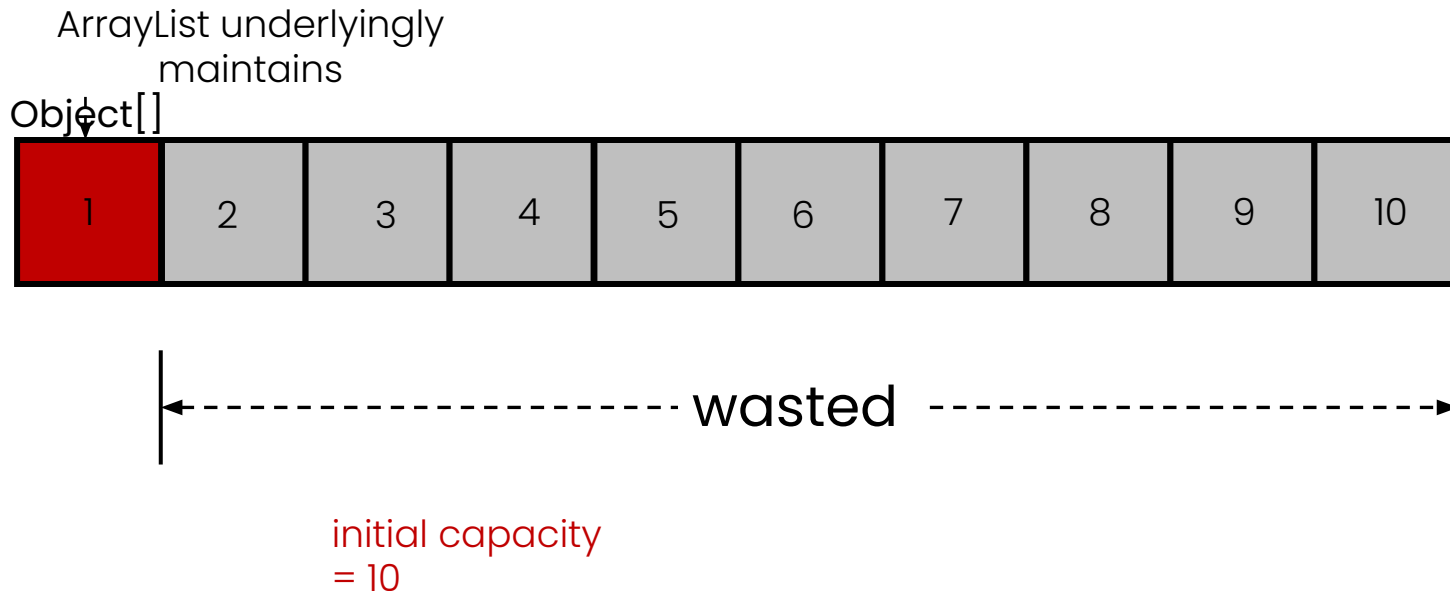


<https://tinyurl.com/y3q5j5oj>

oj

Inefficiency in Collections

```
List<User> users = new ArrayList<>();  
users.add(user);
```



Inefficiency in Collections

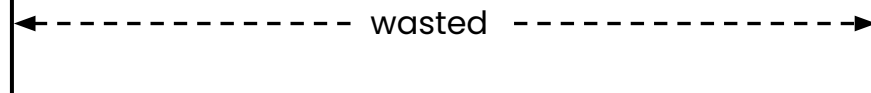
```
List<User> users = new ArrayList<>();  
for (int counter = 1; counter <= 11; ++counter)  
{
```

```
    users.add(user);  
}
```



```
    for (int counter = 1; counter <= 21; ++counter) {
```

```
        users.add(user);  
    }
```



Recommendation 1: use capacity

```
new ArrayList<>();
```



```
new ArrayList<>(3);
```



Recommendation 2: Lazy Initialization

```
private List<User> users = new ArrayList<>();

public void addUser(User user) {

    users.add(user);
}
```



```
private List<User> users;

public void addUser(User user) {

    if (users == null) {
        users = new ArrayList<>();
    }

    users.add(user);
}
```



Recommendation 3: avoid clear()

```
List<User> users = new ArrayList<>();  
users.clear();
```



```
List<User> users = new ArrayList<>();  
users = null;
```



How all memory is wasted?

1. Duplicate Strings: <https://heaphero.io/heap-recommendations/duplicate-strings.html>
2. Wrong memory size settings
3. Inefficient Collections:
<http://heaphero.io/heap-recommendations/inefficient-collections.html>
4. Duplicate Objects: <https://heaphero.io/heap-recommendations/duplicate-objects.html>
5. Duplicate arrays:
<https://heaphero.io/heap-recommendations/duplicate-primitive-arrays.html>
6. Inefficient arrays:
<https://heaphero.io/heap-recommendations/inefficient-primitive-arrays.html>
7. Objects waiting for finalization:
<https://heaphero.io/heap-recommendations/objects-waiting-finalization.html>
8. Boxed numbers: <https://heaphero.io/heap-recommendations/boxed-numbers.html>
9. Object Headers: <https://heaphero.io/heap-recommendations/object-headers.html>

5. Choice of GC Algorithm

Choice GC algorithm plays a key role in GC performance.

01

Serial

02

Parallel

03

CMS

04

G1

05

Shenandoah

06

Z GC

GC Time

[Times: user=11.53 sys=1.38, real=1.03 secs]

Sys time == time that the GC spends talking to the kernel/OS

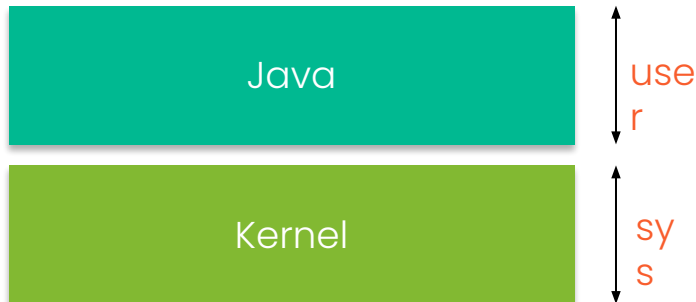
User time == time the GC spends in the Java layer, marking references, unreferenced objects, etc..

User time + sys time == real time - but on a per thread basis

You will have multiple GC threads



Real: Wall clock time



Pattern: Sys > User Time

🕒 System Time greater than User Time ⚠️

In 149 GC event(s), 'sys' time is greater than 'usr' time. It's not a healthy sign.

Timestamp	User Time (secs)	Sys Time (secs)	Real Time (secs)
2016-08-31T01:18:19	0.31	0.75	0.06
2016-08-31T05:11:46	0.02	0.79	0.04
2016-08-31T06:26:10	0.01	0.75	0.04
2016-08-31T07:50:03	0.16	1.39	0.07
2016-08-31T09:04:21	0.12	1.59	0.08
2016-08-31T11:33:31	0.39	1.25	0.08
2016-08-31T14:31:18	0.31	2.08	0.11
2016-08-31T15:46:17	0.4	2.72	0.14
2016-08-31T17:01:16	0.56	1.86	0.11
2016-08-31T18:16:15	0.54	2.59	0.14

6. Process swapping



Lack of RAM (Memory)

Sometimes due to lack of memory (RAM), Operating system could be swapping your application from memory.



Process swapping

script will show all the process that are being swapped:

<https://blog.gceasy.io/2016/11/22/reduce-long-gc-pauses/>

Pattern: Real Time > User Time + Sys Time

🕒 Application waiting for resources

It looks like your application is waiting due to lack of compute resources (either CPU or memory). In 2 GC event(s), 'real' time took more than 'usr' + 'sys' time.

Timestamp	User Time (secs)	Sys Time (secs)	Real Time (secs)
	326.85	38.84	417.04
	92.68	7.11	230.32

Read our recommendations to [fix this problem](#)

7. Less GC Threads

- WARNING: Adding too many GC threads will consume a lot of CPU and takes away a resource from your application. Thus you need to conduct thorough testing before increasing the GC thread count.

What artifacts/logs does the JVM create? 3 things

1. GC (Garbage Collection) Log - only contains #'s, no sensitive information here about your applications (usually not classified as a confidential information by organizations)
2. Thread dump - snapshot of threads it may have your code, IP's, 3rd party libraries - you still want to be careful with this - MAY CONTAIN SENSITIVE INFORMATION
3. Heap dump - this is ULTRA confidential because it's a snapshot of memory, so it could have SSN's, customer information that your application may analyze.
 1. If you upload this ycrash will delete the heap dump right after analysis...BECAUSE ITS VERY SENSITIVE INFORMATION

Thread Dump analysis



JVM generated logs

- The JVM generates 4 main artifacts/logs:
- 1. The GC Logs
- 2. The thread dump (snapshot of all your threads)
- 3. Heap dump (snapshot of your heap memory)
- 4. Hs_err_pid (when the JVM crashes - we are not so focused on this)

Step 1: Confirm

'top' tool is your good friend

```
top - 23:13:26 up 102 days, 21:09, 2 users, load average: 2.91, 2.99, 2.55
Tasks: 99 total, 1 running, 98 sleeping, 0 stopped, 0 zombie
Cpu(s):100.0%us, 0.0%sy, 0.0%ni, 0.0%id, 0.0%wa, 0.0%hi, 0.0%si, 0.0%st
Mem: 8178640k total, 3821576k used, 4357064k free, 158700k buffers
Swap: 0k total, 0k used, 0k free, 646524k cached
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
31294	ec2-user	20	0	4415m	22m	14m	S	199.1	0.3	0:28.53	java
3108	newrelic	20	0	239m	7708	4784	S	0.7	0.1	59:09.51	nrsysmond
15153	tomcat	20	0	9132m	2.6g	17m	S	0.3	33.7	134:34.52	java
1	root	20	0	19640	2676	2344	S	0.0	0.0	0:07.55	init
2	root	20	0	0	0	0	S	0.0	0.0	0:00.00	kthreadd
3	root	20	0	0	0	0	S	0.0	0.0	0:10.95	ksoftirqd/0
5	root	0	-20	0	0	0	S	0.0	0.0	0:00.00	kworker/0:0H
6	root	20	0	0	0	0	S	0.0	0.0	1:27.11	kworker/u30:0
7	root	20	0	0	0	0	S	0.0	0.0	3:22.96	rcu_sched
8	root	20	0	0	0	0	S	0.0	0.0	0:00.00	rcu_bh
9	root	RT	0	0	0	0	S	0.0	0.0	0:01.96	migration/0
10	root	RT	0	0	0	0	S	0.0	0.0	0:01.44	migration/1
11	root	20	0	0	0	0	S	0.0	0.0	0:14.63	ksoftirqd/1
13	root	0	-20	0	0	0	S	0.0	0.0	0:00.00	kworker/1:0H
14	root	20	0	0	0	0	S	0.0	0.0	0:00.00	kdevtmpfs
15	root	0	-20	0	0	0	S	0.0	0.0	0:00.00	netns
18	root	0	-20	0	0	0	S	0.0	0.0	0:00.00	perf
22	root	20	0	0	0	0	S	0.0	0.0	0:00.00	xenwatch
23	root	20	0	0	0	0	S	0.0	0.0	0:00.00	xenbus
134	root	20	0	0	0	0	S	0.0	0.0	0:03.71	khungtaskd
135	root	0	-20	0	0	0	S	0.0	0.0	0:00.00	writeback



Troubleshooting the CPU spike

- Let's say the CPU spikes
- There are plenty of tools that can detect that for us
- But the real question is which line of code led to the spike?
- First, let's talk about the manual approach - so you can see what happens under the hood
- Later we will see how to utilize tools to help us out



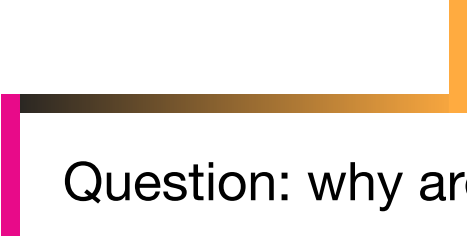
Troubleshooting the CPU spike

- People may say the application has crashed, its unresponsive (only responding to a few transactions, not to others)
- We want to confirm for ourselves and see the data firsthand!
- Let's not trust secondhand information from others



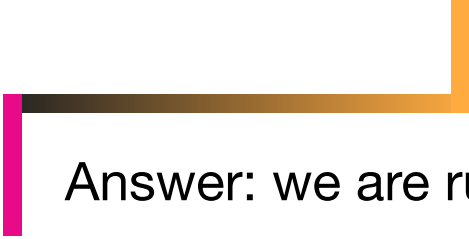
Troubleshooting the CPU spike

- If you happen to have access to the production machine/container - you can use the “top” command shown below
- In screenshot below, we can see PID of 31294 consuming 199% of CPU
- Now, we want to identify the line of code which is causing the CPU to spike...
- Whenever a CPU spike occurs



Question: why are we seeing a 199% CPU usage

- How/why is our CPU going over 100%? What do you think?



Answer: we are running two cores on this machine

Step 1: Confirm

'top' tool is your good friend

```
top - 23:13:26 up 102 days, 21:09, 2 users, load average: 2.91, 2.99, 2.55
Tasks: 99 total, 1 running, 98 sleeping, 0 stopped, 0 zombie
Cpu(s):100.0%us, 0.0%sy, 0.0%ni, 0.0%id, 0.0%wa, 0.0%hi, 0.0%si, 0.0%st
Mem: 8178640k total, 3821576k used, 4357064k free, 158700k buffers
Swap: 0k total, 0k used, 0k free, 646524k cached
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
31294	ec2-user	20	0	4415m	22m	14m	S	199.1	0.3	0:28.53	java
3108	newrelic	20	0	239m	7708	4784	S	0.7	0.1	59:09.51	nrsysmond
15153	tomcat	20	0	9132m	2.6g	17m	S	0.3	33.7	134:34.52	java
1	root	20	0	19640	2676	2344	S	0.0	0.0	0:07.55	init
2	root	20	0	0	0	0	S	0.0	0.0	0:00.00	kthreadd
3	root	20	0	0	0	0	S	0.0	0.0	0:10.95	ksoftirqd/0
5	root	0	-20	0	0	0	S	0.0	0.0	0:00.00	kworker/0:0H
6	root	20	0	0	0	0	S	0.0	0.0	1:27.11	kworker/u30:0
7	root	20	0	0	0	0	S	0.0	0.0	3:22.96	rcu_sched
8	root	20	0	0	0	0	S	0.0	0.0	0:00.00	rcu_bh
9	root	RT	0	0	0	0	S	0.0	0.0	0:01.96	migration/0
10	root	RT	0	0	0	0	S	0.0	0.0	0:01.44	migration/1
11	root	20	0	0	0	0	S	0.0	0.0	0:14.63	ksoftirqd/1
13	root	0	-20	0	0	0	S	0.0	0.0	0:00.00	kworker/1:0H
14	root	20	0	0	0	0	S	0.0	0.0	0:00.00	kdevtmpfs
15	root	0	-20	0	0	0	S	0.0	0.0	0:00.00	netns
18	root	0	-20	0	0	0	S	0.0	0.0	0:00.00	perf
22	root	20	0	0	0	0	S	0.0	0.0	0:00.00	xenwatch
23	root	20	0	0	0	0	S	0.0	0.0	0:00.00	xenbus
134	root	20	0	0	0	0	S	0.0	0.0	0:03.71	khungtaskd
135	root	0	-20	0	0	0	S	0.0	0.0	0:00.00	writeback



Troubleshooting the CPU spike - KEY takeaway

- Whenever a CPU spike happens it only happens because of ***threads***
- If there is a slowdown in a backend system, or there is some sort of block, that means your application will become unresponsive
- But if CPU spikes up it means the threads are going into some sort of loop or execute some code infinitely
- Now, we want to identify the specific threads which are causing the CPU to spike up...
- So, we want to look inside the problematic process a bit more to see the underlying threads - how can we do that?



Troubleshooting the CPU spike - top command

- For this, top has an option like so: **top -H -p {process_id}**
- This will display all the threads running within this process (remember that you can have many threads within a single process)
- And also it will tell us what the CPU and memory consumption of those threads are!



Troubleshooting the CPU spike - top command

- Below we can see the result of running **top -H -p {process_id}**:

Step 2: Identify Threads

 ec2-user@ip-172-31-15-129:/usr/share/tomcat8/logs `top -H -p {pid}`

```
top - 23:14:36 up 102 days, 21:10,  2 users,  load average: 2.97, 3.00, 2.58
Tasks:  15 total,   3 running, 12 sleeping,   0 stopped,   0 zombie
Cpu(s):100.0%us,   0.0%sy,   0.0%ni,   0.0%id,   0.0%wa,   0.0%hi,   0.0%si,   0.0%st
Mem:   8178640k total, 3822428k used, 4356212k free, 158700k buffers
Swap:        0k total,    0k used,    0k free, 646536k cached
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
31306	ec2-user	20	0	4415m	22m	14m	R	69.3	0.3	0:56.53	java
31307	ec2-user	20	0	4415m	22m	14m	R	65.6	0.3	0:56.83	java
31308	ec2-user	20	0	4415m	22m	14m	R	64.0	0.3	0:55.48	java
31294	ec2-user	20	0	4415m	22m	14m	S	0.0	0.3	0:00.00	java
31295	ec2-user	20	0	4415m	22m	14m	S	0.0	0.3	0:00.05	java
31296	ec2-user	20	0	4415m	22m	14m	S	0.0	0.3	0:00.00	java
31297	ec2-user	20	0	4415m	22m	14m	S	0.0	0.3	0:00.00	java
31298	ec2-user	20	0	4415m	22m	14m	S	0.0	0.3	0:00.00	java
31299	ec2-user	20	0	4415m	22m	14m	S	0.0	0.3	0:00.00	java
31300	ec2-user	20	0	4415m	22m	14m	S	0.0	0.3	0:00.00	java
31301	ec2-user	20	0	4415m	22m	14m	S	0.0	0.3	0:00.00	java
31302	ec2-user	20	0	4415m	22m	14m	S	0.0	0.3	0:00.00	java
31303	ec2-user	20	0	4415m	22m	14m	S	0.0	0.3	0:00.00	java
31304	ec2-user	20	0	4415m	22m	14m	S	0.0	0.3	0:00.00	java
31305	ec2-user	20	0	4415m	22m	14m	S	0.0	0.3	0:00.02	java



Troubleshooting the CPU spike - top command

- In the screenshot above, we can see that if we sum up the three threads at the top, it totals to about 199
- All other threads do not consume any CPU
- So, now we have found the culprit threads which cause the CPU spike
- What's the next step?
- Let's understand the lines of code the threads are executing
- How to do that? We need a thread dump!!
- What are some of the options to get a thread dump? Let's explore...

Step 3: Capture thread dumps

<https://blog.fastthread.io/2016/06/06/how-to-take-thread-dumps-7-options/>

01 

jstack (since Java 5)

jstack -l <pid> >
/tmp/threadDump.txt

02 

kill -3

Kill -3 <pid>
Useful when only JRE is
installed

03

jVisualVM

JDK tool. Now Open source.
GUI based option.

04

JMC

JDK tool. Now Open source.
GUI based option.

05

Windows (Ctrl +
Break)
Helpful during development
phase

06

ThreadMXBean

Programmatic way to
capture thread dumps

07

APM Tools

Few APM Tools does provide
this support

08 

Jcmd (since Java 7)

jcmd <pid> Thread.print >
/tmp/threadDump.txt



Troubleshooting the CPU spike -

- Some companies - like financial institutions and government agencies do not install the JDK in production
- Why?
- For security reasons...
- They only install the JRE which is a subset of the JDK and contains just enough tools to run your Java
- So, you won't have tools like jstack available
- In that case you can run option 2 from slide above - **kill -3 <pid>**, this will give you the thread dump



Troubleshooting the CPU spike - **anatomy of a thread dump**

- Some companies - like financial institutions and government agencies do not install the JDK in production
- Why?
- For security reasons...
- They only install the JRE which is a subset of the JDK and contains just enough tools to run your Java
- So, you won't have tools like jstack available
- In that case you can run option 2 from slide above - **kill -3 <pid>**, this will give you the thread dump

Anatomy of thread dump

2019-12-26 17:13:23

Full thread dump Java HotSpot(TM) 64-Bit Server VM (23.7-b01 mixed mode):

"InvoiceThread-A996" prio=10 tid=0x00002b7cfc6fb000 nid=0x4479 runnable [0x00002b7d17ab8000]

java.lang.Thread.State: RUNNABLE

at com.buggycompany.rt.util.ItinerarySegmentProcessor.setConnectingFlight(ItinerarySegmentProcessor.java:380)
at com.buggycompany.rt.util.ItinerarySegmentProcessor.processTripType0(ItinerarySegmentProcessor.java:366)
at com.buggycompany.rt.util.ItinerarySegmentProcessor.processItineraryByTripType(ItinerarySegmentProcessor.java:254)
at com.buggycompany.rt.util.ItinerarySegmentProcessor.templateMethod(ItinerarySegmentProcessor.java:399)
at com.buggycompany.qc.gds.InvoiceGeneratedFacade.readTicketImage(InvoiceGeneratedFacade.java:252)
at com.buggycompany.qc.gds.InvoiceGeneratedFacade.doOrchestrate(InvoiceGeneratedFacade.java:151)
at com.buggycompany.framework.gdstask.BaseGDSFacade.orchestrate(BaseGDSFacade.java:32)
at com.buggycompany.framework.gdstask.BaseGDSFacade.doWork(BaseGDSFacade.java:22)
at com.buggycompany.framework.concurrent.BuggycompanyCallable.call(buggycompanyCallable.java:80)
at java.util.concurrent.FutureTask\$Sync.innerRun(FutureTask.java:334)
at java.util.concurrent.FutureTask.run(FutureTask.java:166)
at java.util.concurrent.ThreadPoolExecutor.runWorker(ThreadPoolExecutor.java:1145)
at java.util.concurrent.ThreadPoolExecutor\$Worker.run(ThreadPoolExecutor.java:615)
at java.lang.Thread.run(Thread.java:722)

"Reconnection-1" prio=10 tid=0x00007f0442e10800 nid=0x112a waiting on condition [0x00007f042f719000]

java.lang.Thread.State: WAITING (parking)

at sun.misc.Unsafe.park(Native Method)

- parking to wait for <0x007b3953a98> (a java.util.concurrent.locks.AbstractQueuedSynchronizer)

at java.util.concurrent.locks.LockSupport.park(LockSupport.java:186)

at java.lang.Thread.run(Thread.java:722)

:
:
:

- 1 Timestamp at which thread dump was triggered
- 2 JVM Version info
- 3 Thread Details - <<details in following slides>>



Troubleshooting the CPU spike - **anatomy of a thread dump**

- 1 == timestamp
- 2 == who is JVM vendor, is it a 32 or 64 bit JVM?
- 3 == a segment of the thread, if you have 300 threads you will see 300 segments like this!
- Each segment will give information about that particular thread
- Let's see more information about this thread!

```

1      2      3      4      5
"InvoiceThread-A996" prio=10 tid=0x00002b7cfc6fb000 nid=0x4479 runnable
6 [0x00002b7d17ab8000]
  java.lang.Thread.State: RUNNABLE
    at com.buggycompany.rt.util.ItinerarySegmentProcessor.setConnectingFlight(ItinerarySegmentProcessor.java:380)
    at com.buggycompany.rt.util.ItinerarySegmentProcessor.processTripType0(ItinerarySegmentProcessor.java:366)
    at com.buggycompany.rt.util.ItinerarySegmentProcessor.processItineraryByTripType(ItinerarySegmentProcessor.java:254)
    at com.buggycompany.rt.util.ItinerarySegmentProcessor.templateMethod(ItinerarySegmentProcessor.java:399)
    7 at com.buggycompany.qc.gds.InvoiceGeneratedFacade.readTicketImage(InvoiceGeneratedFacade.java:252)
    at com.buggycompany.qc.gds.InvoiceGeneratedFacade.doOrchestrate(InvoiceGeneratedFacade.java:151)
    at com.buggycompany.framework.gdstask.BaseGDSFacade.orchestrate(BaseGDSFacade.java:32)
    at com.buggycompany.framework.gdstask.BaseGDSFacade.doWork(BaseGDSFacade.java:22)
    at com.buggycompany.framework.concurrent.BuggycompanyCallable.call(buggycompanyCallable.java:80)
    at java.util.concurrent.FutureTask$Sync.innerRun(FutureTask.java:334)
    at java.util.concurrent.FutureTask.run(FutureTask.java:166)
    at java.util.concurrent.ThreadPoolExecutor.runWorker(ThreadPoolExecutor.java:1145)
    at java.util.concurrent.ThreadPoolExecutor$Worker.run(ThreadPoolExecutor.java:615)
    at java.lang.Thread.run(Thread.java:722)

```

1 Thread Name

InvoiceThread-A996

2 Priority - Can have values

from 1 to 10

3 Thread Id - 0x00002b7cfc6fb000 - Unique ID assigned by JVM. It's returned by calling the Thread.getId() method.

4 Native Id - 0x4479 - This ID is highly platform dependent. On Linux, it's the pid of the thread. On Windows, it's simply the OS-level thread ID within

5 Address space. On Mac OS X, it is said to be the native pthread_t value.

6 0x00002b7d17ab8000 -

7 Thread State -

RUNNABLE

Stack

trace -



Thread segment details

- 1 == name of thread, given by developer or 3rd party
- 2 == priority from 1-10, 10 being highest priority, 1 is lowest
 - If there is contention for CPU, the threads with higher priority get a chance to run first before the threads with a lower priority
 - Who defines this priority? developers
- 3 == the thread ID. JVM assigns an ID for every thread
 - Nothing stops a developer from assigning a name that's the same, but thread ID is always UNIQUE!

Thread segment details

- 4 == native ID, the ID assigned by the OS/Kernel for this thread
 - So, if you are using some OS tools they will use this ID and you would want to marry this data with what those tools show you
 - Earlier, we saw “top” was returning a PID - that PID is the native ID!
 - So, in the thread dump we want to look for the PID under Native ID (nid) - ie PID from top == nid in thread dump
- 5 == address space in which thread is running
 - Not typically used for practical purposes
- 6 == thread state - 6 different states (see next slide)
- 7 == stack trace - read it from the bottom up
 - Basically this is like a stack frame



Thread segment details

- The thread dump is relatively simple.
- But from the thread dump, we can get some amazing insights!
- Let's talk about thread states...

6 thread states

01 NEW

03 RUNNABLE

05 WAITING

`wait();`

02 TERMINATED

04 BLOCKED

Thread 2: Blocked

```
public void synchronized getData() {  
    makeDBCall();  
}
```

Thread 1: Runnable

06 TIMED_WAITING

`Thread.sleep(10);`

Thread states

- **New** == object just created
 - Seldom you would see a thread in this state in the thread dump
 - Very rare - and if you see it, it's not much help since it was just created
- **Terminated** state = momentary, right before it's terminated
 - Rare as well - you likely will never see a thread in this state in your dump
- Both **new** and **terminated** states are something you don't really have to worry about!
- The remaining 4 - the thread is either running or stuck..

Thread states

- **Runnable** == if the thread is able to progress and execute some lines of code...
 - Whenever a CPU spike happens it only happens because of runnable state threads - so you want to focus on threads in this state
 - Runnable means its not stuck anywhere
- Remaining 3 states mean that the thread is basically stuck - **Blocked, Waiting, Timed_Waiting**
- Effectively, a thread is either running (Runnable) or stuck

Blocked state

- **Blocked state**

- Synchronized keyword is applied to a method - what's the purpose?
- Only one thread at a time will be allowed to enter and execute this method (used as a “lock” to prevent data corruption typically)
- When a thread comes in to execute this method it's in a Runnable state
- Then, the JVM will only allow one thread to come into this method at a time and it will allow that thread to execute if no other thread is there
- Then it will execute the makeDBCall()
- If another thread comes in at that same time what will happen?

Blocked state

- **Blocked state**
 - Another thread will NOT be allowed to enter
 - So, thread 2 will enter the **BLOCKED** state
 - If even another thread (thread 3) comes in and tries to enter that method while thread 1 is still executing it will also enter the **BLOCKED** state as well
 - If the method takes 1 minute to execute other customer transactions will be stuck waiting
 - Like standing in line at a coffee shop where there's only 1 cashier...

Waiting state

- **Waiting state**
 - There are certain APIs in the Java programming language
 - Developer can intentionally write code like `wait()` in order to intentionally wait to do some work (maybe b/c waiting for user to input data, or for a resource like a file to be available)
 - Maybe the developer wants to wait for the next request to come
 - Again, this is intentional - the programmer does not want the thread to progress
- **Timed_waiting state** - the thread will sleep for a set amount of time
- In **BLOCKED**, **WAITING**, and **TIMED_WAITING** states the thread is not doing any work
- The difference is that in **waiting** and **timed_waiting** the developer entered those states deliberately

Step 4: Identify lines of code causing CPU spike



Thread Ids: 31306, 31307, 31308

High CPU consuming Threads Ids reported in 'top -H'.



Let's look up these thread Ids in Thread dump

HexaDecimal equivalent:

- 31306 □ 7a4a
- 31307 □ 7a4b
- 31308 □ 7a4c

Thread block

- If you recall, we have to match the thread ID returned from **top** with the **nid** returned from the ***thread dump***
- While certainly doable, this can be tedious
- Can we utilize tools to do this for us? Certainly!
- **The line of code that tells us where the problem is here (from the stack trace):**

at

com.buggycompany.rt.util.ItinerarySegmentProcessor.setConnectingFlight
(ItinerarySegmentProcessor.java:380)



How often should we take thread dumps?

- As a rule of thumb, you want to take 3 snapshots of thread dumps in a gap of 10 seconds
- <https://blog.fastthread.io/2021/08/17/overhead-added-by-collecting-thread-dumps/>
- The more frequently you take snapshots the more overhead your application will have

'Free' Thread dump analysis tools

Freely available Thread dump analysis tools

01

FastThread

<http://fastThread.io/>

02

Samurai

<http://samuraism.jp/samurai/en/index.html>

03

IBM Thread & Monitor analyzer

<https://developer.ibm.com/javasdk/tools/>

04

Visual VM

<https://visualvm.github.io/>



Troubleshooting CPU spike

Major Trading
application

Analysis Report: <https://tinyurl.com/wzs8kpb>



Troubleshooting unresponsive app

Travel App processes 70% N. America overseas
~~booking~~

Analysis Report:

<https://tinyurl.com/wq95weo>



TrafficJam Pattern

8 types - OutOfMemoryError

<https://blog.gceasy.io/2015/09/25/outofmemoryerror-beautiful-1-page-document/>

```
java.lang.OutOfMemoryError:  
<type>
```

01 Java heap space

02 GC overhead limit exceeded

03 Requested array size exceed VM limit

04 Permgen space

05 Metaspace

06 Unable to create new native thread

07 Kill process or sacrifice child

08 reason stack_trace_with_native method

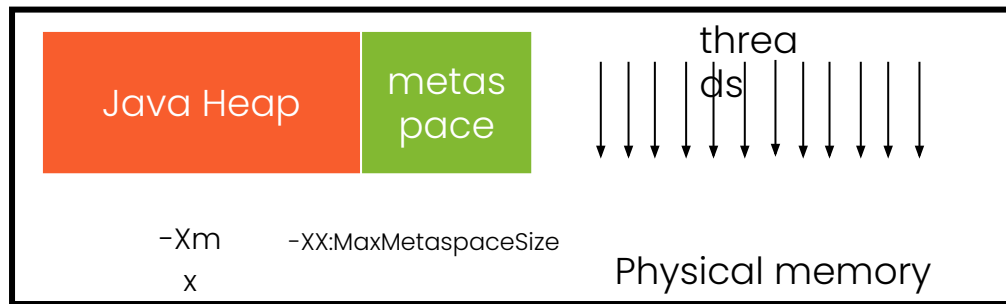


Troubleshooting OOMError: Unable to create new native thread

One of world's largest middleware
app

Analysis Report: <http://fastthread.io/my-thread-report.jsp?p=c2hhcmVklZlwMTcvMDMvMTQvLSl0aHJlYWREdWIwLTludHh0LS0xMi0yOC0zMw==&s=t>

OOM: Unable to create new native thread

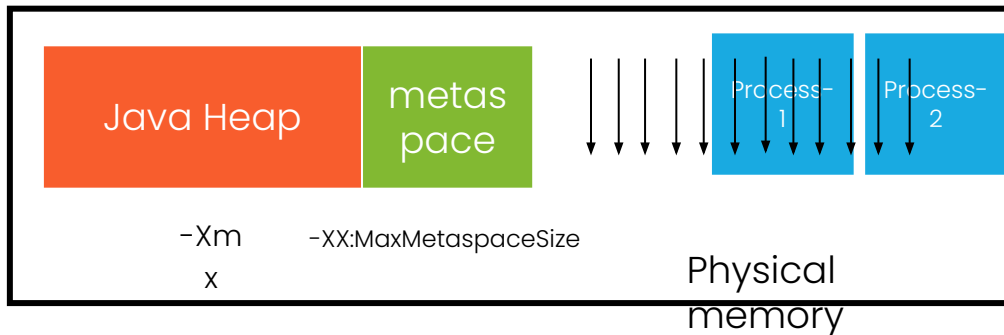


Key: Threads are created **outside heap, metaspace**

Solution:

1. Fix thread leak
2. Increase the Thread Limits Set at Operating System(`ulimit -u`)
3. Reduce Java Heap Size
4. Kills other processes
5. Increase physical memory size
6. Reduce thread stack size (`-Xss`).

Note: can cause





Troubleshooting Microservices/Big data app

Major Financial institution in N.
America

Analysis Report:

<https://tinyurl.com/yywdmvyy>



Same RSI Pattern



Deadlock

Open-Source apache
library

Analysis Report: Deadlock in Apache pdfbox library - yCrash
Answers



Deadlock Pattern



Analysis Report: <https://fastthread.io/my-thread-report.jsp?p=c2hhcmVklzlwMjVMDcvMzEvdGhyZWfkX2thc3RsZV8yNjA3MjIudHh0LS03LTmWLTmZLS0xNi0zMjY0Zng==&s=t>



All roads lead to Rome Pattern



HTTP 502 in AWS – EBS

Analysis Report: [Troubleshooting HTTP 502 bad gateway in AWS EBS – yCrash](#)



Kernel Logs

Thread dump analysis Patterns

<https://blog.fastthread.io/category/thread-dump-patterns/>



RSI Pattern



Treadmill Pattern



Leprechaun Pattern



All Roads leads to
Rome Pattern



Traffic Jam Pattern



Athlete Pattern



Atherosclerosis Pattern



Several Scavengers Pattern



few more ...



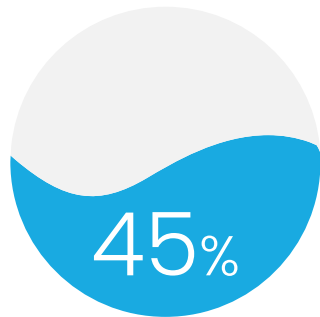
Micro-metrics

Let's Forecast
problems

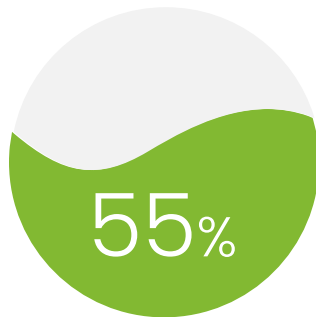
<https://blog.gceasy.io/2019/03/13/micrometrics-to-forecast-application-performance/>

Macro-Metrics

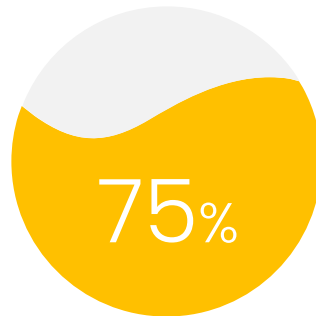
Can't forecast scalability, availability, performance problems



CPU



MEMORY



RESPONSE TIME



GC Logs – key micrometrics

Till Java 8:

-XX:+PrintGCDetails -XX:+PrintGCDateStamps
-Xloggc:<file-path>

From Java 9:

-Xlog:gc*:file=<file-path>

'Free' GC Log analysis tools

Freely available Garbage collection log analysis tools

01

GCeasy

<http://gceasy.io/>

02

GC Viewer

<https://github.com/chewiebug/GCViewer>

03

IBM GC & Memory visualizer

<https://developer.ibm.com/javasdk/tools/>

04

HP Jmeter

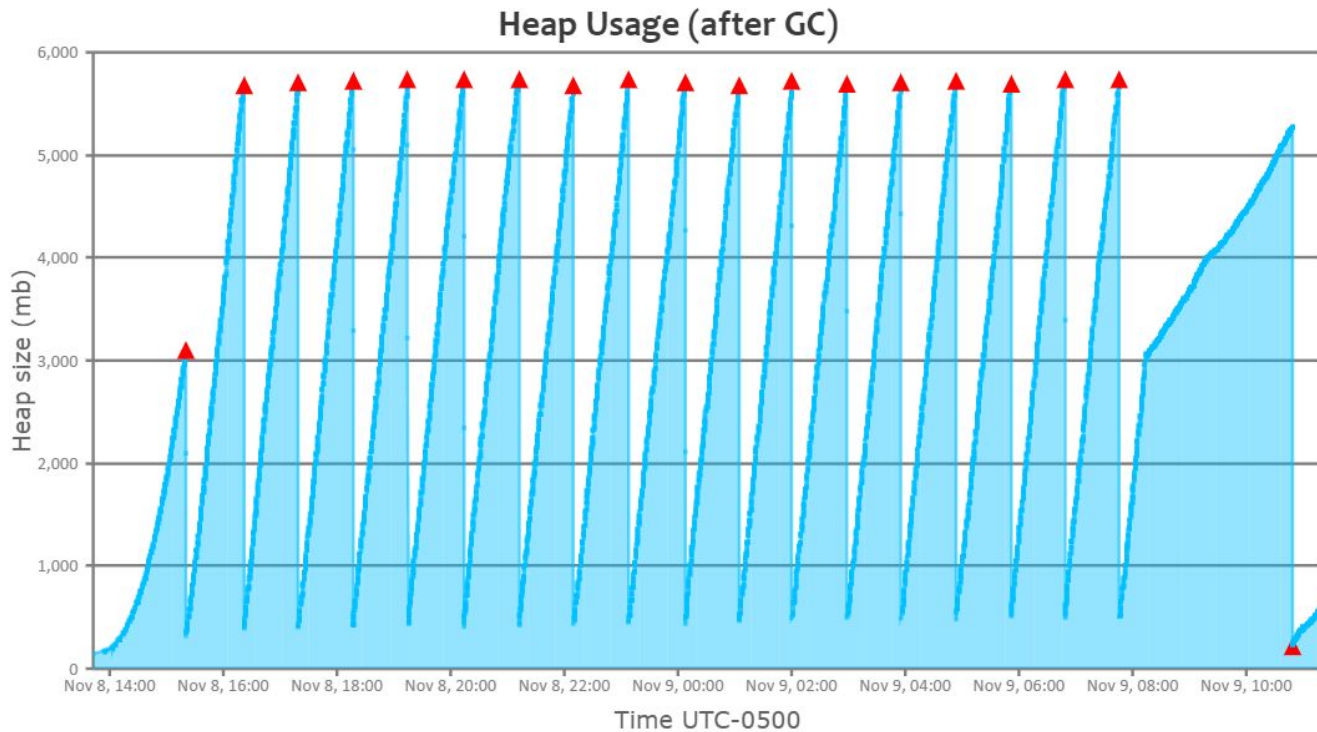
<https://h20392.www2.hp.com/portal/swdepot/displayProductInfo.do?productNumber=HPJMeter>

05

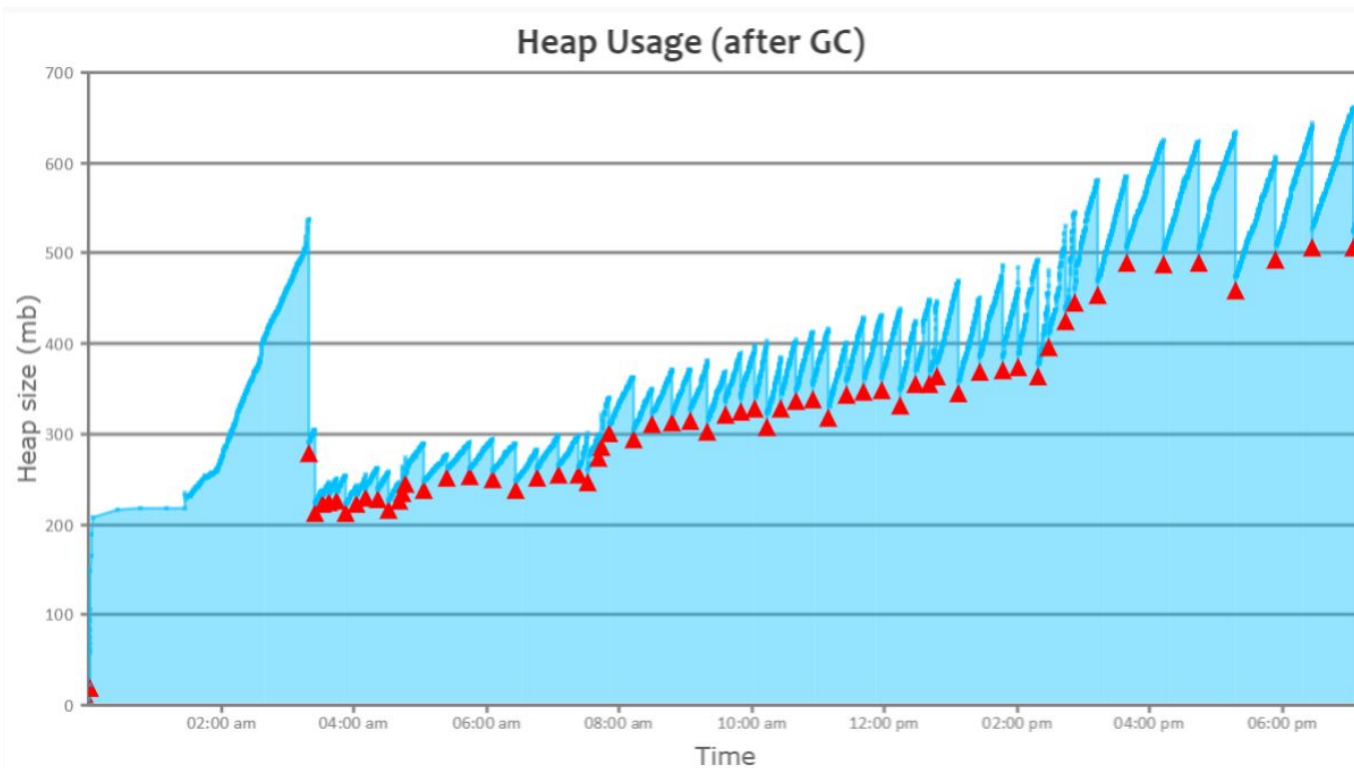
Google Garbage cat (cms)

<https://code.google.com/archive/a/eclipse-labs.org/project/garbagecat>

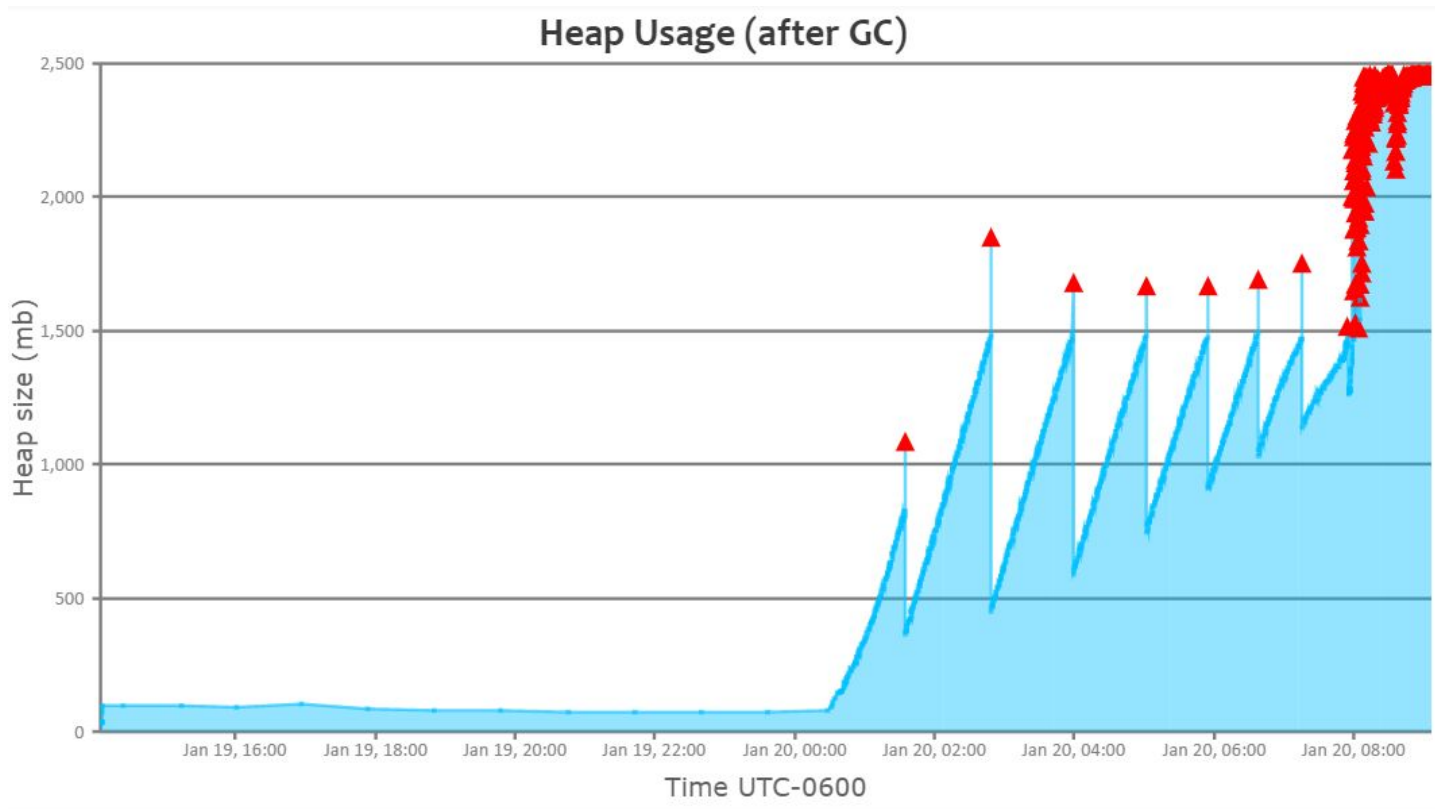
Heap usage graph



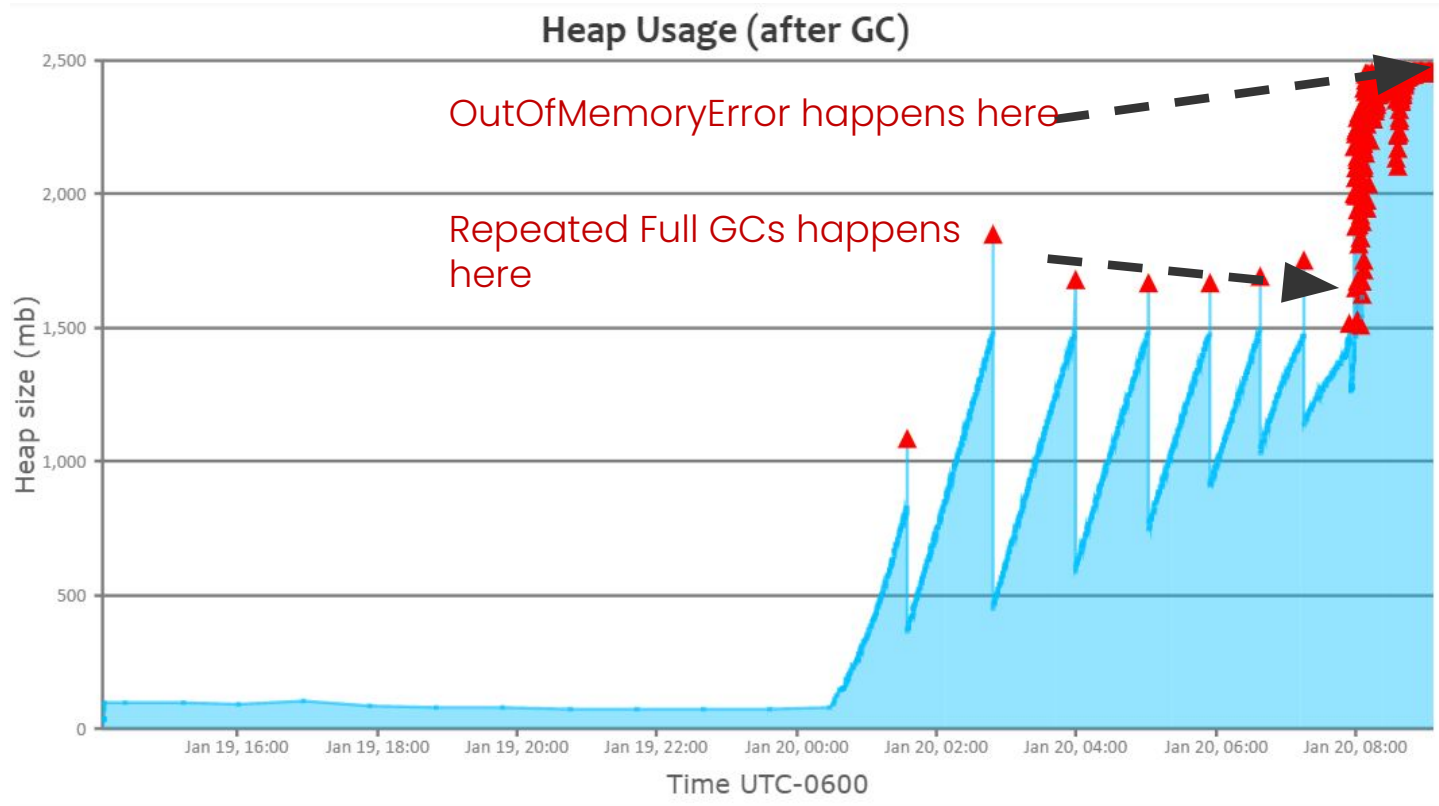
What is your observation?



Memory Problem



Micro-metric: GC Throughput



More Micrometrics

<https://blog.gceasy.io/2019/03/13/micrometrics-to-forecast-application-performance/>

GC Throughput

Amount time application spends in processing customer transactions vs amount of time application spend in doing GC

GC Latency

If pause time starts to increase, then it's an indication that app is suffering from memory problems

Object Reclamation rate

If number of objects created in unit time

File Descriptors

File descriptor is a handle to access: File, Pipe, Network Connections. If count grows it's a lead indicator that application isn't closing resources properly.

Thread States

If BLOCKED thread state count grows, it's an early indication that your application has potential to become unresponsive

Few more...

TCP/IP States, Hosts count, IOPS, ..



Utilizing IntelliJ profiler - Lab



Profiling lab

- Execute this lab here:
 - <https://blog.jetbrains.com/idea/2022/01/fixing-the-parrot-party/>
 -

A thick yellow L-shaped line is positioned in the top-left corner. Three concentric circles are centered on the slide, with the text 'Thank you!' placed within the innermost circle.

Thank you!

